

Build Playbook

Create the files in the order given. Run the commands where they appear. Open a PR where marked. Every file is reproduced in full, so a step is done when the file matches.

`JackpotKit` is one package; each folder under `Sources/` is a module, and later PRs append targets to the same manifest. The whole suite runs in a simulator via `xcodebuild -scheme JackpotKit-Package` — there is no macOS destination, because `JackpotUI` imports `UIKit`.

152 tests through PR 5. The follow-up section takes it to 181.

Contents

- PR 1 — JackpotUI 3**
 - ▶ Create PR — JackpotUI 24
- PR 2 — JackpotForms 25**
 - ▶ Create PR — JackpotForms 65
- PR 3 — JackpotNetworking + the live repository 66**
 - ▶ Create PR — JackpotNetworking + live repository 91
- PR 4 — JackpotRegistration 92**
 - ▶ Create PR — JackpotRegistration 97
- PR 5 — Replace the current flow 98**
 - ▶ Create PR — Replace registration flow 99
- Follow-up — app-data and the translations store 100**

PR 1 — JackpotUI

The design system, with no knowledge of forms. Components take a value and a binding; the theme and the per-field configuration travel through the environment, so no component carries styling parameters.

1

```
mkdir -p JackpotKit/Sources/JackpotUI/{Theme,Styles,Fields,Components,Preview}
mkdir -p JackpotKit/Tests/JackpotUITests
cd JackpotKit
printf '.build/\n.swiftpm/\n*.xcuserdatad\n' > .gitignore
```

2 JackpotKit/Package.swift

```
// swift-tools-version: 5.7
import PackageDescription

let package = Package(
    name: "JackpotKit",
    defaultLocalization: "en",
    platforms: [.iOS(.v15)],
    products: [
        .library(name: "JackpotUI", targets: ["JackpotUI"]),
    ],
    targets: [
        .target(name: "JackpotUI"),
        .testTarget(name: "JackpotUITests", dependencies: ["JackpotUI"]),
    ]
)
```

3 JackpotKit/Sources/JackpotUI/Theme/JackpotTheme.swift

```
import SwiftUI
import UIKit

public struct JackpotTheme: Equatable, Sendable {
    public var colors: JackpotColors = .jackpotCity
    public var metrics: JackpotMetrics = .standard
    public var typography: JackpotTypography = .standard

    public static let jackpotCity = JackpotTheme()

    public func with(_ transform: (inout JackpotTheme) -> Void) -> JackpotTheme {
        var copy = self
        transform(&copy)
        return copy
    }
}

// MARK: - Colours

public struct JackpotColors: Equatable, Sendable {
    public var surface = Palette.surface

    public var fieldBackground = Palette.fieldBackground
    public var fieldBorder = Palette.fieldBorder
    public var fieldBorderFocused = Palette.accent
    public var fieldBorderInvalid = Palette.error

    public var textPrimary = Palette.textPrimary
    public var textSecondary = Palette.textSecondary
    public var textOnAccent = Palette.textOnAccent

    public var accent = Palette.accent
    public var accentFill = Palette.accentFill

    public var error = Palette.error
    public var warning = Palette.warning
    public var success = Palette.success
```

```

    public static let jackpotCity = JackpotColors()
}

/// Held as shared constants rather than inline literals so two separately built `JackpotColors`
/// still compare equal – a dynamic `Color` compares by identity.
enum Palette {
    static let surface = Color.adaptive(light: Color(red: 1.00, green: 1.00, blue: 1.00),
                                         dark: Color(red: 0.07, green: 0.08, blue: 0.09))

    /// #F7F8FA. A tint off the surface, not white on white – the fill is what identifies a
    /// field, a secondary button and a checklist panel.
    static let fieldBackground = Color.adaptive(light: Color(red: 0.969, green: 0.973, blue: 0.980),
                                                dark: Color(red: 0.13, green: 0.14, blue: 0.16))

    /// #E4E5EA. A hairline by design: the fill carries identification, and the focused and
    /// invalid borders carry state.
    static let fieldBorder = Color.adaptive(light: Color(red: 0.894, green: 0.898, blue: 0.918),
                                            dark: Color.white.opacity(0.32))

    /// #22252C.
    static let textPrimary = Color.adaptive(light: Color(red: 0.133, green: 0.145, blue: 0.173),
                                            dark: .white)

    /// #565A63. Measured against `fieldBackground`, the tighter of its two backgrounds.
    static let textSecondary = Color.adaptive(light: Color(red: 0.337, green: 0.353, blue: 0.388),
                                              dark: Color.white.opacity(0.6))

    /// Sits on the accent and action fills, never on the surface, so it does not invert.
    static let textOnAccent = Color.white

    /// Accent as *text* – the tertiary button label. It cannot share a value with `accentFill`:
    /// white on a fill needs the fill at or below 0.18 luminance, while a label on the dark
    /// surface needs 0.20 up.
    static let accent = Color.adaptive(light: Color(red: 0.000, green: 0.376, blue: 0.925),
                                       dark: Color(red: 0.302, green: 0.561, blue: 1.000))

    /// #0060EC. Carries `textOnAccent` at 5.4:1, which is why the same brand blue serves as a
    /// fill in both appearances.
    static let accentFill = Color(red: 0.000, green: 0.376, blue: 0.925)

    /// #BC1A1A.
    static let error = Color.adaptive(light: Color(red: 0.737, green: 0.102, blue: 0.102),
                                       dark: Color(red: 0.98, green: 0.45, blue: 0.35))

    static let warning = Color.adaptive(light: Color(red: 0.58, green: 0.36, blue: 0.02),
                                        dark: Color(red: 0.96, green: 0.62, blue: 0.13))

    static let success = Color.adaptive(light: Color(red: 0.06, green: 0.46, blue: 0.26),
                                        dark: Color(red: 0.20, green: 0.72, blue: 0.44))
}

extension Color {
    /// Resolves against the trait collection, so one palette serves both appearances and
    /// honours a `preferredColorScheme` override anywhere in the hierarchy.
    static func adaptive(light: Color, dark: Color) -> Color {
        let light = UIColor(light)
        let dark = UIColor(dark)
        return Color(UIColor { $0.userInterfaceStyle == .dark ? dark : light })
    }
}

// MARK: - Metrics

public struct JackpotMetrics: Equatable, Sendable {
    public var cornerRadius: CGFloat = 10
    public var controlHeight: CGFloat = 52
    public var spacing: CGFloat = 12
    public var contentPadding: CGFloat = 14
    public var borderWidth: CGFloat = 1
    public var emphasizedBorderWidth: CGFloat = 2
    public var minimumHitTarget: CGFloat = 44
    public var progressBarHeight: CGFloat = 4
    public var textAreaMinHeight: CGFloat = 110
    public var cardMinHeight: CGFloat = 140

    public static let standard = JackpotMetrics()

    public var fieldShape: RoundedRectangle {
        RoundedRectangle(cornerRadius: cornerRadius, style: .continuous)
    }
}

```

```

}

// MARK: - Typography

public struct JackpotTypography: Equatable, Sendable {
    public var fieldText = Font.body
    public var label = Font.footnote
    public var error = Font.caption
    public var rowLabel = Font.subheadline
    public var sectionTitle = Font.subheadline.weight(.semibold)
    public var button = Font.headline

    public static let standard = JackpotTypography()
}

```

4 JackpotKit/Sources/JackpotUI/Theme/JackpotEnvironment.swift

`JackpotFieldConfiguration` is what keeps the field views parameter-free: a caller sets `jackpotFieldState` / `jackpotFieldError` once and every field below reads it.

```

import SwiftUI

extension EnvironmentValues {
    @Entry public var jackpotTheme: JackpotTheme = .jackpotCity
    @Entry public var jackpotValidationMessage: String? = nil
    @Entry public var jackpotFieldLabel: String? = nil
    @Entry public var jackpotFieldPrefix: String = ""
    @Entry public var jackpotFieldSuffix: String = ""
    @Entry public var jackpotSecureEntry: Bool = false
    @Entry public var jackpotIsLoading: Bool = false
    @Entry public var jackpotIsSelected: Bool = false
    @Entry public var jackpotFocusedField: Binding<String?>? = nil
    @Entry public var jackpotFieldIdentity: String? = nil
    @Entry public var jackpotSubmitLabel: SubmitLabel = .return
}

// MARK: - Theme

public extension View {
    /// Sets `tint` alongside the theme, so system controls (pickers, toggles) inherit the brand
    /// accent without every call site remembering to.
    func jackpotTheme(_ theme: JackpotTheme) -> some View {
        environment(\.jackpotTheme, theme).tint(theme.colors.accent)
    }
}

// MARK: - Field configuration

public extension View {
    func jackpotValidationMessage(_ message: String?) -> some View {
        environment(\.jackpotValidationMessage, message?.isEmpty == false ? message : nil)
    }

    func jackpotFieldPrefix(_ text: String) -> some View {
        environment(\.jackpotFieldPrefix, text)
    }

    func jackpotFieldSuffix(_ text: String) -> some View {
        environment(\.jackpotFieldSuffix, text)
    }

    func jackpotSecureEntry(_ isEnabled: Bool = true) -> some View {
        environment(\.jackpotSecureEntry, isEnabled)
    }
}

// MARK: - Control state

public extension View {
    /// Disables as well as spins: a button that spins but still fires is a double submit.
    func jackpotLoading(_ isLoading: Bool = true) -> some View {
        environment(\.jackpotIsLoading, isLoading).disabled(isLoading)
    }

    func jackpotSelected(_ isSelected: Bool = true) -> some View {
        environment(\.jackpotIsSelected, isSelected)
    }
}

```

```

    }
}

// MARK: – Keyboard focus

public extension View {
    /// Shares one "which field is focused" value across a group of fields so the return key
    /// can walk them. Tag each field with `jackpotFieldIdentity(_:)`.
    func jackpotFocusedField(_ binding: Binding<String?>) -> some View {
        environment(\.jackpotFocusedField, binding)
    }

    func jackpotFieldIdentity(_ identity: String) -> some View {
        environment(\.jackpotFieldIdentity, identity)
    }

    func jackpotSubmitLabel(_ label: SubmitLabel) -> some View {
        environment(\.jackpotSubmitLabel, label)
    }
}

```

5 JackpotKit/Sources/JackpotUI/Theme/JackpotStyling.swift

```

import SwiftUI

public extension View {
    func jackpotForegroundColor(_ color: KeyPath<JackpotColors, Color>) -> some View {
        modifier(JackpotForegroundColor(color: color))
    }

    func jackpotBackground(_ color: KeyPath<JackpotColors, Color>) -> some View {
        modifier(JackpotBackgroundStyle(color: color, shape: Rectangle()))
    }

    func jackpotBackground<S: Shape>(_ color: KeyPath<JackpotColors, Color>, in shape: S) -> some View {
        modifier(JackpotBackgroundStyle(color: color, shape: shape))
    }

    func jackpotFont(_ font: KeyPath<JackpotTypography, Font>) -> some View {
        modifier(JackpotFontStyle(font: font))
    }

    func jackpotTextStyle(_ font: KeyPath<JackpotTypography, Font>,
        color: KeyPath<JackpotColors, Color> = \.textPrimary) -> some View {
        jackpotFont(font).jackpotForegroundColor(color)
    }
}

private struct JackpotForegroundColor: ViewModifier {
    let color: KeyPath<JackpotColors, Color>
    @Environment(\.jackpotTheme) private var theme

    func body(content: Content) -> some View {
        content.foregroundColor(theme.colors[keyPath: color])
    }
}

private struct JackpotBackgroundStyle<S: Shape>: ViewModifier {
    let color: KeyPath<JackpotColors, Color>
    let shape: S
    @Environment(\.jackpotTheme) private var theme

    func body(content: Content) -> some View {
        content.background(theme.colors[keyPath: color], in: shape)
    }
}

private struct JackpotFontStyle: ViewModifier {
    let font: KeyPath<JackpotTypography, Font>
    @Environment(\.jackpotTheme) private var theme

    func body(content: Content) -> some View {
        content.font(theme.typography[keyPath: font])
    }
}

```

6

JackpotKit/Sources/JackpotUI/Styles/JackpotButtonStyle.swift

```

import SwiftUI

public struct JackpotButtonStyle: ButtonStyle {
    public enum Prominence: Hashable, Sendable {
        case primary
        case secondary
        case tertiary
    }

    private let prominence: Prominence

    public init(_ prominence: Prominence = .primary) {
        self.prominence = prominence
    }

    public func makeBody(configuration: Configuration) -> some View {
        // A ButtonStyle is not a View, so @Environment on the style itself never updates.
        ButtonBody(prominence: prominence, configuration: configuration)
    }

    private struct ButtonBody: View {
        let prominence: Prominence
        let configuration: ButtonStyleConfiguration

        @Environment(\.jackpotTheme) private var theme
        @Environment(\.isEnabled) private var isEnabled
        @Environment(\.jackpotIsLoading) private var isLoading

        var body: some View {
            configuration.label
                .jackpotFont(\.button)
                .opacity(isLoading ? 0 : 1)
                .overlay {
                    if isLoading {
                        ProgressView().tint(foreground).accessibilityHidden(true)
                    }
                }
                .frame(maxWidth: .infinity, minHeight: theme.metrics.controlHeight)
                .foregroundColor(foreground)
                .background(background, in: theme.metrics.fieldShape)
                .contentShape(theme.metrics.fieldShape)
                .opacity(configuration.isPressed ? 0.85 : 1)
                .scaleEffect(configuration.isPressed ? 0.98 : 1)
                .animation(.spring(response: 0.3, dampingFraction: 1), value: configuration.isPressed)
                .accessibilityValue(isLoading ? Text("Loading") : Text(verbatim: ""))
        }

        /// `jackpotLoading(_)` disables the button, but mid-submit it should still look live.
        private var isDimmed: Bool { !isEnabled && !isLoading }

        private var background: Color {
            switch prominence {
            case .primary: return isDimmed ? theme.colors.fieldBackground : theme.colors.accentFill
            case .secondary: return theme.colors.fieldBackground
            case .tertiary: return .clear
            }
        }

        private var foreground: Color {
            switch prominence {
            case .primary: return isDimmed ? theme.colors.textSecondary : theme.colors.textOnAccent
            case .secondary: return isDimmed ? theme.colors.textSecondary : theme.colors.textPrimary
            case .tertiary: return isDimmed ? theme.colors.textSecondary : theme.colors.accent
            }
        }
    }
}

public extension ButtonStyle where Self == JackpotButtonStyle {
    static var jackpot: JackpotButtonStyle { JackpotButtonStyle(.primary) }

    static func jackpot(_ prominence: JackpotButtonStyle.Prominence) -> JackpotButtonStyle {
        JackpotButtonStyle(prominence)
    }
}

// MARK: - Selectable card

```

```

public struct JackpotCardButtonStyle: ButtonStyle {
    public init() {}

    public func makeBody(configuration: Configuration) -> some View {
        CardBody(configuration: configuration)
    }

    private struct CardBody: View {
        let configuration: ButtonStyleConfiguration

        @Environment(\.jackpotTheme) private var theme
        @Environment(\.isEnabled) private var isEnabled
        @Environment(\.jackpotIsSelected) private var isSelected

        var body: some View {
            configuration.label
                .frame(maxWidth: .infinity, minHeight: theme.metrics.cardMinHeight)
                .jackpotBackground(\.fieldBackground, in: theme.metrics.fieldShape)
                .overlay {
                    theme.metrics.fieldShape
                        .strokeBorder(isSelected ? theme.colors.accent : theme.colors.fieldBorder,
                                    lineWidth: isSelected ? theme.metrics.emphasizedBorderWidth
                                    : theme.metrics.borderWidth)
                }
                .contentShape(theme.metrics.fieldShape)
                .opacity(isEnabled ? 1 : 0.6)
                .scaleEffect(configuration.isPressed ? 0.98 : 1)
                .animation(.spring(response: 0.3, dampingFraction: 1), value: configuration.isPressed)
                .animation(.easeOut(duration: 0.2), value: isSelected)
                .accessibilityAddTraits(isSelected ? [.isButton, .isSelected] : .isButton)
        }
    }
}

public extension ButtonStyle where Self == JackpotCardButtonStyle {
    static var jackpotCard: JackpotCardButtonStyle { JackpotCardButtonStyle() }
}

```

7 JackpotKit/Sources/JackpotUI/Styles/JackpotToggleStyle.swift

```

import SwiftUI

public struct JackpotToggleStyle: ToggleStyle {
    public enum Appearance: Hashable, Sendable {
        case checkbox
        case `switch`
    }

    private let appearance: Appearance

    public init(_ appearance: Appearance = .checkbox) {
        self.appearance = appearance
    }

    public func makeBody(configuration: Configuration) -> some View {
        ToggleBody(appearance: appearance, configuration: configuration)
    }

    private struct ToggleBody: View {
        let appearance: Appearance
        let configuration: ToggleStyleConfiguration

        @Environment(\.jackpotTheme) private var theme
        @Environment(\.jackpotValidationMessage) private var validationMessage

        var body: some View {
            switch appearance {
            case .checkbox: checkbox
            case .switch: platformSwitch
            }
        }

        private var checkbox: some View {
            Button {
                configuration.isOn.toggle()
            } label: {

```



```

        HStack(alignment: .top, spacing: theme.metrics.spacing) {
            Image(systemName: configuration.isOn ? "checkmark.square.fill" : "square")
                .font(.title3)
                .foregroundColor(boxColor)
                .frame(width: 24, height: 24)
                .animation(.easeOut(duration: 0.15), value: configuration.isOn)
            configuration.label
                .jackpotTextStyle(\.rowLabel)
                .multilineTextAlignment(.leading)
                .fixedSize(horizontal: false, vertical: true)
            Spacer(minLength: 0)
        }
        .frame(minHeight: theme.metrics.minimumHitTarget)
    }
    .buttonStyle(.plain)
    .accessibilityRepresentation {
        // The explicit style stops the stand-in resolving back to this one.
        Toggle(isOn: configuration.$isOn) { configuration.label }
            .toggleStyle(.switch)
    }
}

private var platformSwitch: some View {
    Toggle(configuration)
        .toggleStyle(.switch)
        .jackpotTextStyle(\.rowLabel)
}

private var boxColor: Color {
    if configuration.isOn { return theme.colors.accent }
    return validationMessage == nil ? theme.colors.textSecondary : theme.colors.fieldBorderInvalid
}

}

public extension ToggleStyle where Self == JackpotToggleStyle {
    static var jackpotCheckbox: JackpotToggleStyle { JackpotToggleStyle(.checkbox) }
    static var jackpotSwitch: JackpotToggleStyle { JackpotToggleStyle(.switch) }
}

```

8 JackpotKit/Sources/JackpotUI/Styles/JackpotProgressViewStyle.swift

```

import SwiftUI

public struct JackpotBarProgressViewStyle: ProgressViewStyle {
    private let height: CGFloat?

    public init(height: CGFloat? = nil) {
        self.height = height
    }

    public func makeBody(configuration: Configuration) -> some View {
        BarBody(height: height, fraction: configuration.fractionCompleted ?? 0)
    }

    private struct BarBody: View {
        let height: CGFloat?
        let fraction: Double

        @Environment(\.jackpotTheme) private var theme

        var body: some View {
            GeometryReader { proxy in
                ZStack(alignment: .leading) {
                    Capsule().fill(theme.colors.fieldBorder)
                    Capsule()
                        .fill(.tint)
                        .frame(width: proxy.size.width * fraction.clampedToUnitInterval)
                }
            }
            .frame(height: height ?? theme.metrics.progressBarHeight)
            .animation(.easeOut(duration: 0.25), value: fraction)
        }
    }
}

public extension ProgressViewStyle where Self == JackpotBarProgressViewStyle {

```

```

static var jackpotBar: JackpotBarProgressViewStyle { JackpotBarProgressViewStyle() }

static func jackpotBar(height: CGFloat) -> JackpotBarProgressViewStyle {
    JackpotBarProgressViewStyle(height: height)
}

private extension Double {
    var clampedToUnitInterval: Double { min(max(self, 0), 1) }
}

```

9 JackpotKit/Sources/JackpotUI/Fields/JackpotFieldChrome.swift

The shared background and the label/error row every field sits in.

```

import SwiftUI

public struct JackpotFieldBackground: ViewModifier {
    private let isFocused: Bool

    @Environment(\.jackpotTheme) private var theme
    @Environment(\.jackpotValidationMessage) private var validationMessage
    @Environment(\.isEnabled) private var isEnabled

    public init(isFocused: Bool = false) {
        self.isFocused = isFocused
    }

    public func body(content: Content) -> some View {
        content
            .jackpotBackground(\.fieldBackground, in: theme.metrics.fieldShape)
            .overlay {
                theme.metrics.fieldShape
                    .strokeBorder(borderColor, lineWidth: borderWidth)
                    .animation(.easeOut(duration: 0.15), value: emphasis)
            }
            .opacity(isEnabled ? 1 : 0.6)
    }

    private enum Emphasis: Equatable { case none, focused, invalid }

    private var emphasis: Emphasis {
        if validationMessage != nil { return .invalid }
        return isFocused ? .focused : .none
    }

    private var borderColor: Color {
        switch emphasis {
        case .invalid: return theme.colors.fieldBorderInvalid
        case .focused: return theme.colors.fieldBorderFocused
        case .none: return theme.colors.fieldBorder
        }
    }

    /// Width as well as colour, so focus and errors are not carried by colour alone.
    private var borderWidth: CGFloat {
        emphasis == .none ? theme.metrics.borderWidth : theme.metrics.emphasizedBorderWidth
    }
}

public extension View {
    func jackpotFieldBackground(isFocused: Bool = false) -> some View {
        modifier(JackpotFieldBackground(isFocused: isFocused))
    }
}

// MARK: - Form row

public struct JackpotLabeledField<Content: View>: View {
    private let label: String?
    private let error: String?
    private let content: Content

    public init(_ label: String? = nil, error: String? = nil, @ViewBuilder content: () -> Content) {
        self.label = label
        self.error = error
        self.content = content()
    }
}

```

```

}

public var body: some View {
    VStack(alignment: .leading, spacing: 6) {
        if let label, !label.isEmpty {
            Text(label).jackpotTextStyle(\.label, color: \.textSecondary)
        }

        content
            .jackpotValidationMessage(error)
            .environment(\.jackpotFieldLabel, label)

        if let error, !error.isEmpty {
            Text(error)
                .jackpotTextStyle(\.error, color: \.error)
                .fixedSize(horizontal: false, vertical: true)
                .accessibilityLabel("Error: \(error)")
        }
    }
    .animation(.easeOut(duration: 0.2), value: error)
}
}

// MARK: - Divider

public struct JackpotDivider: View {
    @Environment(\.jackpotTheme) private var theme

    public init() {}

    public var body: some View {
        Rectangle()
            .fill(theme.colors.fieldBorder)
            .frame(height: 1)
            .padding(.vertical, 4)
            .accessibilityHidden(true)
    }
}

```

10 JackpotKit/Sources/JackpotUI/Fields/JackpotFieldKind.swift

One value per input type the registration schema asks for, applied with `jackpotField(_)`.

```

import SwiftUI

/// `TextInputAutocapitalization` is neither `Equatable` nor inspectable, so the kind stores its
/// own case and converts when applying.
public enum JackpotCapitalization: Equatable, Sendable {
    case never, words, sentences, characters

    var textInput: TextInputAutocapitalization {
        switch self {
        case .never:      return .never
        case .words:      return .words
        case .sentences:  return .sentences
        case .characters: return .characters
        }
    }
}

/// Keyboard, autofill and autocorrection settings applied as one unit, so a field's semantics
/// are declared once instead of as four loose modifiers per call site.
public struct JackpotFieldKind: Equatable, Sendable {
    public var keyboard: UIKeyboardType = .default
    public var contentType: UITextContentType?
    public var capitalization: JackpotCapitalization = .sentences
    public var disablesAutocorrection = false
    public var isSecure = false

    public static let text = JackpotFieldKind()

    public static let givenName = JackpotFieldKind(contentType: .givenName,
                                                    capitalization: .words,
                                                    disablesAutocorrection: true)

    public static let familyName = JackpotFieldKind(contentType: .familyName,
                                                    capitalization: .words,

```

```

        disablesAutocorrection: true)

    public static let email = JackpotFieldKind(keyboard: .emailAddress,
        contentType: .emailAddress,
        capitalization: .never,
        disablesAutocorrection: true)

    public static let phoneNumber = JackpotFieldKind(keyboard: .phonePad,
        contentType: .telephoneNumber,
        capitalization: .never,
        disablesAutocorrection: true)

    /// `.newPassword`, not `.password`: it opts the field into iOS's strong-password suggestion,
    /// which is what a registration form wants.
    public static let newPassword = JackpotFieldKind(contentType: .newPassword,
        capitalization: .never,
        disablesAutocorrection: true,
        isSecure: true)

    public static let oneTimeCode = JackpotFieldKind(keyboard: .numberPad,
        contentType: .oneTimeCode,
        capitalization: .never,
        disablesAutocorrection: true)

    public static let number = JackpotFieldKind(keyboard: .numberPad,
        capitalization: .never,
        disablesAutocorrection: true)

    public func with(_ transform: (inout JackpotFieldKind) -> Void) -> JackpotFieldKind {
        var copy = self
        transform(&copy)
        return copy
    }
}

public extension View {
    func jackpotField(_ kind: JackpotFieldKind) -> some View {
        modifier(JackpotFieldKindModifier(kind: kind))
    }
}

private struct JackpotFieldKindModifier: ViewModifier {
    let kind: JackpotFieldKind

    func body(content: Content) -> some View {
        content
            .keyboardType(kind.keyboard)
            .textContentType(kind.contentType)
            .textInputAutocapitalization(kind.capitalization.textInput)
            .autocorrectionDisabled(kind.disablesAutocorrection)
            .jackpotSecureEntry(kind.isSecure)
    }
}

```

11 JackpotKit/Sources/JackpotUI/Fields/JackpotTextField.swift

```

import SwiftUI

public struct JackpotTextField: View {
    @Binding private var text: String
    private let placeholder: String
    private var editingEndedAction: (() -> Void)?
    private var focusChangedAction: ((Bool) -> Void)?

    @Environment(\.jackpotTheme) private var theme
    @Environment(\.jackpotFieldPrefix) private var prefix
    @Environment(\.jackpotFieldSuffix) private var suffix
    @Environment(\.jackpotSecureEntry) private var isSecure
    @Environment(\.jackpotFocusedField) private var focusedField
    @Environment(\.jackpotFieldIdentity) private var identity
    @Environment(\.jackpotSubmitLabel) private var submitLabel
    @FocusState private var isFocused: Bool
    @State private var isRevealed = false

    private var requestedFocus: String? { focusedField?.wrappedValue }

    public init(_ placeholder: String, text: Binding<String>) {

```

```

        self.placeholder = placeholder
        self._text = text
    }

    /// Fires on blur. Chain it before any `View` modifier, like `Gesture.onEnded`.
    public func onEditingEnded(_ action: @escaping () -> Void) -> Self {
        var copy = self
        copy.editingEndedAction = action
        return copy
    }

    /// Fires on both focus and blur. The field owns its `FocusState`, so this is the only way
    /// out for callers that reveal supporting content while the field is being edited.
    public func onFocusChange(_ action: @escaping (Bool) -> Void) -> Self {
        var copy = self
        copy.focusChangedAction = action
        return copy
    }

    public var body: some View {
        HStack(spacing: 0) {
            if !prefix.isEmpty {
                Text(prefix)
                    .jackpotTextStyle(\.fieldText)
                    .padding(.horizontal, theme.metrics.contentPadding)
                    .frame(height: theme.metrics.controlHeight)
                    .overlay(alignment: .trailing) {
                        Rectangle().fill(theme.colors.fieldBorder).frame(width: 1)
                    }
                    .contentShape(Rectangle())
                    .onTapGesture { isFocused = true }
                    .accessibilityHidden(true)
            }

            input
                .jackpotTextStyle(\.fieldText)
                .focused($isFocused)
                .submitLabel(submitLabel)
                .padding(.horizontal, theme.metrics.contentPadding)
                .frame(height: theme.metrics.controlHeight)
                // The prefix cell is hidden above, so fold it in rather than leaving
                // VoiceOver to stumble over a stray "+27".
                .accessibilityLabel(prefix.isEmpty ? Text(placeholder) : Text("\(placeholder), \(prefix)"))

            if !suffix.isEmpty {
                Text(suffix)
                    .jackpotTextStyle(\.fieldText, color: \.textSecondary)
                    .padding(.trailing, theme.metrics.contentPadding)
                    .contentShape(Rectangle())
                    .onTapGesture { isFocused = true }
            }

            if isSecure {
                Button {
                    isRevealed.toggle()
                } label: {
                    Image(systemName: isRevealed ? "eye.slash" : "eye")
                        .jackpotForegroundColor(\.textPrimary)
                }
                .frame(width: theme.metrics.minimumHitTarget, height: theme.metrics.minimumHitTarget)
                .padding(.trailing, 6)
                .accessibilityLabel(isRevealed ? "Hide password" : "Show password")
            }
        }
        .jackpotFieldBackground(isFocused: isFocused)
        .onChange(of: isFocused) { focused in
            focusChangedAction?(focused)
            if focused {
                if let identity { focusedField?.wrappedValue = identity }
            } else {
                editingEndedAction?()
            }
        }
    }

    /// The other half of the sync: the form moves the shared value, the field follows.
    /// Guarded both ways so the two `onChange`s cannot ping-pong.
    .onChange(of: requestedFocus) { requested in
        guard focusedField != nil, let identity else { return }
        let shouldFocus = requested == identity
        if isFocused != shouldFocus { isFocused = shouldFocus }
    }
}

```

```

}

@ViewBuilder
private var input: some View {
    if isSecure, !isRevealed {
        SecureField(placeholder, text: $text)
    } else {
        TextField(placeholder, text: $text)
    }
}
}
}

```

12 JackpotKit/Sources/JackpotUI/Fields/JackpotTextArea.swift

```

import SwiftUI

public struct JackpotTextArea: View {
    @Binding private var text: String
    private let placeholder: String
    private var editingEndedAction: (() -> Void)?

    @Environment(\.jackpotTheme) private var theme
    @FocusState private var isFocused: Bool

    public init(_ placeholder: String, text: Binding<String>) {
        self.placeholder = placeholder
        self._text = text
    }

    /// Fires on blur. Chain it before any `View` modifier, like `Gesture.onEnded`.
    public func onEditingEnded(_ action: @escaping () -> Void) -> Self {
        var copy = self
        copy.editingEndedAction = action
        return copy
    }

    public var body: some View {
        ZStack(alignment: .topLeading) {
            editor
                .focused($isFocused)
                .frame(minHeight: theme.metrics.textAreaMinHeight)
                .padding(8)
                .jackpotTextStyle(\.fieldText)
                .accessibilityLabel(placeholder)

            if text.isEmpty {
                Text(placeholder)
                    .jackpotTextStyle(\.fieldText, color: \.textSecondary)
                    .padding(.horizontal, 13)
                    .padding(.vertical, 16)
                    .allowsHitTesting(false)
                    .accessibilityHidden(true)
            }
        }
        .jackpotFieldBackground(isFocused: isFocused)
        .onChange(of: isFocused) { focused in
            if !focused { editingEndedAction?() }
        }
    }

    /// Before iOS 16 the only way to clear the editor's opaque background is
    /// `UITextView.appearance()`, which would affect every text view in the app.
    @ViewBuilder
    private var editor: some View {
        if #available(iOS 16.0, *) {
            TextEditor(text: $text).scrollContentBackground(.hidden)
        } else {
            TextEditor(text: $text)
        }
    }
}

```

13 [JackpotKit/Sources/JackpotUI/Fields/JackpotDropdown.swift](#)

```

import SwiftUI

public struct JackpotOption: Identifiable, Hashable, Sendable {
    public let id: String
    public let label: String

    public init(id: String, label: String) {
        self.id = id
        self.label = label
    }
}

public struct JackpotDropdown: View {
    @Binding private var selection: String?
    private let placeholder: String
    private let options: [JackpotOption]

    @Environment(\.jackpotTheme) private var theme

    public init(_ placeholder: String, selection: Binding<String?>, options: [JackpotOption]) {
        self.placeholder = placeholder
        self._selection = selection
        self.options = options
    }

    public var body: some View {
        Menu {
            ForEach(options) { option in
                Button {
                    selection = option.id
                } label: {
                    if option.id == selection {
                        Label(option.label, systemImage: "checkmark")
                    } else {
                        Text(option.label)
                    }
                }
            }
        } label: {
            HStack {
                Text(selected?.label ?? placeholder)
                    .jackpotForegroundColor(valueColor)
                Spacer()
                Image(systemName: "chevron.down").jackpotForegroundColor(\.textPrimary)
            }
            .jackpotFont(\.fieldText)
            .padding(.horizontal, theme.metrics.contentPadding)
            .frame(height: theme.metrics.controlHeight)
            .jackpotFieldBackground()
        }
        .accessibilityLabel(placeholder)
        .accessibilityValue(selected?.label ?? "None")
    }

    private var selected: JackpotOption? {
        options.first { $0.id == selection }
    }

    private var valueColor: KeyPath<JackpotColors, Color> {
        selected == nil ? \.textSecondary : \.textPrimary
    }
}

```

14 [JackpotKit/Sources/JackpotUI/Fields/JackpotRadioGroup.swift](#)

```

import SwiftUI

public struct JackpotRadioGroup: View {
    @Binding private var selection: String?
    private let options: [JackpotOption]

    @Environment(\.jackpotTheme) private var theme

    public init(selection: Binding<String?>, options: [JackpotOption]) {

```

```

        self._selection = selection
        self.options = options
    }

    public var body: some View {
        VStack(alignment: .leading, spacing: 8) {
            ForEach(options) { option in
                row(for: option)
            }
        }
    }

    private func row(for option: JackpotOption) -> some View {
        let isSelected = selection == option.id
        return Button {
            selection = option.id
        } label: {
            HStack(spacing: theme.metrics.spacing) {
                Image(systemName: isSelected ? "largecircle.fill.circle" : "circle")
                    .foregroundColor(isSelected ? theme.colors.accent : theme.colors.textSecondary)
                    .animation(.easeOut(duration: 0.15), value: isSelected)
                Text(option.label).jackpotTextStyle(\.rowLabel)
                Spacer(minLength: 0)
            }
            .frame(minHeight: theme.metrics.minimumHitTarget)
        }
        .buttonStyle(.plain)
        .accessibilityAddTraits(isSelected ? [.isButton, .isSelected] : .isButton)
    }
}

```

15 JackpotKit/Sources/JackpotUI/Fields/JackpotDateField.swift

The `Calendar` input type: a read-only field presenting a graphical picker in a sheet.

```

import SwiftUI

public struct JackpotDateField: View {
    @Binding private var selection: Date?
    private let placeholder: String
    private let range: PartialRangeThrough<Date>

    @Environment(\.jackpotTheme) private var theme
    @Environment(\.jackpotFieldLabel) private var fieldLabel
    @State private var isPresented = false

    public init(_ placeholder: String,
                selection: Binding<Date?>,
                in range: PartialRangeThrough<Date> = ...Date()) {
        self.placeholder = placeholder
        self._selection = selection
        self.range = range
    }

    public var body: some View {
        Button {
            isPresented = true
        } label: {
            HStack {
                Text(displayedValue)
                    .jackpotForegroundColor(valueColor)
                Spacer()
                Image(systemName: "calendar").jackpotForegroundColor(\.textPrimary)
            }
            .jackpotFont(\.fieldText)
            .padding(.horizontal, theme.metrics.contentPadding)
            .frame(height: theme.metrics.controlHeight)
            .jackpotFieldBackground()
        }
        .buttonStyle(.plain)
        .accessibilityLabel(fieldLabel ?? placeholder)
        .accessibilityValue(selection == nil ? "None" : displayedValue)
        .sheet(isPresented: $isPresented) {
            if #available(iOS 16.0, *) {
                sheet
                    .presentationDetents([.height(sheetHeight)])
                    .presentationDragIndicator(.visible)
            }
        }
    }
}

```



```

        } else {
            sheet
        }
    }
}

/// Wheel plus title and button. Before iOS 16 there are no detents, so the sheet is full
/// height and the spacer pins the button to the bottom instead.
private var sheetHeight: CGFloat { wheelHeight + 160 }

private var wheelHeight: CGFloat { 216 }

private var sheet: some View {
    ZStack {
        theme.colors.surface.ignoresSafeArea()

        VStack(spacing: theme.metrics.spacing) {
            Text(fieldLabel ?? placeholder)
                .jackpotTextStyle(\.button)
                .padding(.top, 20)

            DatePicker("",
                selection: Binding(get: { selection ?? range.upperBound },
                                    set: { selection = $0 }),
                in: range,
                displayedComponents: .date)
                .datePickerStyle(.wheel)
                .labelsHidden()
                .frame(height: wheelHeight)
                .padding(.horizontal)

            Spacer(minLength: 0)

            Button("Done") {
                // Confirming without dragging still counts as a choice, otherwise the
                // field silently stays empty.
                if selection == nil { selection = range.upperBound }
                isPresented = false
            }
                .buttonStyle(.jackpot)
                .padding([.horizontal, .bottom], 16)
        }
    }
}

private var displayedValue: String {
    guard let selection else { return placeholder }
    return selection.formatted(date: .abbreviated, time: .omitted)
}

private var valueColor: KeyPath<JackpotColors, Color> {
    selection == nil ? \.textSecondary : \.textPrimary
}
}

```

16 JackpotKit/Sources/JackpotUI/Components/JackpotChecklist.swift

The live password-rules panel.

```

import SwiftUI

public struct JackpotChecklistItem: Identifiable, Equatable, Sendable {
    public let id: String
    public let text: String
    public let isSatisfied: Bool

    public init(id: String, text: String, isSatisfied: Bool) {
        self.id = id
        self.text = text
        self.isSatisfied = isSatisfied
    }
}

public struct JackpotChecklist: View {
    private let title: String
    private let section: String
    private let items: [JackpotChecklistItem]
}

```

```

@Environment(\.jackpotTheme) private var theme
@State private var isExpanded = true

public init(_ title: String, section: String = "Required", items: [JackpotCheckListItem]) {
    self.title = title
    self.section = section
    self.items = items
}

public var body: some View {
    if !items.isEmpty {
        DisclosureGroup(isExpanded: $isExpanded) {
            VStack(alignment: .leading, spacing: 10) {
                ProgressView(value: satisfiedFraction)
                    .progressViewStyle(.jackpotBar(height: 6))
                    .tint(satisfiedFraction < 1 ? theme.colors.warning : theme.colors.success)
                    .accessibilityLabel("Requirements met")

                Text(section).jackpotTextStyle(\.sectionTitle)

                ForEach(items) { item in
                    row(for: item)
                }
            }
            .padding(.top, 10)
            .frame(maxWidth: .infinity, alignment: .leading)
        } label: {
            Text(title).jackpotTextStyle(\.sectionTitle)
        }
        // The chevron follows the tint, which the theme points at the accent colour.
        .tint(theme.colors.textPrimary)
        .padding(theme.metrics.contentPadding)
        .jackpotBackground(\.fieldBackground, in: theme.metrics.fieldShape)
        .animation(.spring(response: 0.3, dampingFraction: 1), value: isExpanded)
    }
}

private func row(for item: JackpotCheckListItem) -> some View {
    HStack(spacing: 10) {
        Image(systemName: item.isSatisfied ? "checkmark.circle.fill" : "circle")
            .foregroundColor(item.isSatisfied ? theme.colors.accent : theme.colors.textSecondary)
            .animation(.easeOut(duration: 0.15), value: item.isSatisfied)
        Text(item.text).jackpotTextStyle(\.rowLabel)
        Spacer()
    }
    .accessibilityElement(children: .combine)
    .accessibilityValue(item.isSatisfied ? "Met" : "Not met")
}

private var satisfiedFraction: Double {
    guard !items.isEmpty else { return 0 }
    return Double(items.filter(\.isSatisfied).count) / Double(items.count)
}
}

```

17 JackpotKit/Sources/JackpotUI/Components/JackpotErrorView.swift

Shown when a form fails to load.

```

import SwiftUI

public struct JackpotErrorView: View {
    private let message: String
    private let title: String
    private var retryAction: (() -> Void)?

    public init(_ message: String, title: String = "Something went wrong") {
        self.message = message
        self.title = title
    }

    /// Adds the retry button. Without it the view is message-only.
    public func onRetry(_ action: @escaping () -> Void) -> Self {
        var copy = self
        copy.retryAction = action
        return copy
    }
}

```

```

}

public var body: some View {
    VStack(spacing: 12) {
        Image(systemName: "exclamationmark.triangle")
            .font(.largeTitle)
            .jackpotForegroundColorStyle(\.textSecondary)
            .accessibilityHidden(true)

        Text(title).jackpotTextStyle(\.button)

        Text(message)
            .jackpotTextStyle(\.label, color: \.textSecondary)
            .multilineTextAlignment(.center)
            .fixedSize(horizontal: false, vertical: true)

        if let retryAction {
            Button("Retry", action: retryAction)
                .buttonStyle(.jackpot)
                .frame(width: 160)
        }
    }
    .frame(maxWidth: .infinity)
    .padding(24)
}
}

```

18 JackpotKit/Sources/JackpotUI/Components/JackpotLockedOverlay.swift

Dims and un-hits content gated behind something else — the Welcome offer picker, until the fields above it validate.

```

import SwiftUI

public struct JackpotLockedOverlay: ViewModifier {
    private let isLocked: Bool
    private let message: String

    @Environment(\.jackpotTheme) private var theme

    public init(isLocked: Bool, message: String) {
        self.isLocked = isLocked
        self.message = message
    }

    public func body(content: Content) -> some View {
        ZStack {
            content
                .opacity(isLocked ? 0.35 : 1)
                .allowsHitTesting(!isLocked)
                .accessibilityHidden(isLocked)

            if isLocked {
                Text(message)
                    .jackpotTextStyle(\.sectionTitle)
                    .multilineTextAlignment(.center)
                    .padding(.horizontal, 24)
                    .shadow(radius: 4)
            }
        }
        .animation(.easeOut(duration: 0.2), value: isLocked)
    }
}

public extension View {
    func jackpotLocked(_ isLocked: Bool, message: String) -> some View {
        modifier(JackpotLockedOverlay(isLocked: isLocked, message: message))
    }
}

```

19 JackpotKit/Sources/JackpotUI/Preview/JackpotPreviewPanel.swift

The panel wraps a preview in the themed surface; the gallery is how you check the components without running the app.

```

#if DEBUG
import SwiftUI

public struct JackpotPreviewPanel<Content: View>: View {
    private let title: String?
    private let content: Content

    public init(_ title: String? = nil, @ViewBuilder content: () -> Content) {
        self.title = title
        self.content = content()
    }

    public var body: some View {
        VStack(alignment: .leading, spacing: 12) {
            if let title {
                Text(title).font(.caption).jackpotForegroundColorStyle(\.textSecondary)
            }
            content
        }
        .padding(16)
        .frame(width: 390)
        .jackpotTheme(.jackpotCity)
        .jackpotBackground(\.surface)
    }
}

// MARK: – Component gallery

struct JackpotUI_Previews: PreviewProvider {
    struct Harness: View {
        @State private var mobile = ""
        @State private var email = ""
        @State private var secret = "Passwo1"
        @State private var touched = false
        @State private var promotions = false
        @State private var agreed = true
        @State private var persistLogin = false
        @State private var income: String? = nil
        @State private var contactMethod: String? = "email"
        @State private var dateOfBirth: Date? = nil
        @State private var offer: String? = "depositMatch"

        var body: some View {
            ScrollView {
                JackpotPreviewPanel("Gallery") {
                    JackpotLabeledField("Mobile") {
                        JackpotTextField("Enter Mobile Number", text: $mobile)
                            .onEditingEnded { touched = true }
                            .jackpotField(.phoneNumber)
                            .jackpotFieldPrefix("+27")
                    }

                    JackpotLabeledField("Email") {
                        JackpotTextField("Enter Email Address", text: $email)
                            .jackpotField(.email)
                    }

                    JackpotLabeledField(error: "Password must be 8–20 characters") {
                        JackpotTextField("Password", text: $secret)
                            .jackpotField(.newPassword)
                    }

                    JackpotChecklist("Password Validity", items: [
                        .init(id: "min", text: "Minimum of 8 characters", isSatisfied: false),
                        .init(id: "max", text: "Maximum of 20 characters", isSatisfied: true),
                    ])

                    JackpotLabeledField("Source Of Income", error: "Please choose one") {
                        JackpotDropdown("Enter Source Of Income", selection: $income, options: [
                            .init(id: "salary", label: "Salary or Wages"),
                            .init(id: "pension", label: "Pension or Grant"),
                        ])
                    }

                    JackpotLabeledField("Date Of Birth") {
                        JackpotDateField("Enter Date Of Birth", selection: $dateOfBirth)
                    }

                    JackpotLabeledField("Preferred Contact") {

```

```

        JackpotRadioGroup(selection: $contactMethod, options: [
            .init(id: "sms", label: "SMS"),
            .init(id: "email", label: "Email"),
        ])
    }

    Toggle("Send Jackpot City Promotions to me", isOn: $promotions)
        .toggleStyle(.jackpotCheckbox)
    Toggle("I am over 18 years of age & I accept the Terms & Conditions", isOn: $agreed)
        .toggleStyle(.jackpotCheckbox)
    Toggle("Keep me logged in", isOn: $persistLogin)
        .toggleStyle(.jackpotSwitch)

    ProgressView(value: 0.45).progressViewStyle(.jackpotBar)

    HStack(spacing: 10) {
        offerCard("100% Deposit Match", id: "depositMatch")
        offerCard("50 Free Spins", id: "freeSpins")
    }
    .jackpotLocked(true, message: "Complete your registration above to unlock your Welcome offer selection")

    Button("Next") {}.buttonStyle(.jackpot).disabled(true)
    Button("Sign Up") {}.buttonStyle(.jackpot).jackpotLoading()
    Button("Previous") {}.buttonStyle(.jackpot(.secondary))
    Button("Skip for now") {}.buttonStyle(.jackpot(.tertiary))
}
.jackpotTheme(.jackpotCity)
.jackpotBackground(\.surface)
}

private func offerCard(_ label: String, id: String) -> some View {
    Button { offer = id } label: {
        Text(label).jackpotTextStyle(\.button).multilineTextAlignment(.center)
    }
    .buttonStyle(.jackpotCard)
    .jackpotSelected(offer == id)
}

static var previews: some View {
    Group {
        Harness().preferredColorScheme(.dark).previewDisplayName("Gallery - dark")
        Harness().preferredColorScheme(.light).previewDisplayName("Gallery - light")
        JackpotPreviewPanel("Error") {
            JackpotErrorView("The network connection was lost.").onRetry {}
        }
        .preferredColorScheme(.dark)
        .previewDisplayName("Error - dark")
        JackpotPreviewPanel("Error") {
            JackpotErrorView("The network connection was lost.").onRetry {}
        }
        .preferredColorScheme(.light)
        .previewDisplayName("Error - light")
        Harness()
        .environment(\.sizeCategory, .accessibilityLarge)
        .previewDisplayName("Gallery - XL text")
    }
    .previewLayout(.sizeThatFits)
}
}
#endif

```

20 JackpotKit/Tests/JackpotUITests/JackpotThemeTests.swift

```

import SwiftUI
import XCTest
@testable import JackpotUI

final class JackpotThemeTests: XCTestCase {
    func testBrandThemeComposesTheStandardPresets() {
        XCTAssertEqual(JackpotTheme.jackpotCity.colors, .jackpotCity)
        XCTAssertEqual(JackpotTheme.jackpotCity.metrics, .standard)
        XCTAssertEqual(JackpotTheme.jackpotCity.typography, .standard)
    }

    func testWithReturnsACopyAndLeavesTheOriginalAlone() {

```

```

    let brand = JackpotTheme.jackpotCity
    let roomy = brand.with { $0.metrics.cornerRadius = 24 }

    XCTAssertEqual(roomy.metrics.cornerRadius, 24)
    XCTAssertEqual(brand.metrics.cornerRadius, 10)
    XCTAssertEqual(roomy.colors, brand.colors, "An unrelated slice should carry over untouched")
}

func testFieldShapeFollowsCornerRadius() {
    let theme = JackpotTheme.jackpotCity.with { $0.metrics.cornerRadius = 4 }
    XCTAssertEqual(theme.metrics.fieldShape.cornerSize, CGSize(width: 4, height: 4))
    XCTAssertEqual(theme.metrics.fieldShape.style, .continuous)
}

final class JackpotColorSchemeTests: XCTestCase {
    private let light = UITraitCollection(userInterfaceStyle: .light)
    private let dark = UITraitCollection(userInterfaceStyle: .dark)

    func testSurfaceAndTextInvertBetweenAppearances() {
        let colors = JackpotColors.jackpotCity
        for keyPath in [\JackpotColors.surface, \.fieldBackground, \.textPrimary, \.textSecondary] {
            let color = UIColor(colors[keyPath: keyPath])
            XCTAssertNotEqual(color.resolvedColor(with: light),
                              color.resolvedColor(with: dark),
                              "\ (keyPath) should differ per appearance")
        }
    }

    func testTextOnAccentDoesNotInvert() {
        let color = UIColor(JackpotColors.jackpotCity.textOnAccent)
        XCTAssertEqual(color.resolvedColor(with: light), color.resolvedColor(with: dark))
    }

    /// Text has to clear 4.5:1 on every background it can land on, not just `surface`.
    func testTextClearsWCAGContrastOnEveryBackgroundItLandsOn() {
        let colors = JackpotColors.jackpotCity
        let backgrounds: [(String, KeyPath<JackpotColors, Color>)] = [
            ("surface", \.surface),
            ("fieldBackground", \.fieldBackground),
        ]
        let foregrounds: [(String, KeyPath<JackpotColors, Color>)] = [
            ("textPrimary", \.textPrimary),
            ("textSecondary", \.textSecondary),
            ("accent", \.accent),
            ("error", \.error),
            ("warning", \.warning),
            ("success", \.success),
        ]

        for traits in [light, dark] {
            for (bgName, bg) in backgrounds {
                let background = resolve(colors[keyPath: bg], traits)
                for (fgName, fg) in foregrounds {
                    let text = flatten(resolve(colors[keyPath: fg], traits), over: background)
                    XCTAssertGreaterThan(
                        contrastRatio(text, background), 4.5,
                        "\ (fgName) on \ (bgName) in \ (name(traits)) is unreadable")
                }
            }
        }
    }

    func testButtonLabelClearsContrastOnItsFill() {
        let colors = JackpotColors.jackpotCity
        for traits in [light, dark] {
            let fill = resolve(colors.accentFill, traits)
            let label = flatten(resolve(colors.textOnAccent, traits), over: fill)
            XCTAssertGreaterThan(contrastRatio(label, fill), 4.5,
                                "textOnAccent on accentFill in \ (name(traits))")
        }
    }

    func testFieldFillIsDistinguishableFromTheSurfaceBehindIt() {
        let colors = JackpotColors.jackpotCity
        for traits in [light, dark] {
            let fill = resolve(colors.fieldBackground, traits)
            let surface = resolve(colors.surface, traits)
            XCTAssertNotEqual(fill, surface, "field fill matches the surface in \ (name(traits))")
            // 1.05, not 3:1 – the design's own fill separates by 1.06, so this pins the

```

```

        // regression (fill identical to surface) without overruling the design.
        XCTAssertGreaterThan(contrastRatio(fill, surface), 1.05,
            "field fill is too close to the surface in \(name(traits))")
    }
}

// MARK: Helpers

private func name(_ traits: UITraitCollection) -> String {
    traits.userInterfaceStyle == .dark ? "dark" : "light"
}

private func resolve(_ color: Color, _ traits: UITraitCollection) -> UIColor {
    UIColor(color).resolvedColor(with: traits)
}

/// Several palette entries are translucent white. Reading their components straight back
/// reports the contrast of opaque white, so they have to be composited first.
private func flatten(_ color: UIColor, over background: UIColor) -> UIColor {
    let fg = components(color), bg = components(background)
    return UIColor(red: fg.r * fg.a + bg.r * (1 - fg.a),
                    green: fg.g * fg.a + bg.g * (1 - fg.a),
                    blue: fg.b * fg.a + bg.b * (1 - fg.a),
                    alpha: 1)
}

/// WCAG 2.1 relative luminance.
private func contrastRatio(_ a: UIColor, _ b: UIColor) -> CGFloat {
    let lighter = max(luminance(a), luminance(b))
    let darker = min(luminance(a), luminance(b))
    return (lighter + 0.05) / (darker + 0.05)
}

private func components(_ color: UIColor) -> (r: CGFloat, g: CGFloat, b: CGFloat, a: CGFloat) {
    var r: CGFloat = 0, g: CGFloat = 0, b: CGFloat = 0, a: CGFloat = 0
    color.getRed(&r, green: &g, blue: &b, alpha: &a)
    return (r, g, b, a)
}

private func luminance(_ color: UIColor) -> CGFloat {
    let c = components(color)
    func channel(_ value: CGFloat) -> CGFloat {
        value <= 0.03928 ? value / 12.92 : pow((value + 0.055) / 1.055, 2.4)
    }
    return 0.2126 * channel(c.r) + 0.7152 * channel(c.g) + 0.0722 * channel(c.b)
}

final class JackpotFieldKindTests: XCTestCase {
    func testEmailOptsOutOfCapitalisationAndAutocorrection() {
        XCTAssertEqual(JackpotFieldKind.email.keyboard, .emailAddress)
        XCTAssertEqual(JackpotFieldKind.email.contentType, .emailAddress)
        XCTAssertTrue(JackpotFieldKind.email.disablesAutocorrection)
        XCTAssertFalse(JackpotFieldKind.email.isSecure)
    }

    func testNewPasswordIsSecureAndOptsIntoStrongPasswordSuggestions() {
        XCTAssertTrue(JackpotFieldKind.newPassword.isSecure)
        XCTAssertEqual(JackpotFieldKind.newPassword.contentType, .newPassword)
    }

    func testWithOverridesASingleTrait() {
        let kind = JackpotFieldKind.text.with { $0.capitalization = .words }
        XCTAssertEqual(kind.capitalization, .words)
        XCTAssertEqual(kind.keyboard, JackpotFieldKind.text.keyboard)
    }
}

final class JackpotOptionTests: XCTestCase {
    func testOptionsAreIdentifiedByTheirStoredValue() {
        let option = JackpotOption(id: "SalaryOrWages", label: "Salary or Wages")
        XCTAssertEqual(option.id, "SalaryOrWages")
    }

    func testCheckListItemEquatable() {
        let a = JackpotCheckListItem(id: "min", text: "Minimum of 8", isSatisfied: false)
        XCTAssertEqual(a, JackpotCheckListItem(id: "min", text: "Minimum of 8", isSatisfied: false))
        XCTAssertNotEqual(a, JackpotCheckListItem(id: "min", text: "Minimum of 8", isSatisfied: true))
    }
}

```

21

```
cd JackpotKit
xcodebuild -scheme JackpotKit-Package -destination 'platform=iOS Simulator,name=iPhone 17 Pro' test
```

► Create PR — JackpotUI

JackpotUITests: Executed 13 tests, with 0 failures

Open `JackpotPreviewPanel.swift` and resume the **Gallery** preview.

PR 2 — JackpotForms

Three modules pointing one way: `JackpotFormsDomain` (types and rules, no I/O), `JackpotFormsData` (wire shapes and the bundled stub), `JackpotFormsUI` (the engine and the renderer). Nothing here knows how a form is fetched — the engine only ever sees `FormRepository`, which is what lets PR 3 swap the stub for the network without touching it.

22

```
mkdir -p JackpotKit/Sources/{JackpotFormsDomain,JackpotFormsData}
mkdir -p JackpotKit/Sources/JackpotFormsUI/{Fields,Demo,Resources}
mkdir -p JackpotKit/Tests/JackpotFormsTests/Fixtures
```

23 **JackpotKit/Package.swift — add to products: and targets:**

```
.library(
    name: "JackpotForms",
    targets: ["JackpotFormsDomain", "JackpotFormsData", "JackpotFormsUI"]
),

.target(name: "JackpotFormsDomain"),
.target(name: "JackpotFormsData", dependencies: ["JackpotFormsDomain"]),
.target(
    name: "JackpotFormsUI",
    dependencies: ["JackpotFormsDomain", "JackpotUI"],
    resources: [.process("Resources")]
),
.testTarget(
    name: "JackpotFormsTests",
    dependencies: ["JackpotFormsDomain", "JackpotFormsData", "JackpotFormsUI"],
    resources: [.process("Fixtures")]
),
```

24 **JackpotKit/Sources/JackpotFormsDomain/FormName.swift**

```
import Foundation

/// Identifies a form in the CRM — the `formCodeName` in the schema, and the last path
/// component of the fetch URL.
///
/// Deliberately not an enum. Forms are authored server-side and new ones appear without an app
/// release, so a closed set would fight the architecture. This is the `Notification.Name`
/// pattern instead: a `RawRepresentable` wrapper with static members for the forms this build
/// knows about, and `FormName("deposit")` for anything else.
///
/// Not `ExpressibleByStringLiteral`, on purpose: `formName: "registraton"` would compile and
/// give a runtime 404. Constructing one from an arbitrary string has to be written out, so
/// `FormName()` stays greppable at review time.
/// Not `Codable`: a form name never crosses a serialisation boundary as itself. Decoding goes
/// `FormDTO.formCodeName` (a `String`) → `FormName(_)` in the mapper, and encoding goes
/// `FormSubmitBody.formName = submission.formCodeName.rawValue`. Both directions are explicit at
/// the wire type, which is where the JSON key names already live.
public struct FormName: RawRepresentable, Hashable, Sendable, CustomStringConvertible {

    public let rawValue: String

    public init(rawValue: String) {
        self.rawValue = rawValue
    }

    public init(_ rawValue: String) {
        self.rawValue = rawValue
    }

    public var description: String { rawValue }
}

// MARK: — Known forms
//
// Add a member here when the CRM starts serving a form you reference by name in code. Forms
// you only ever reach dynamically — from a sitemap, deep link or the forms list endpoint —
```

```
// never need one.

public extension FormName {
    /// The two-section sign-up form: credentials + name + email, then FICA.
    static let registration = FormName("registration")

    /// Development-only schema exercising every supported field type.
    static let kitchenSink = FormName("kitchenSink")

    /// Forms bundled with the package as JSON, for mocks, previews and the sandbox.
    static let bundled: [FormName] = [.registration, .kitchenSink]
}
```

25 JackpotKit/Sources/JackpotFormsDomain/FormValue.swift

```
import Foundation

/// A field's current value.
///
/// Every case renders as a string because the schema validates with regexes – including the
/// checkboxes, whose patterns are literally `^true$`. `stringValue` is therefore both what
/// gets validated and what gets submitted.
public enum FormValue: Equatable, Hashable, Sendable {
    case empty
    case text(String)
    case bool(Bool)
    case option(String)
    case date(Date)

    public var stringValue: String {
        switch self {
            case .empty: return ""
            case .text(let s): return s
            case .bool(let b): return b ? "true" : "false"
            case .option(let v): return v
            case .date(let d): return FormValue.iso8601.string(from: d)
        }
    }

    public var isEmpty: Bool {
        switch self {
            case .empty: return true
            case .text(let s): return s.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty
            case .option(let v): return v.isEmpty
            // An unticked required checkbox counts as empty, which is what makes `terms` work.
            case .bool(let b): return !b
            case .date: return false
        }
    }

    public var boolValue: Bool {
        if case .bool(let b) = self { return b }
        return stringValue.lowercased() == "true"
    }

    public var dateValue: Date? {
        switch self {
            case .date(let d): return d
            case .text(let s): return FormValue.iso8601.date(from: s)
            default: return nil
        }
    }

    /// The `dateOfBirth` regex in the schema expects an ISO-8601 date-time with
    /// optional fractional seconds and offset, so that is what we emit.
    public static let iso8601: ISO8601DateFormatter = {
        let formatter = ISO8601DateFormatter()
        formatter.formatOptions = [.withInternetDateTime]
        return formatter
    }()
}

/// What the host receives in the submit callback, and what the cron submit endpoint expects.
/// Encoding lives in `JackpotFormsRemote` so this type stays free of JSON key names.
public struct FormSubmission: Equatable, Sendable {
    public let formId: String
    public let formCodeName: FormName
}
```

```

public let submittedAt: Date
public let values: [String: FormValue]
public let metadata: [String: String]?

public init(formCodeName: FormName,
            values: [String: FormValue],
            formId: String = "",
            submittedAt: Date = Date(),
            metadata: [String: String]? = nil) {
    self.formId = formId
    self.formCodeName = formCodeName
    self.submittedAt = submittedAt
    self.values = values
    self.metadata = metadata
}

public subscript(identifier: String) -> FormValue {
    values[identifier] ?? .empty
}

/// Flat string payload, for hosts that want one. The real submit body uses `values`,
/// because fields are typed on the wire (bool stays bool, empty becomes null).
public var stringValue: [String: String] {
    values.mapValues(\.stringValue)
}
}

```

26 JackpotKit/Sources/JackpotFormsDomain/FormField.swift

```

import Foundation

/// What component renders this field.
///
/// `unknown` is load-bearing: the schema is served from a CRM that product edits without
/// shipping an app build, so an unrecognised `fieldType` that threw would brick registration
/// for every installed version. Unknown fields are skipped and reported through
/// `DynamicFormModel.unsupportedFields` instead.
public enum FieldType: Equatable, Hashable, Sendable {
    case input
    case button
    case checkbox
    case radio
    case radioGroup
    case dropdown
    case divider
    case textArea
    case recaptchaV2
    case recaptchaV3
    case toggle
    case welcomeOffer
    case unknown(String)

    public init(raw: String) {
        switch raw.lowercased().replacingOccurrences(of: " ", with: "") {
            case "input": self = .input
            case "button": self = .button
            case "checkbox": self = .checkbox
            case "radio": self = .radio
            case "radiogroup": self = .radioGroup
            case "dropdown", "select": self = .dropdown
            case "divider": self = .divider
            case "textarea": self = .textArea
            case "recapchav2", "recaptchav2": self = .recaptchaV2
            case "recapchav3", "recaptchav3": self = .recaptchaV3
            case "toggle": self = .toggle
            case "welcomeoffer": self = .welcomeOffer
            default: self = .unknown(raw)
        }
    }

    /// Layout-only types hold no value and are never validated or submitted.
    public var isDecorative: Bool {
        switch self {
            case .divider, .button: return true
            default: return false
        }
    }
}

```

```

}

/// Keyboard and formatting hint for `.input`.
public enum InputType: Equatable, Hashable, Sendable {
    case text
    case number
    case password
    case email
    case calendar
    case phone
    case unknown(String)

    public init(raw: String) {
        switch raw.lowercased() {
            case "text": self = .text
            case "number", "numeric": self = .number
            case "password": self = .password
            case "email": self = .email
            // The schema spells this "Calender". Accept both so a server-side fix
            // doesn't silently turn every date field into a plain text box.
            case "calender", "calendar", "date": self = .calendar
            case "phone", "tel": self = .phone
            default: self = .unknown(raw)
        }
    }
}

public struct DropdownOption: Identifiable, Equatable, Hashable, Sendable {
    /// Submitted value, e.g. "SalaryOrWages".
    public let value: String
    /// Localization key for the visible text, e.g. "jpc-reg-SalaryOrWages".
    public let textKey: String
    /// Either a regex pattern or the *name* of one – see `RegexResolving`.
    public let regex: String?

    public var id: String { value }

    public init(value: String, textKey: String, regex: String?) {
        self.value = value
        self.textKey = textKey
        self.regex = regex
    }
}

public struct RadioOption: Identifiable, Equatable, Hashable, Sendable {
    public let value: String
    public let textKey: String

    public var id: String { value }

    public init(value: String, textKey: String) {
        self.value = value
        self.textKey = textKey
    }
}

public struct FormField: Identifiable, Equatable, Hashable, Sendable {
    public let id: Int
    /// Key used for state, validation and the submitted payload. e.g. "idNumber".
    public let identifier: String
    public let name: String
    /// Localization key for the label.
    public let labelKey: String
    public let type: FieldType
    public let inputType: InputType
    public let textStyle: String
    /// Localization key for the validation failure message.
    public let validationMessageKey: String
    public let isRequired: Bool
    public let isVisible: Bool
    public let readOnly: Bool
    /// Regex the value must match. Server-supplied, so it may be invalid – see `FieldValidator`.
    public let regex: String?
    public let prefix: String
    public let suffix: String
    /// Localization key for the placeholder.
    public let placeholderKey: String
    public let dropdownOptions: [DropdownOption]
    public let radioOptions: [RadioOption]

```

```

public init(id: Int, identifier: String, name: String, labelKey: String,
           type: FieldType, inputType: InputType, textStyle: String,
           validationMessageKey: String, isRequired: Bool, isVisible: Bool, isReadOnly: Bool,
           regex: String?, prefix: String, suffix: String, placeholderKey: String,
           dropdownOptions: [DropdownOption], radioOptions: [RadioOption]) {
    self.id = id
    self.identifier = identifier
    self.name = name
    self.labelKey = labelKey
    self.type = type
    self.inputType = inputType
    self.textStyle = textStyle
    self.validationMessageKey = validationMessageKey
    self.isRequired = isRequired
    self.isVisible = isVisible
    self.isReadOnly = isReadOnly
    self.regex = regex
    self.prefix = prefix
    self.suffix = suffix
    self.placeholderKey = placeholderKey
    self.dropdownOptions = dropdownOptions
    self.radioOptions = radioOptions
}

/// Fields that hold a value: rendered, validated and submitted.
public var carriesValue: Bool {
    isVisible && !type.isDecorative && !(type == .recaptchaV2 || type == .recaptchaV3)
}

public var isSecure: Bool { inputType == .password }
}

```

27 JackpotKit/Sources/JackpotFormsDomain/Form.swift

```

import Foundation

/// A whole form as the CRM form-builder describes it. A section is one page of the wizard;
/// fields sharing a row sit side by side.
public struct FormSchema: Identifiable, Equatable, Sendable {
    public let id: Int
    public let codeName: FormName
    public let title: String
    public let subTitle: String
    public let regionCode: String
    public let sections: [FormSection]

    public init(id: Int, codeName: FormName, title: String, subTitle: String,
               regionCode: String, sections: [FormSection]) {
        self.id = id
        self.codeName = codeName
        self.title = title
        self.subTitle = subTitle
        self.regionCode = regionCode
        self.sections = sections
    }

    /// Every visible field, in render order, across all sections.
    public var allFields: [FormField] {
        sections.flatMap(\.fields)
    }

    public func field(identifiedBy identifier: String) -> FormField? {
        allFields.first { $0.identifier == identifier }
    }
}

public struct FormSection: Identifiable, Equatable, Sendable {
    public let id: Int
    public let codeName: String
    public let title: String
    public let subTitle: String
    public let order: Int
    public let rows: [FormRow]

    public init(id: Int, codeName: String, title: String, subTitle: String, order: Int, rows: [FormRow]) {
        self.id = id
        self.codeName = codeName
    }
}

```

```

        self.title = title
        self.subTitle = subTitle
        self.order = order
        self.rows = rows
    }

    public var fields: [FormField] { rows.flatMap(\.fields) }
}

public struct FormRow: Identifiable, Equatable, Sendable {
    public let number: Int
    public let fields: [FormField]

    public var id: Int { number }

    public init(number: Int, fields: [FormField]) {
        self.number = number
        self.fields = fields
    }
}

```

28 JackpotKit/Sources/JackpotFormsDomain/FormSubmitResult.swift

```

import Foundation

/// What `POST /cron/forms/submit` returns on a 2xx body.
///
/// HTTP 200 is not success – the envelope carries `isSuccessful` and an `error` object.
/// A created account can still need manual FICA (`partialRegistrationStatus`, compliance).
public struct FormSubmitResult: Equatable, Sendable {
    public let accountId: String?
    public let message: String?
    public let status: String?
    public let partialRegistrationStatus: Int?
    public let compliance: FormComplianceResult?

    public init(accountId: String? = nil,
                message: String? = nil,
                status: String? = nil,
                partialRegistrationStatus: Int? = nil,
                compliance: FormComplianceResult? = nil) {
        self.accountId = accountId
        self.message = message
        self.status = status
        self.partialRegistrationStatus = partialRegistrationStatus
        self.compliance = compliance
    }

    public var isPartial: Bool { (partialRegistrationStatus ?? 0) != 0 }
}

public struct FormComplianceResult: Equatable, Sendable {
    public let complianceStatus: Int?
    public let requiredComplianceStatus: Int?
    public let isValidId: Bool?
    public let message: String?
    public let accessToken: String?

    public init(complianceStatus: Int? = nil,
                requiredComplianceStatus: Int? = nil,
                isValidId: Bool? = nil,
                message: String? = nil,
                accessToken: String? = nil) {
        self.complianceStatus = complianceStatus
        self.requiredComplianceStatus = requiredComplianceStatus
        self.isValidId = isValidId
        self.message = message
        self.accessToken = accessToken
    }
}

```

29 JackpotKit/Sources/JackpotFormsDomain/PasswordPolicy.swift

```

import Foundation

```

```

/// One rule shown in the "Password Validity" panel.
public struct PasswordRule: Identifiable, Equatable, Sendable {
    public let id: String
    public let description: String
    private let test: @Sendable (String) -> Bool

    public init(id: String, description: String, test: @escaping @Sendable (String) -> Bool) {
        self.id = id
        self.description = description
        self.test = test
    }

    public func isSatisfied(by password: String) -> Bool { test(password) }

    public static func == (lhs: PasswordRule, rhs: PasswordRule) -> Bool { lhs.id == rhs.id }
}

/// Supplies the checklist behind the password field.
///
/// The schema gives password a single regex, `^(.){8,20}$`, but the design shows two
/// independently ticking rules, which one regex match cannot produce. Parsing the `{min,max}`
/// quantifier reproduces the design for this form; see OPEN-QUESTIONS.
public protocol PasswordPolicyProviding: Sendable {
    func rules(for field: FormField) -> [PasswordRule]
}

public struct PasswordPolicy: PasswordPolicyProviding {
    public init() {}

    public func rules(for field: FormField) -> [PasswordRule] {
        let bounds = Self.lengthBounds(in: field.regex)
        var rules: [PasswordRule] = []

        if let minimum = bounds.min {
            rules.append(PasswordRule(id: "min",
                                      description: "Minimum of \ (minimum) characters",
                                      test: { $0.count >= minimum }))
        }
        if let maximum = bounds.max {
            rules.append(PasswordRule(id: "max",
                                      description: "Maximum of \ (maximum) characters",
                                      test: { !$0.isEmpty && $0.count <= maximum }))
        }
        return rules
    }

    /// Pulls `{8,20}` out of `^(.){8,20}$`.
    static func lengthBounds(in pattern: String?) -> (min: Int?, max: Int?) {
        guard let pattern,
              let expression = try? NSRegularExpression(pattern: #"\\{(\d+),(\d+)\}"#),
              let match = expression.firstMatch(in: pattern, range: NSRange(pattern.startIndex..., in: pattern)),
              let minRange = Range(match.range(at: 1), in: pattern),
              let maxRange = Range(match.range(at: 2), in: pattern)
        else { return (nil, nil) }
        return (Int(pattern[minRange]), Int(pattern[maxRange]))
    }
}

```

30 [JackpotKit/Sources/JackpotFormsDomain/RegexResolving.swift](#)

How a dropdown changes another field's rule: the schema names a pattern, this resolves it.

```

import Foundation

/// Dropdown options in the schema carry a `regex` that is sometimes a pattern (`"[a-zA-Z]"` on
/// sourceOfFunds) and sometimes the *name* of one (`"idNumberRegex"` on idNumberType). A name
/// has to resolve to a pattern somewhere; this protocol is that somewhere.
///
/// Named regexes *replace* the dependent field's rule: choosing "South African ID" vs
/// "Passport" changes what a valid ID Number is. `DynamicFormModel` applies that, gated on
/// `FormDependencies.appliesOptionRegexToDependentField`.
public protocol RegexResolving: Sendable {
    /// Pattern for a named regex, or nil if the name is unknown.
    func pattern(named name: String) -> String?
}

public struct RegexCatalog: RegexResolving {

```

```

private let patterns: [String: String]

public init(patterns: [String: String]) {
    self.patterns = patterns
}

public func pattern(named name: String) -> String? {
    patterns[name]
}

/// Named option regexes that redirect onto a dependent field. `passportNumberRegex` is a
/// **length** rule, not a character class: any 5–20 characters.
public static let jpcDefaults = RegexCatalog(patterns: [
    "idNumberRegex": "^[0-9]{13}$",
    "passportNumberRegex": "^.{5,20}$",
])

}

public extension String {
    /// Tells a regex *pattern* from a regex *name*. A bare identifier has no metacharacters;
    /// a real pattern almost always does.
    var looksLikeRegexPattern: Bool {
        let metacharacters = CharacterSet(charactersIn: "^$[]{}()*+?\\.")
        return rangeOfCharacter(from: metacharacters) != nil
    }
}

```

31 JackpotKit/Sources/JackpotFormsDomain/FieldValidator.swift

```

import Foundation

public enum ValidationResult: Equatable, Sendable {
    case valid
    /// Localization key for the message to show.
    case invalid(messageKey: String)

    public var isValid: Bool { self == .valid }
}

/// Validates a value against a field's schema rules.
///
/// The regexes come from a server, so they can be malformed. A pattern that will not compile
/// is treated as "no constraint" rather than propagating the throw — one CRM typo must not
/// break signup. Compiled expressions are cached by pattern string, because compiling on every
/// keystroke for every field is wasteful.
public struct FieldValidator: Sendable {
    private let regexResolver: any RegexResolving
    private let cache = RegexCache()

    public init(regexResolver: any RegexResolving = RegexCatalog.jpcDefaults) {
        self.regexResolver = regexResolver
    }

    /// - Parameter overrideRegex: pattern that replaces `field.regex`, used when a
    /// dropdown selection changes a dependent field's rule (ID type → ID number).
    public func validate(_ value: FormValue,
        against field: FormField,
        overrideRegex: String? = nil) -> ValidationResult {
        guard field.carriesValue, !field.isReadOnly else { return .valid }

        if field.isRequired, value.isEmpty {
            return .invalid(messageKey: field.validationMessageKey)
        }

        // Some schema regexes permit empty explicitly (referralCode: `...|^$`) but not all do,
        // so this guard is what keeps optional fields genuinely optional.
        if !field.isRequired, value.isEmpty { return .valid }

        guard let pattern = resolvedPattern(overrideRegex ?? field.regex), !pattern.isEmpty else {
            return .valid
        }
        guard let expression = cache.expression(for: pattern) else { return .valid }

        let subject = value.stringValue
        let range = NSRange(subject.startIndex..

```



```

}

/// Pattern for a dropdown option's `regex`, resolving names via the catalogue.
public func optionPattern(_ raw: String?) -> String? {
    resolvedPattern(raw)
}

private func resolvedPattern(_ raw: String?) -> String? {
    guard let raw, !raw.isEmpty else { return nil }
    if let named = regexResolver.pattern(named: raw) { return named }
    return raw.looksLikeRegexPattern ? raw : nil
}

}

/// Thread-safe compiled-regex cache.
private final class RegexCache: @unchecked Sendable {
    private var storage: [String: NSRegularExpression?] = [:]
    private let lock = NSLock()

    func expression(for pattern: String) -> NSRegularExpression? {
        lock.lock()
        defer { lock.unlock() }
        if let cached = storage[pattern] { return cached }
        let compiled = try? NSRegularExpression(pattern: pattern)
        storage[pattern] = compiled
        return compiled
    }
}

```

32 JackpotKit/Sources/JackpotFormsDomain/FormLoadError.swift

```

import Foundation

/// Why a form operation failed, in terms the UI can render.
///
/// `JackpotFormsUI` depends on Domain only, so `APIError` cannot reach a view model and the
/// repository has to translate at the boundary. Without this type the server's own wording
/// ("Mobile number already registered") gets decoded, carried up, and then thrown away in
/// favour of a generic string.
public enum FormLoadError: LocalizedError, Equatable {
    case offline
    case notFound(FormName)
    /// The server explained itself. Prefer its wording over ours.
    case server(message: String)
    case unexpected

    public var errorDescription: String? {
        switch self {
        case .offline:
            return "You're offline. Check your connection and try again."
        case .notFound(let name):
            return "We couldn't find the \(name.rawValue) form. Please try again later."
        case .server(let message):
            return message
        case .unexpected:
            return "Something went wrong. Please try again."
        }
    }
}

```

33 JackpotKit/Sources/JackpotFormsDomain/FormLocalizing.swift

```

import Foundation

/// The schema ships localization *keys*, not display text: `fieldLabel` is "username", dropdown
/// text is "jpc-reg-idnumber", `validationMessage` is "regex". Something has to resolve them
/// against a string catalogue.
public protocol FormLocalizing: Sendable {
    func string(forKey key: String) -> String?

    /// Copy for a server error code, when the localisation table carries one.
    ///
    /// The app-data response contains entries keyed by error code, so a numeric `code` in an
    /// API error envelope is a localisation key. Defaulted to nil so an implementation with no
    /// such table doesn't have to care.

```

```

    func message(forErrorCode code: Int) -> String?
}

public extension FormLocalizing {
    func message(forErrorCode code: Int) -> String? { nil }

    /// Resolve, or fall back to a humanised version of the key so nothing renders blank.
    func display(_ key: String) -> String {
        string(forKey: key) ?? key.humanisedKey
    }

    /// Every field carries the same `validationMessage` ("regex") while the UI shows per-field
    /// copy, so the real key is composed: `jpc-reg-{fieldIdentifier}-{validationMessage}`. See
    /// OPEN-QUESTIONS; confirming the format is a one-line change here.
    func validationMessage(for field: FormField) -> String {
        let composed = "jpc-reg-\\(field.identifier)-\\(field.validationMessageKey)"
        if let resolved = string(forKey: composed) { return resolved }
        if let resolved = string(forKey: field.validationMessageKey) { return resolved }
        return "Please enter a valid \\(field.identifier.humanisedKey.lowercased())"
    }
}

/// Looks up an in-memory table, then a bundle's `strings`.
public struct ComposedKeyLocalizer: FormLocalizing {
    private let table: [String: String]
    private let bundle: Bundle?

    public init(table: [String: String] = [:], bundle: Bundle? = nil) {
        self.table = table
        self.bundle = bundle
    }

    public func string(forKey key: String) -> String? {
        if let value = table[key] { return value }
        guard let bundle else { return nil }
        let value = bundle.localizedString(forKey: key, value: "\u{0}", table: nil)
        return value == "\u{0}" ? nil : value
    }
}

public extension String {
    /// "jpc-reg-idnumber" -> "Idnumber"; "dateOfBirth" -> "Date Of Birth"
    var humanisedKey: String {
        var working = self
        if let range = working.range(of: "jpc-reg-") { working.removeSubrange(range) }
        working = working.replacingOccurrences(of: "-", with: " ").replacingOccurrences(of: "_", with: " ")

        var spaced = ""
        for character in working {
            if character.isUppercase, !spaced.isEmpty, spaced.last != " " { spaced.append(" ") }
            spaced.append(character)
        }
        return spaced.prefix(1).uppercased() + spaced.dropFirst()
    }
}

/// Wraps any `(key) -> String?` as a localizer, so a host can hand the form engine its
/// existing translation function without either package knowing the other exists.
///
/// `FormLocalizing` uses nil to mean "unresolved" so the engine can fall back to humanised
/// copy. A host function that returns the key on a miss must map that back to nil, or a
/// missing string renders as the raw key.
public struct ClosureLocalizer: FormLocalizing {
    private let resolve: @Sendable (String) -> String?
    private let resolveCode: @Sendable (Int) -> String?

    public init(_ resolve: @escaping @Sendable (String) -> String?,
        errorCode resolveCode: @escaping @Sendable (Int) -> String? = { _ in nil }) {
        self.resolve = resolve
        self.resolveCode = resolveCode
    }

    public func string(forKey key: String) -> String? { resolve(key) }
    public func message(forErrorCode code: Int) -> String? { resolveCode(code) }
}

```

34 [JackpotKit/Sources/JackpotFormsDomain/FormRepository.swift](#)

```
import Foundation

/// Contract for form data operations: fetch a definition and submit it.
///
/// Fetch-by-name and fetch-by-id share one URL:
/// `GET {cron}/forms/{brand}/{region}/{identifier}?api-version=2.0`.
/// Submit posts a `FormSubmission` to `{cron}/forms/submit`.
public protocol FormRepository: Sendable {
    func form(named name: FormName) async throws -> FormSchema
    func form(id: String) async throws -> FormSchema
    func submitForm(_ submission: FormSubmission) async throws -> FormSubmitResult
}

public extension FormRepository {
    func form(id: String) async throws -> FormSchema {
        throw FormLoadError.notFound(FormName(id))
    }
}
```

35 [JackpotKit/Sources/JackpotFormsData/FormDT0.swift](#)

```
import Foundation

// Wire shapes, exactly as the CRM form-builder sends them. Nothing outside this file knows
// about "formSectionCodeName" or the "Calender" spelling.
//
// Everything is optional except the identifiers we cannot render without: the schema is edited
// by product in a CMS, so a missing `prefix` must not fail the whole decode.

public struct FormDT0: Decodable {
    let formId: Int
    let formCodeName: String
    let formTitle: String?
    let formSubTitle: String?
    let regionCode: String?
    let sections: [FormSectionDT0]?
}

public struct FormSectionDT0: Decodable {
    let formSectionId: Int
    let formSectionCodeName: String?
    let formSectionTitle: String?
    let formSectionSubTitle: String?
    let formSectionOrder: Int?
    let rows: [FormRowDT0]?
}

public struct FormRowDT0: Decodable {
    let rowNumber: Int
    let fields: [FormFieldDT0]?
}

public struct FormFieldDT0: Decodable {
    let fieldId: Int
    let fieldIdentifier: String
    let fieldName: String?
    let fieldLabel: String?
    let fieldType: String
    let inputType: String?
    let textStyle: String?
    let validationMessage: String?
    let isRequired: Bool?
    let isVisible: Bool?
    let isReadOnly: Bool?
    let fieldRegex: String?
    let prefix: String?
    let suffix: String?
    let fieldPlaceholder: String?
    let fieldDropdowns: [FieldDropdownDT0]?
    let fieldRadioGroup: [FieldRadioDT0]?
}

public struct FieldDropdownDT0: Decodable {
    let value: String
}
```

```

    let text: String?
    let regex: String?
}

public struct FieldRadioDTO: Decodable {
    let value: String
    let text: String?
}

```

36 JackpotKit/Sources/JackpotFormsData/FormMapper.swift

```

import Foundation
import JackpotFormsDomain

public enum FormMapper {
    public static func map(_ dto: FormDTO) -> FormSchema {
        FormSchema(
            id: dto.formId,
            codeName: FormName(dto.formCodeName),
            title: dto.formTitle ?? dto.formCodeName,
            subTitle: dto.formSubTitle ?? "",
            regionCode: dto.regionCode ?? "",
            sections: (dto.sections ?? [])
                .map(mapSection)
                .sorted { $0.order < $1.order }
        )
    }

    private static func mapSection(_ dto: FormSectionDTO) -> FormSection {
        FormSection(
            id: dto.formSectionId,
            codeName: dto.formSectionCodeName ?? "\(dto.formSectionId)",
            title: dto.formSectionTitle ?? "",
            subTitle: dto.formSectionSubTitle ?? "",
            order: dto.formSectionOrder ?? dto.formSectionId,
            rows: (dto.rows ?? [])
                .map(mapRow)
                .sorted { $0.number < $1.number }
        )
    }

    private static func mapRow(_ dto: FormRowDTO) -> FormRow {
        FormRow(number: dto.rowNumber, fields: (dto.fields ?? []).map(mapField))
    }

    private static func mapField(_ dto: FormFieldDTO) -> FormField {
        FormField(
            id: dto.fieldId,
            identifier: dto.fieldIdentifier,
            name: dto.fieldName ?? dto.fieldIdentifier,
            labelKey: dto.fieldLabel ?? dto.fieldIdentifier,
            type: FieldType(raw: dto.fieldType),
            inputType: InputType(raw: dto.inputType ?? "Text"),
            textStyle: dto.textStyle ?? "Regular",
            validationMessageKey: dto.validationMessage ?? "regex",
            // Defaults chosen to fail safe: an unspecified field is optional and
            // visible rather than silently blocking submission.
            isRequired: dto.isRequired ?? false,
            isVisible: dto.isVisible ?? true,
            readOnly: dto.isReadOnly ?? false,
            regex: dto.fieldRegex?.isEmpty == true ? nil : dto.fieldRegex,
            prefix: dto.prefix ?? "",
            suffix: dto.suffix ?? "",
            placeholderKey: dto.fieldPlaceholder ?? dto.fieldIdentifier,
            dropdownOptions: (dto.fieldDropdowns ?? []).map {
                DropdownOption(value: $0.value, textKey: $0.text ?? $0.value, regex: $0.regex)
            },
            radioOptions: (dto.fieldRadioGroup ?? []).map {
                RadioOption(value: $0.value, textKey: $0.text ?? $0.value)
            }
        )
    }
}

```

37 [JackpotKit/Sources/JackpotFormsData/StubFormRepository.swift](#)

```

import Foundation
import JackpotFormsDomain

/// Serves forms from bundled JSON. Backs the demo harness and previews, and lets the whole
/// feature be built and reviewed before the endpoint is reachable from the app.
public struct StubFormRepository: FormRepository {
    private let forms: [FormName: Data]
    /// Seconds. (`Duration` is iOS 16 – this package targets 15.)
    private let delay: TimeInterval
    private let error: (any Error)?

    public init(forms: [FormName: Data], delay: TimeInterval = 0.35, error: (any Error)? = nil) {
        self.forms = forms
        self.delay = delay
        self.error = error
    }

    public func form(named name: FormName) async throws -> FormSchema {
        try await prepare()
        guard let data = forms[name] else { throw FormLoadError.notFound(name) }
        return FormMapper.map(try JSONDecoder().decode(FormDTO.self, from: data))
    }

    public func form(id: String) async throws -> FormSchema {
        try await prepare()
        for data in forms.values {
            let form = FormMapper.map(try JSONDecoder().decode(FormDTO.self, from: data))
            if String(form.id) == id { return form }
        }
        throw FormLoadError.notFound(FormName(id))
    }

    public func submitForm(_ submission: FormSubmission) async throws -> FormSubmitResult {
        try await prepare()
        return FormSubmitResult()
    }

    /// Decodes raw JSON straight to a form – for tests and previews.
    public static func decode(_ data: Data) throws -> FormSchema {
        FormMapper.map(try JSONDecoder().decode(FormDTO.self, from: data))
    }

    private func prepare() async throws {
        if delay > 0 {
            try await Task.sleep(nanoseconds: UInt64(delay * 1_000_000_000))
        }
        if let error { throw error }
    }
}

```

38

```

cp registration.json JackpotKit/Sources/JackpotFormsUI/Resources/registration.json
cp kitchenSink.json JackpotKit/Sources/JackpotFormsUI/Resources/kitchenSink.json
cp JackpotKit/Sources/JackpotFormsUI/Resources/registration.json \
  JackpotKit/Tests/JackpotFormsTests/Fixtures/registration.json

```

`registration.json` is the CRM's response saved verbatim — 12 fields over two sections; `kitchenSink.json` exercises the field types the real schema doesn't use. `JackpotFormsUI` processes both as resources. The mirror under `Fixtures/` lets the suites load the schema from their own `Bundle.module` instead of reaching into another target's.

39 [JackpotKit/Sources/JackpotFormsUI/FormDependencies.swift](#)

```

import SwiftUI
import JackpotFormsDomain

/// Everything `DynamicFormView` needs that isn't the form name or the callback.
///
/// Carried through the environment so the public call site stays at two arguments –
/// `DynamicFormView(formName: .registration) { submission in ... }` – while every dependency

```

```

/// is still injected explicitly at the composition root.
public struct FormDependencies {
    public var repository: any FormRepository
    public var validator: FieldValidator
    public var localizer: any FormLocalizing
    public var passwordPolicy: any PasswordPolicyProviding
    /// Whether a dropdown option's *named* regex overrides another field's rule. The ID Number
    /// Type dropdown selects SA ID vs passport, and ID Number must validate accordingly.
    public var appliesOptionRegexToDependentField: Bool

    /// Explicit "this dropdown drives this field's regex" links, keyed by driver identifier.
    /// Stated outright rather than inferred from row order, so a CRM reorder cannot silently
    /// relax validation on a regulated field.
    public var regexDependencies: [String: String]
    /// Latest date a `Calendar` field allows. Defaults to 18 years ago: the form's only age gate
    /// today is the T&C checkbox, and a picker that cannot select an under-18 date is a cheap
    /// second line of defence.
    public var maximumDateOfBirth: Date

    public init(repository: any FormRepository,
                validator: FieldValidator = FieldValidator(),
                localizer: any FormLocalizing = ComposedKeyLocalizer(),
                passwordPolicy: any PasswordPolicyProviding = PasswordPolicy(),
                appliesOptionRegexToDependentField: Bool = true,
                regexDependencies: [String: String] = ["idNumberType": "idNumber"],
                maximumDateOfBirth: Date = Calendar(identifier: .gregorian)
                .date(byAdding: .year, value: -18, to: Date()) ?? Date()) {
        self.repository = repository
        self.validator = validator
        self.localizer = localizer
        self.passwordPolicy = passwordPolicy
        self.appliesOptionRegexToDependentField = appliesOptionRegexToDependentField
        self.regexDependencies = regexDependencies
        self.maximumDateOfBirth = maximumDateOfBirth
    }
}

private struct FormDependenciesKey: EnvironmentKey {
    static var defaultValue: FormDependencies {
        FormDependencies(repository: UnavailableFormRepository(assertsWhenCalled: true))
    }
}

/// Never returns a form. Used wherever a repository is structurally required but must not be
/// called: the environment default (which asserts, because reaching it means the caller forgot
/// `.formDependencies(_)`), the placeholder a two-argument `DynamicFormView` holds until
/// `.task` swaps in the environment's, and previews that seed a model directly.
struct UnavailableFormRepository: FormRepository {
    let assertsWhenCalled: Bool

    init(assertsWhenCalled: Bool = false) {
        self.assertsWhenCalled = assertsWhenCalled
    }

    func form(named name: FormName) async throws -> FormSchema {
        assertConfigured()
        throw CancellationError()
    }

    func submitForm(_ submission: FormSubmission) async throws -> FormSubmitResult {
        assertConfigured()
        throw CancellationError()
    }

    private func assertConfigured() {
        if assertsWhenCalled {
            assertionFailure("No FormDependencies in the environment. Call .formDependencies(_) above DynamicFormView.")
        }
    }
}

public extension EnvironmentValues {
    var formDependencies: FormDependencies {
        get { self[FormDependenciesKey.self] }
        set { self[FormDependenciesKey.self] = newValue }
    }
}

public extension View {
    func formDependencies(_ dependencies: FormDependencies) -> some View {

```

```

        environment(\.formDependencies, dependencies)
    }
}

#if DEBUG
public extension FormDependencies {
    /// Never fetches – previews seed the schema directly – but carries the real localizer so
    /// the copy matches the designs.
    static var preview: FormDependencies {
        FormDependencies(repository: UnavailableFormRepository(),
            localizer: ComposedKeyLocalizer.jpcRegistration)
    }
}
#endif

```

40 [JackpotKit/Sources/JackpotFormsUI/DynamicFormModel.swift](#)

The engine. Two things here are worth reading closely: `touched` is why an untouched field stays silent until Next, and `applyRegexDependencies` is how selecting Passport relaxes the SA-ID rule on a different field.

```

import Foundation
import Combine
import JackpotFormsDomain

/// The engine. Owns the fetched schema, every field's value, touched state and errors, and
/// which section (page) is showing.
///
/// `ObservableObject` rather than `Observable` because this package targets iOS 15.
@MainActor
public final class DynamicFormModel: ObservableObject {

    public enum ViewState: Equatable {
        case loading
        case loaded(FormSchema)
        case failed(String)
    }

    // MARK: Published state
    @Published public internal(set) var viewState: ViewState = .loading
    @Published public private(set) var values: [String: FormValue] = [:]
    @Published public private(set) var errors: [String: String] = [:]
    @Published public internal(set) var sectionIndex: Int = 0
    @Published public private(set) var isSubmitting = false
    @Published public private(set) var submitError: String?

    /// Field types the schema asked for that this build cannot render. Non-fatal by design
    /// (see `FieldType.unknown`); surfaced so QA and logs can see them.
    @Published public private(set) var unsupportedFields: [String] = []

    /// `@Published` because `objectWillChange` has to fire *before* the set changes, or the
    /// error it reveals lands a render late.
    @Published private var touched: Set<String> = []

    // MARK: Inputs
    private let formName: FormName
    private var dependencies: FormDependencies
    private var isConfigured: Bool
    private var loadTask: Task<Void, Never>?

    /// - Parameter isConfigured: true when the caller supplied real dependencies. The
    ///   two-argument `DynamicFormView.init` passes false and lets `configureIfNeeded` swap in
    ///   the environment's copy on first appearance, because `@StateObject` cannot read
    ///   `@Environment` from an initializer.
    public init(formName: FormName, dependencies: FormDependencies, isConfigured: Bool = true) {
        self.formName = formName
        self.dependencies = dependencies
        self.isConfigured = isConfigured
    }

    /// Adopts environment-provided dependencies exactly once. No-op afterwards, so a
    /// re-render never resets a half-filled form.
    public func configureIfNeeded(with dependencies: FormDependencies) {
        guard !isConfigured else { return }
        self.dependencies = dependencies
        isConfigured = true
    }
}

```

```

// MARK: Derived

public var form: FormSchema? {
    if case .loaded(let form) = viewState { return form }
    return nil
}

public var sections: [FormSection] { form?.sections ?? [] }
public var currentSection: FormSection? {
    sections.indices.contains(sectionIndex) ? sections[sectionIndex] : nil
}

public var isFirstSection: Bool { sectionIndex == 0 }
public var isLastSection: Bool { sectionIndex >= sections.count - 1 }

/// Fraction of the form's required fields that currently validate. Drives the progress bar.
public var progress: Double {
    guard let form else { return 0 }
    let required = form.allFields.filter { $0.carriesValue && $0.isRequired }
    guard !required.isEmpty else { return 1 }
    return Double(required.filter(isValid).count) / Double(required.count)
}

/// Whether the visible section can be advanced past / submitted.
public var isCurrentSectionValid: Bool {
    guard let section = currentSection else { return false }
    return section.fields.filter(\.carriesValue).allSatisfy(isValid)
}

public var isFormValid: Bool {
    guard let form else { return false }
    return form.allFields.filter(\.carriesValue).allSatisfy(isValid)
}

public func value(for field: FormField) -> FormValue {
    values[field.identifier] ?? defaultValue(for: field)
}

/// Error text for a field, or nil while it is untouched.
public func error(for field: FormField) -> String? {
    touched.contains(field.identifier) ? errors[field.identifier] : nil
}

public func localized(_ key: String) -> String { dependencies.localizer.display(key) }

public func passwordRules(for field: FormField) -> [PasswordRule] {
    dependencies.passwordPolicy.rules(for: field)
}

public var maximumDateOfBirth: Date { dependencies.maximumDateOfBirth }

// MARK: Loading

public func load() {
    loadTask?.cancel()
    viewState = .loading
    loadTask = Task { [weak self] in
        guard let self else { return }
        do {
            let form = try await dependencies.repository.form(named: formName)
            guard !Task.isCancelled else { return }
            self.apply(form)
        } catch is CancellationError {
        } catch {
            guard !Task.isCancelled else { return }
            self.viewState = .failed(Self.message(for: error))
        }
    }
}

private func apply(_ form: FormSchema) {
    viewState = .loaded(form)
    sectionIndex = 0
    touched = []
    errors = [:]
    values = Dictionary(uniqueKeysWithValues:
        form.allFields.filter(\.carriesValue).map { ($0.identifier, defaultValue(for: $0)) }
    )
    unsupportedFields = form.allFields.compactMap {
        if case .unknown(let raw) = $0.type { return "\($0.identifier) \($raw)" }
    }
}

```



```

        return nil
    }
    revalidateAll()
}

private func defaultValue(for field: FormField) -> FormValue {
    switch field.type {
    case .checkbox, .toggle: return .bool(false)
    case .dropdown, .radio, .radioGroup: return .option("")
    default: return field.inputType == .calendar ? .empty : .text("")
    }
}

// MARK: Editing

public func setValue(_ value: FormValue, for field: FormField) {
    values[field.identifier] = value
    validate(field)
    // A dropdown can change a dependent field's rule (ID type → ID number), so anything
    // downstream of it has to be re-checked, not just this field.
    if field.type == .dropdown, dependencies.appliesOptionRegexToDependentField {
        revalidateDependents(of: field)
    }
}

/// Call on blur so errors don't appear before the user has typed. Re-touching is a no-op
/// rather than another render: the return key marks a field, then the blur that follows
/// marks it again.
public func markTouched(_ field: FormField) {
    guard !touched.contains(field.identifier) else { return }
    touched.insert(field.identifier)
}

/// Focus navigation only carries identifiers.
public func markTouched(identifiedBy identifier: String) {
    guard let field = form?.field(identifiedBy: identifier) else { return }
    markTouched(field)
}

// MARK: Validation

private func isValid(_ field: FormField) -> Bool {
    errors[field.identifier] == nil
}

@discardableResult
private func validate(_ field: FormField) -> Bool {
    let result = dependencies.validator.validate(
        value(for: field),
        against: field,
        overrideRegex: overrideRegex(for: field)
    )
    switch result {
    case .valid:
        errors[field.identifier] = nil
        return true
    case .invalid:
        errors[field.identifier] = dependencies.localizer.validationMessage(for: field)
        return false
    }
}

private func revalidateAll() {
    form?.allFields.filter(\.carriesValue).forEach { validate($0) }
}

private func revalidateDependents(of field: FormField) {
    guard let form else { return }
    let dependents = form.allFields.filter { candidate in
        candidate.carriesValue && regexDriver(for: candidate)?.id == field.id
    }
    dependents.forEach { validate($0) }
}

/// The dropdown that drives `field`'s regex, if any. Links come from
/// `FormDependencies.regexDependencies` and are never inferred from field order.
private func regexDriver(for field: FormField) -> FormField? {
    guard dependencies.appliesOptionRegexToDependentField,
        let form,
        let driverIdentifier = dependencies.regexDependencies.first(where: { $0.value == field.identifier })?.key,

```

```

        let driver = form.field(identifiedBy: driverIdentifier),
        driver.type == .dropdown
    else { return nil }
    return driver
}

/// The pattern a dropdown's current selection imposes on `field`, if any.
///
/// Only *named* regexes redirect. `idNumberType`'s options carry `"idNumberRegex"` /
/// `"passportNumberRegex"` – names – while `sourceOfFunds`'s carry `"[a-zA-Z]"`, a literal
/// pattern that describes the selection itself and must not leak onto another field.
private func overrideRegex(for field: FormField) -> String? {
    guard let driver = regexDriver(for: field),
        case .option(let selected) = value(for: driver),
        !selected.isEmpty,
        let option = driver.dropdownOptions.first(where: { $0.value == selected }),
        let raw = option.regex,
        !raw.looksLikeRegexPattern
    else { return nil }

    return dependencies.validator.optionPattern(raw)
}

// MARK: Paging

/// Advances if the visible section validates; otherwise reveals its errors.
@discardableResult
public func advance() -> Bool {
    guard let section = currentSection else { return false }
    touched.formUnion(section.fields.filter(\.carriesValue).map(\.identifier))
    revalidateAll()
    guard isCurrentSectionValid else { return false }
    if !isLastSection { sectionIndex += 1 }
    return true
}

public func goBack() {
    guard sectionIndex > 0 else { return }
    sectionIndex -= 1
}

// MARK: Submitting

public func submit(_ handler: @escaping (FormSubmission) async throws -> Void) async {
    guard let form else { return }
    touched.formUnion(form.allFields.filter(\.carriesValue).map(\.identifier))
    revalidateAll()
    guard isValid else { return }

    isSubmitting = true
    submitError = nil
    defer { isSubmitting = false }

    do {
        try await handler(FormSubmission(
            formCodeName: form.codeName,
            values: values,
            formId: String(form.id)
        ))
    } catch is CancellationError {
    } catch {
        submitError = Self.message(for: error)
    }
}

deinit { loadTask?.cancel() }

private static func message(for error: any Error) -> String {
    if let localized = (error as? LocalizedError)?.errorDescription { return localized }
    return "Something went wrong. Please try again."
}

}

#if DEBUG
// Preview support lives in this file because `private` is file-scoped in Swift, so an
// extension here can call `apply(_:)` without widening the type's real API.
public extension DynamicFormModel {

    /// A model already holding `schema`, with no async load, so previews render instantly and
    /// deterministically instead of flashing a skeleton.

```

```

static func preview(schema: FormSchema,
                    dependencies: FormDependencies = .preview,
                    values: [String: FormValue] = [:],
                    touched: [String] = [],
                    sectionIndex: Int = 0) -> DynamicFormModel {
    let model = DynamicFormModel(formName: schema.codeName, dependencies: dependencies)
    model.apply(schema)
    for (identifier, value) in values {
        guard let field = schema.field(identifiedBy: identifier) else { continue }
        model.setValue(value, for: field)
    }
    for identifier in touched {
        guard let field = schema.field(identifiedBy: identifier) else { continue }
        model.markTouched(field)
    }
    model.sectionIndex = min(max(0, sectionIndex), max(0, schema.sections.count - 1))
    return model
}

/// Stuck on the loading state, for previewing the skeleton.
static func previewLoading(dependencies: FormDependencies = .preview) -> DynamicFormModel {
    DynamicFormModel(formName: FormName("preview"), dependencies: dependencies)
}

/// Parked on the failure state.
static func previewFailed(_ message: String = "The network connection was lost.") -> DynamicFormModel {
    let model = DynamicFormModel(formName: FormName("preview"), dependencies: .preview)
    model.viewState = .failed(message)
    return model
}
}
#endif

```

41 [JackpotKit/Sources/JackpotFormsUI/Demo/PreviewFixtures.swift](#)

```

#if DEBUG
import SwiftUI
import JackpotUI
import JackpotFormsDomain

/// Hand-built fixtures mirroring the real registration schema.
///
/// Built in Swift rather than decoded from the bundled JSON on purpose: `JackpotFormsUI`
/// does not depend on `JackpotFormsData`, so it has no decoder – and previews that don't
/// touch the bundle render faster and can't fail on a missing resource. The JSON-backed
/// previews live in `JackpotForms`, which can see both layers.
public enum FormPreview {

    // MARK: Field builder

    public static func field(_ identifier: String,
                           type: FieldType = .input,
                           inputType: InputType = .text,
                           label: String? = nil,
                           placeholder: String? = nil,
                           required: Bool = true,
                           regex: String? = nil,
                           prefix: String = "",
                           dropdowns: [DropdownOption] = [],
                           radios: [RadioOption] = []) -> FormField {
        FormField(
            id: abs(identifier.hashValue % 10_000),
            identifier: identifier,
            name: identifier,
            labelKey: label ?? identifier,
            type: type,
            inputType: inputType,
            textStyle: "Regular",
            validationMessageKey: "regex",
            isRequired: required,
            isVisible: true,
            isReadOnly: false,
            regex: regex,
            prefix: prefix,
            suffix: "",
            placeholderKey: placeholder ?? identifier,
            dropdownOptions: dropdowns,

```

```

        radioOptions: radios
    )
}

// MARK: Individual fields, matching the real schema's rules

public static let mobile = field("username", inputType: .number,
    regex: "^(27|0)?[1-9][0-9]{8}$", prefix: "+27")

public static let password = field("password", inputType: .password,
    regex: "^(.){8,20}$")

public static let firstName = field("firstname",
    regex: "[a-zA-Z][a-zA-Z\\-\\.\\'\\s]{1,20}$")

public static let email = field("email", inputType: .email,
    regex: "[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$")

public static let referralCode = field("referralCode", required: false,
    regex: "[a-zA-Z0-9]{3,25}$|^$")

public static let idNumberType = field(
    "idNumberType", type: .dropdown, regex: "[a-zA-Z]+$",
    dropdowns: [
        DropdownOption(value: "idNumber", textKey: "jpc-reg-idnumber", regex: "idNumberRegex"),
        DropdownOption(value: "passport", textKey: "jpc-reg-passport", regex: "passportNumberRegex"),
    ]
)

public static let idNumber = field("idNumber", regex: "[0-9]{13}$")

public static let dateOfBirth = field(
    "dateOfBirth", inputType: .calendar,
    regex: "^(\\d{4})-(\\d{2})-(\\d{2})T(\\d{2}):(\\d{2}):(\\d{2}(?:\\.\\d*)?)(-?(\\d{2}):(\\d{2})|Z)?$"
)

public static let sourceOfFunds = field(
    "sourceOfFunds", type: .dropdown, regex: "[a-zA-Z]+$",
    dropdowns: [
        DropdownOption(value: "SalaryOrWages", textKey: "jpc-reg-SalaryOrWages", regex: "[a-zA-Z]"),
        DropdownOption(value: "PensionOrGrant", textKey: "jpc-reg-PensionOrGrant", regex: "[a-zA-Z]"),
        DropdownOption(value: "AllowanceOrBursary", textKey: "jpc-reg-AllowanceOrBursary", regex: "[a-zA-Z]"),
        DropdownOption(value: "SavingsOrRentalOrOther", textKey: "jpc-reg-SavingsOrRentalOrOther", regex: "[a-zA-Z]"),
        DropdownOption(value: "SelfEmployed", textKey: "jpc-reg-SelfEmployed", regex: "[a-zA-Z]"),
    ]
)

public static let promoOptIn = field("receivePromotionalInformation", type: .checkbox,
    label: "receivePromotionalInformation-jza",
    required: false, regex: "^true|^false$")

public static let terms = field("terms", type: .checkbox, required: true, regex: "^true$")

public static let notes = field("notes", type: .textArea, label: "Notes",
    placeholder: "Anything else?", required: false)

public static let contactMethod = field(
    "contactMethod", type: .radioGroup, label: "Preferred contact method", required: true,
    regex: "^.+$",
    radios: [
        RadioOption(value: "sms", textKey: "SMS"),
        RadioOption(value: "email", textKey: "Email"),
        RadioOption(value: "whatsapp", textKey: "WhatsApp"),
    ]
)

public static let welcomeOffer = field(
    "welcomeOffer", type: .welcomeOffer, label: "Select your Welcome Offer:", required: false,
    dropdowns: [
        DropdownOption(value: "depositMatch", textKey: "100% Deposit Match", regex: nil),
        DropdownOption(value: "freeSpins", textKey: "50 Free Spins", regex: nil),
    ]
)

// MARK: Whole schemas

/// The two-section registration form, structured exactly as the CRM sends it.
public static let registration = FormSchema(
    id: 1052, codeName: .registration, title: "registration", subTitle: "registration",
    regionCode: "JZA",

```

```

sections: [
    FormSection(id: 45, codeName: "1", title: "1", subTitle: "1", order: 1, rows: [
        FormRow(number: 1, fields: [mobile]),
        FormRow(number: 2, fields: [password]),
        FormRow(number: 3, fields: [firstName]),
        FormRow(number: 4, fields: [field("lastname", regex: "[a-zA-Z][a-zA-Z\\-\\.\\s]{1,20}$"))],
        FormRow(number: 5, fields: [email]),
        FormRow(number: 6, fields: [referralCode]),
    ]),
    FormSection(id: 46, codeName: "2", title: "2", subTitle: "2", order: 2, rows: [
        FormRow(number: 1, fields: [idNumberType]),
        FormRow(number: 2, fields: [idNumber]),
        FormRow(number: 3, fields: [dateOfBirth]),
        FormRow(number: 4, fields: [sourceOfFunds]),
        FormRow(number: 5, fields: [promoOptIn]),
        FormRow(number: 6, fields: [terms]),
    ]),
]
)

// A single field wrapped in a one-section schema – for previewing components alone.
public static func schema(_ fields: [FormField]) -> FormSchema {
    FormSchema(id: 1, codeName: FormName("preview"), title: "", subTitle: "", regionCode: "JZA",
        sections: [FormSection(id: 1, codeName: "1", title: "", subTitle: "", order: 1,
            rows: fields.enumerated().map { FormRow(number: $0.offset + 1, fields:
[$0.element]) }))]
    }

    // Model holding just these fields, optionally pre-filled and pre-touched so the
    // invalid (red) state can be previewed without interacting.
    @MainActor
    public static func model(_ fields: [FormField],
        values: [String: FormValue] = [:],
        touched: [String] = []) -> DynamicFormModel {
        .preview(schema: schema(fields), values: values, touched: touched)
    }

    // Values that make section one valid – useful for previewing the unlocked
    // Welcome Offer and the enabled Next button.
    public static let validSectionOne: [String: FormValue] = [
        "username": .text("849134302"),
        "password": .text("Password1"),
        "firstname": .text("Malcolm"),
        "lastname": .text("Collin"),
        "email": .text("hi@example.com"),
    ]
}
#endif

```

42 JackpotKit/Sources/JackpotFormsUI/Fields/FieldRenderer.swift

The switch below is the whole contract between the form builder and the app.

```

import SwiftUI
import JackpotUI
import JackpotFormsDomain

// Maps a schema `fieldType` to a component. This switch is the entire contract between the
// form builder and the app: a new type on the server means a new case here, and until then
// the unknown case keeps the form usable.
//
// The `JackpotUI` components know nothing about forms; the views in this folder are the only
// place the two meet.
struct FieldRenderer: View {
    let field: FormField
    @ObservedObject var model: DynamicFormModel

    var body: some View {
        switch field.type {
            case .input:
                InputFieldView(field: field, model: model)
            case .textArea:
                TextAreaFieldView(field: field, model: model)
            case .dropdown:
                DropdownFieldView(field: field, model: model)
            case .checkbox:
                CheckboxFieldView(field: field, model: model)
            case .toggle:
                ToggleFieldView(field: field, model: model)
            case .radio, .radioGroup:
                RadioGroupFieldView(field: field, model: model)
            case .divider:
                JackpotDivider()
            case .welcomeOffer:
                WelcomeOfferFieldView(field: field, model: model)
        }
    }
}

```

```

        case .button: EmptyView() // the form's footer owns navigation
        case .recaptchaV2, .recaptchaV3: RecaptchaPlaceholderView(field: field)
        case .unknown: EmptyView() // reported via model.unsupportedFields
    }
}

/// reCAPTCHA needs a `WKWebView` bridge. The type is recognised without one so the form still
/// renders and validates around it.
struct RecaptchaPlaceholderView: View {
    let field: FormField
    @Environment(\.jackpotTheme) private var theme

    var body: some View {
        #if DEBUG
        Text("reCAPTCHA (\(field.identifier)) – not implemented")
            .jackpotTextStyle(\.error, color: \.textSecondary)
            .frame(maxWidth: .infinity, minHeight: 60)
            .jackpotFieldBackground()
        #else
        EmptyView()
        #endif
    }
}

// MARK: – Keyboard focus order

extension FormField {
    /// Only single-line text entry joins return-key navigation. A date opens a picker, and a
    /// text area needs the return key for newlines.
    var acceptsKeyboardFocus: Bool {
        type == .input && inputType != .calendar
    }
}

extension DynamicFormModel {
    /// The current section's text fields, in the order the return key walks them.
    var focusableIdentifiers: [String] {
        (currentSection?.rows ?? [])
            .flatMap(\.fields)
            .filter { $0.isVisible && $0.acceptsKeyboardFocus }
            .map(\.identifier)
    }

    /// Returns the field after `identifier`, or nil at the end so the keyboard dismisses.
    func fieldAfter(_ identifier: String?) -> String? {
        let ids = focusableIdentifiers
        guard let identifier, let index = ids.firstIndex(of: identifier) else { return nil }
        let next = ids.index(after: index)
        return next < ids.endIndex ? ids[next] : nil
    }
}

// MARK: – Shared bindings

extension DynamicFormModel {
    /// Text binding for a field, writing back as `.text`.
    func text(for field: FormField) -> Binding<String> {
        Binding(get: { self.value(for: field).stringValue },
            set: { self.setValue(.text($0), for: field) })
    }

    /// The three below mark the field touched as they write. A discrete choice is a complete
    /// answer, so validating it immediately is right – unlike typing, where `text(for:)` stays
    /// silent and the field reports blur through `onEditingEnded`.

    /// Selection binding for dropdowns and radio groups. An empty selection is `.option("")`.
    func selection(for field: FormField) -> Binding<String?> {
        Binding(get: { let v = self.value(for: field).stringValue; return v.isEmpty ? nil : v },
            set: { self.setValue(.option($0 ?? ""), for: field); self.markTouched(field) })
    }

    func bool(for field: FormField) -> Binding<Bool> {
        Binding(get: { self.value(for: field).boolValue },
            set: { self.setValue(.bool($0), for: field); self.markTouched(field) })
    }

    func date(for field: FormField) -> Binding<Date?> {
        Binding(get: { self.value(for: field).dateValue },
            set: { self.setValue($0.map(FormValue.date) ?? .empty, for: field); self.markTouched(field) })
    }
}

```

```

}

func options(for field: FormField) -> [JackpotOption] {
    field.dropdownOptions.map { JackpotOption(id: $0.value, label: localized($0.textKey)) }
}

func radioOptions(for field: FormField) -> [JackpotOption] {
    field.radioOptions.map { JackpotOption(id: $0.value, label: localized($0.textKey)) }
}
}

```

43 JackpotKit/Sources/JackpotFormsUI/Fields/InputFieldView.swift

```

import SwiftUI
import JackpotUI
import JackpotFormsDomain

struct InputFieldView: View {
    let field: FormField
    @ObservedObject var model: DynamicFormModel

    @State private var isEditing = false

    var body: some View {
        if field.inputType == .calendar {
            DateFieldView(field: field, model: model)
        } else {
            JackpotLabeledField(error: model.error(for: field)) {
                VStack(spacing: 6) {
                    JackpotTextField(model.localized(field.placeholderKey),
                                    text: model.text(for: field))
                        .onEditingEnded { model.markTouched(field) }
                        .onFocusChange { isEditing = $0 }
                        .jackpotField(kind)
                        .jackpotFieldPrefix(field.prefix)
                        .jackpotFieldSuffix(field.suffix)
                        .jackpotFieldIdentity(field.identifier)
                        .jackpotSubmitLabel(isLastFocusable ? .done : .next)
                        .disabled(field.isReadOnly)

                    // The rules are guidance while composing a password, not a permanent
                    // fixture — once the field is left the error line carries the verdict.
                    if field.isSecure, isEditing {
                        JackpotChecklist("Password Validity", items: passwordItems)
                            .transition(.scale(scale: 0.97, anchor: .top).combined(with: .opacity))
                    }
                }
                .animation(.spring(response: 0.35, dampingFraction: 0.9), value: isEditing)
            }
        }
    }

    private var isLastFocusable: Bool {
        model.focusableIdentifiers.last == field.identifier
    }

    private var passwordItems: [JackpotChecklistItem] {
        model.passwordRules(for: field).map { rule in
            JackpotChecklistItem(id: rule.id,
                                  text: rule.description,
                                  isSatisfied: rule.isSatisfied(by: model.value(for: field).stringValue))
        }
    }

    private var kind: JackpotFieldKind {
        if field.isSecure { return .newPassword }
        switch field.inputType {
        case .email: return .email
        case .phone: return .phoneNumber
        case .number: return .number
        default: break
        }
        // Falls back to the identifier, because the schema's `inputType` is coarser than the
        // autofill hints iOS can use.
        switch field.identifier.lowercased() {
        case "username", "mobile", "mobilenumber": return .phoneNumber
        case "firstname": return .givenName
        }
    }
}

```

```

        case "lastname", "surname": return .familyName
        case "email": return .email
        case "otp", "pin", "code": return .oneTimeCode
        default: return .text.with { $0.capitalization = .words }
    }
}

#if DEBUG
struct InputFieldView_Previews: PreviewProvider {
    static var previews: some View {
        Group {
            JackpotPreviewPanel("Empty · untouched") {
                InputFieldView(field: FormPreview.mobile, model: FormPreview.model([FormPreview.mobile]))
            }.previewDisplayName("Mobile - empty")
            JackpotPreviewPanel("Invalid · touched") {
                InputFieldView(field: FormPreview.mobile,
                               model: FormPreview.model([FormPreview.mobile], values: ["username": .text("123")], touched:
["username"]))
            }.previewDisplayName("Mobile - invalid")
            // The checklist is focus-gated, so it stays hidden here; focus the field in the
            // live preview to see it.
            JackpotPreviewPanel("Password · 7 chars · unfocused") {
                InputFieldView(field: FormPreview.password,
                               model: FormPreview.model([FormPreview.password], values: ["password": .text("Passwo1")],
touched: ["password"]))
            }.previewDisplayName("Password - rules hidden")
            JackpotPreviewPanel("Optional · empty is fine") {
                InputFieldView(field: FormPreview.referralCode,
                               model: FormPreview.model([FormPreview.referralCode], touched: ["referralCode"]))
            }.previewDisplayName("Referral - optional")
        }
        .previewLayout(.sizeThatFits)
    }
}
#endif

```

44 JackpotKit/Sources/JackpotFormsUI/Fields/TextAreaFieldView.swift

```

import SwiftUI
import JackpotUI
import JackpotFormsDomain

struct TextAreaFieldView: View {
    let field: FormField
    @ObservedObject var model: DynamicFormModel

    var body: some View {
        JackpotLabeledField(model.localized(field.labelKey), error: model.error(for: field)) {
            JackpotTextArea(model.localized(field.placeholderKey),
                            text: model.text(for: field))
                .onEditingEnded { model.markTouched(field) }
                .disabled(field.isReadOnly)
        }
    }
}

#if DEBUG
struct TextAreaFieldView_Previews: PreviewProvider {
    static var previews: some View {
        JackpotPreviewPanel("Empty") {
            TextAreaFieldView(field: FormPreview.notes, model: FormPreview.model([FormPreview.notes]))
        }
        .previewLayout(.sizeThatFits)
    }
}
#endif

```

45 JackpotKit/Sources/JackpotFormsUI/Fields/DropdownFieldView.swift

```

import SwiftUI
import JackpotUI
import JackpotFormsDomain

struct DropdownFieldView: View {

```



```

let field: FormField
@ObservedObject var model: DynamicFormModel

var body: some View {
    JackpotLabeledField(model.localized(field.labelKey), error: model.error(for: field)) {
        JackpotDropdown(model.localized(field.placeholderKey),
            selection: model.selection(for: field),
            options: model.options(for: field))
        .disabled(field.isReadOnly)
    }
}

#if DEBUG
struct DropdownFieldView_Previews: PreviewProvider {
    static var previews: some View {
        Group {
            JackpotPreviewPanel("Placeholder") {
                DropdownFieldView(field: FormPreview.idNumberType, model: FormPreview.model([FormPreview.idNumberType]))
            }.previewDisplayName("ID type - placeholder")
            JackpotPreviewPanel("Selected") {
                DropdownFieldView(field: FormPreview.idNumberType,
                    model: FormPreview.model([FormPreview.idNumberType], values:
["idNumberType": .option("idNumber")]))
            }.previewDisplayName("ID type - selected")
            JackpotPreviewPanel("Required · touched · empty") {
                DropdownFieldView(field: FormPreview.sourceOfFunds,
                    model: FormPreview.model([FormPreview.sourceOfFunds], touched: ["sourceOfFunds"]))
            }.previewDisplayName("Source of funds - error")
        }
        .previewLayout(.sizeThatFits)
    }
}
#endif

```

46 JackpotKit/Sources/JackpotFormsUI/Fields/RadioGroupFieldView.swift

```

import SwiftUI
import JackpotUI
import JackpotFormsDomain

struct RadioGroupFieldView: View {
    let field: FormField
    @ObservedObject var model: DynamicFormModel

    var body: some View {
        JackpotLabeledField(model.localized(field.labelKey), error: model.error(for: field)) {
            JackpotRadioGroup(selection: model.selection(for: field),
                options: model.radioOptions(for: field))
            .disabled(field.isReadOnly)
        }
    }
}

#if DEBUG
struct RadioGroupFieldView_Previews: PreviewProvider {
    static var previews: some View {
        Group {
            JackpotPreviewPanel("Nothing chosen") {
                RadioGroupFieldView(field: FormPreview.contactMethod, model: FormPreview.model([FormPreview.contactMethod]))
            }.previewDisplayName("Radio - empty")
            JackpotPreviewPanel("Chosen") {
                RadioGroupFieldView(field: FormPreview.contactMethod,
                    model: FormPreview.model([FormPreview.contactMethod], values:
["contactMethod": .option("email")]))
            }.previewDisplayName("Radio - selected")
        }
        .previewLayout(.sizeThatFits)
    }
}
#endif

```

47 JackpotKit/Sources/JackpotFormsUI/Fields/DateFieldView.swift

```

import SwiftUI
import JackpotUI
import JackpotFormsDomain

/// `inputType: "Calendar"`. The schema validates against an ISO-8601 date-time, so the
/// picker's `Date` is serialised through `FormValue.iso8601` rather than a display format.
struct DateFieldView: View {
    let field: FormField
    @ObservedObject var model: DynamicFormModel

    var body: some View {
        JackpotLabeledField(model.localized(field.labelKey), error: model.error(for: field)) {
            // The form's only age gate is the T&C checkbox; capping the picker stops an
            // under-18 date being entered at all.
            JackpotDateField(model.localized(field.placeholderKey),
                             selection: model.date(for: field),
                             in: ...model.maximumDateOfBirth)
                .disabled(field.isReadOnly)
        }
    }
}

#if DEBUG
struct DateFieldView_Previews: PreviewProvider {
    static var previews: some View {
        Group {
            JackpotPreviewPanel("No date") {
                DateFieldView(field: FormPreview.dateOfBirth, model: FormPreview.model([FormPreview.dateOfBirth]))
            }.previewDisplayName("Date - placeholder")
            JackpotPreviewPanel("Chosen") {
                DateFieldView(field: FormPreview.dateOfBirth,
                             model: FormPreview.model([FormPreview.dateOfBirth],
                                                         values: ["dateOfBirth": .date(Date(timeIntervalSince1970: 631152000))]))
            }.previewDisplayName("Date - selected")
            JackpotPreviewPanel("Required - touched - empty") {
                DateFieldView(field: FormPreview.dateOfBirth,
                             model: FormPreview.model([FormPreview.dateOfBirth], touched: ["dateOfBirth"]))
            }.previewDisplayName("Date - error")
        }
        .previewLayout(.sizeThatFits)
    }
}
#endif

```

48 JackpotKit/Sources/JackpotFormsUI/Fields/CheckboxFieldView.swift

```

import SwiftUI
import JackpotUI
import JackpotFormsDomain

/// Checkbox values validate as the strings "true"/"false" – the schema's `terms` field
/// literally uses the pattern `^true$` to mean "must be ticked".
struct CheckboxFieldView: View {
    let field: FormField
    @ObservedObject var model: DynamicFormModel

    var body: some View {
        JackpotLabeledField(error: model.error(for: field)) {
            Toggle(model.localized(field.labelKey), isOn: model.bool(for: field))
                .toggleStyle(.jackpotCheckbox)
                .disabled(field.isReadOnly)
        }
    }
}

struct ToggleFieldView: View {
    let field: FormField
    @ObservedObject var model: DynamicFormModel

    var body: some View {
        JackpotLabeledField(error: model.error(for: field)) {
            Toggle(model.localized(field.labelKey), isOn: model.bool(for: field))
                .toggleStyle(.jackpotSwitch)
                .disabled(field.isReadOnly)
        }
    }
}

```

```

    }
}

#if DEBUG
struct CheckboxFieldView_Previews: PreviewProvider {
    static var previews: some View {
        Group {
            JackpotPreviewPanel("Unticked · required · touched") {
                CheckboxFieldView(field: FormPreview.terms, model: FormPreview.model([FormPreview.terms], touched: ["terms"]))
            }.previewDisplayName("Terms – must be ticked")
            JackpotPreviewPanel("Ticked") {
                CheckboxFieldView(field: FormPreview.terms,
                                model: FormPreview.model([FormPreview.terms], values: ["terms": .bool(true)]))
            }.previewDisplayName("Terms – accepted")
            JackpotPreviewPanel("Optional · wraps") {
                CheckboxFieldView(field: FormPreview.promoOptIn, model: FormPreview.model([FormPreview.promoOptIn]))
            }.previewDisplayName("Promotions opt-in")
        }
        .previewLayout(.sizeThatFits)
    }
}
#endif

```

49 JackpotKit/Sources/JackpotFormsUI/Fields/WelcomeOfferFieldView.swift

```

import SwiftUI
import JackpotUI
import JackpotFormsDomain

/// "Welcome Offer" is a first-class field type in the builder's type list. Options come from
/// `fieldDropdowns`; the picker unlocks with `model.isFormValid`, matching the design.
struct WelcomeOfferFieldView: View {
    let field: FormField
    @ObservedObject var model: DynamicFormModel

    private var isUnlocked: Bool { model.isFormValid }
    private var selected: String { model.value(for: field).stringValue }

    var body: some View {
        VStack(spacing: 10) {
            Text(model.localized(field.labelKey))
                .jackpotTextStyle(\.sectionTitle)

            HStack(spacing: 10) {
                ForEach(model.options(for: field)) { option in
                    Button {
                        model.setValue(.option(option.id), for: field)
                        model.markTouched(field)
                    } label: {
                        Text(option.label)
                            .jackpotTextStyle(\.button)
                            .multilineTextAlignment(.center)
                    }
                    .buttonStyle(.jackpotCard)
                    .jackpotSelected(selected == option.id)
                }
            }
            .disabled(!isUnlocked)
            .jackpotLocked(!isUnlocked, message: "Complete your registration above to unlock your Welcome offer selection")

            Button("Not yet") {
                model.setValue(.option(""), for: field)
                model.markTouched(field)
            }
            .buttonStyle(.jackpot.secondary)
            .disabled(!isUnlocked)
        }
    }
}

#if DEBUG
struct WelcomeOfferFieldView_Previews: PreviewProvider {
    private static let fields = [FormPreview.mobile, FormPreview.email, FormPreview.welcomeOffer]

    static var previews: some View {
        Group {

```

```

        JackpotPreviewPanel("Locked") {
            WelcomeOfferFieldView(field: FormPreview.welcomeOffer, model: FormPreview.model(fields))
        }.previewDisplayName("Welcome offer – locked")
        JackpotPreviewPanel("Unlocked · selected") {
            WelcomeOfferFieldView(field: FormPreview.welcomeOffer, model: FormPreview.model(fields, values: [
                "username": .text("849134302"), "email": .text("hi@example.com"), "welcomeOffer": .option("depositMatch"),
            ]))
        }.previewDisplayName("Welcome offer – selected")
    }
    .previewLayout(.sizeThatFits)
}
}
#endif

```

50 JackpotKit/Sources/JackpotFormsUI/DynamicFormView.swift

```

import SwiftUI
import JackpotUI
import JackpotFormsDomain

/// A whole form, from one name and one callback.
///
///     DynamicFormView(formName: .registration) { submission in
///         try await api.register(submission.stringValues)
///     }
///
/// Everything else arrives through the environment (`.formDependencies(_)`),
/// `\.jackpotTheme(_)`), so no dependency is hidden in a global.
public struct DynamicFormView: View {

    public typealias SubmitHandler = (FormSubmission) async throws -> Void

    private let formName: FormName
    private let onSubmit: SubmitHandler

    @Environment(\.formDependencies) private var dependencies
    @Environment(\.jackpotTheme) private var theme
    @StateObject private var model: DynamicFormModel
    @State private var hasLoaded = false

    /// The two-argument form. Dependencies come from the environment.
    public init(formName: FormName, onSubmit: @escaping SubmitHandler) {
        self.formName = formName
        self.onSubmit = onSubmit
        // @StateObject cannot read @Environment in init, so a placeholder is swapped for the
        // environment's dependencies on first appearance.
        _model = StateObject(wrappedValue: DynamicFormModel(
            formName: formName,
            dependencies: FormDependencies(repository: UnavailableFormRepository()),
            isConfigured: false
        ))
    }

    /// Explicit-dependency form, for previews and tests that don't want an environment.
    public init(formName: FormName, dependencies: FormDependencies, onSubmit: @escaping SubmitHandler) {
        self.formName = formName
        self.onSubmit = onSubmit
        _model = StateObject(wrappedValue: DynamicFormModel(formName: formName, dependencies: dependencies))
    }

    public var body: some View {
        DynamicFormBody(model: model, onSubmit: onSubmit)
            .task {
                guard !hasLoaded else { return }
                hasLoaded = true
                model.configureIfNeeded(with: dependencies)
                model.load()
            }
    }
}

/// The rendered form for a given model. Separate from `DynamicFormView` so previews can drive
/// it from a seeded model without a fetch.
struct DynamicFormBody: View {
    @ObservedObject var model: DynamicFormModel
    let onSubmit: DynamicFormView.SubmitHandler
    @Environment(\.jackpotTheme) private var theme

```

```

@Environment(\.accessibilityReduceMotion) private var reduceMotion
@State private var focusedField: String?
@State private var isAdvancing = true

var body: some View {
    switch model.viewState {
    case .loading:
        ProgressView()
            .frame(maxWidth: .infinity, minHeight: 220)
            .accessibilityLabel("Loading form")

    case .failed(let message):
        JackpotErrorView(message, title: "Couldn't load this form")
            .onRetry { model.load() }

    case .loaded:
        VStack(spacing: 0) {
            if model.sections.count > 1 {
                ProgressView(value: model.progress)
                    .progressViewStyle(.jackpotBar)
                    .padding(.horizontal, 16).padding(.top, 12)
                    .accessibilityLabel("Form progress")
            }

            ScrollView {
                VStack(alignment: .leading, spacing: theme.metrics.spacing) {
                    if let section = model.currentSection {
                        ForEach(section.rows) { row in FormRowView(row: row, model: model) }
                    }
                    if let error = model.submitError {
                        Text(error).font(.footnote).jackpotForegroundColor(.error)
                            .fixedSize(horizontal: false, vertical: true)
                    }
                    #if DEBUG
                    if !model.unsupportedFields.isEmpty {
                        Text("Unsupported field types skipped: \(model.unsupportedFields.joined(separator: ", "))")
                            .font(.caption2).foregroundColor(.orange)
                    }
                    #endif
                }
                .padding(16)
                // New identity per section is what lets the transition run at all; the
                // nav bar and progress bar sit outside it so they don't slide too.
                .id(model.sectionIndex)
                .transition(sectionTransition)
            }

            FormNavigationBar(model: model,
                              onSubmit: onSubmit,
                              advance: advance,
                              goBack: goBack)
                .padding(.horizontal, 16).padding(.vertical, 12)
        }
        .jackpotBackground(.surface)
        .jackpotFocusedField($focusedField)
        // Validate before moving focus, so the error and the new focus land in one update
        // rather than the error arriving a render after the keyboard has moved on.
        .onSubmit {
            guard let current = focusedField else { return }
            model.markTouched(identifiedBy: current)
            focusedField = model.fieldAfter(current)
        }
        // Covers section changes that don't come from the nav bar.
        .onChange(of: model.sectionIndex) { _ in focusedField = nil }
    }
}

// MARK: Paging

// Direction has to be set before the index changes, not in an onChange afterwards –
// the transition is resolved in the same update that moves the section.
private func advance() {
    isAdvancing = true
    focusedField = nil
    withAnimation(pagingAnimation) { _ = model.advance() }
}

private func goBack() {
    isAdvancing = false
    focusedField = nil
}

```

```

        withAnimation(pagingAnimation) { model.goBack() }
    }

    /// Offset rather than a full `.move`, so the outgoing and incoming sections don't drag
    /// the scroll view's content width around mid-flight.
    private var sectionTransition: AnyTransition {
        guard !reduceMotion else { return .opacity }
        let travel: CGFloat = isAdvancing ? 60 : -60
        return .asymmetric(insertion: .offset(x: travel).combined(with: .opacity),
                            removal: .offset(x: -travel).combined(with: .opacity))
    }

    private var pagingAnimation: Animation {
        reduceMotion ? .easeOut(duration: 0.2)
            : .spring(response: 0.42, dampingFraction: 0.86)
    }
}

/// Fields sharing a `rowNumber` render side by side; a single field fills the row. This is
/// what makes the "+27 | Mobile Number" pairing fall out of the schema rather than being
/// special-cased.
struct FormRowView: View {
    let row: FormRow
    @ObservedObject var model: DynamicFormModel

    var body: some View {
        let visible = row.fields.filter { $0.isVisible }
        if visible.count == 1 {
            FieldRenderer(field: visible[0], model: model)
        } else if !visible.isEmpty {
            HStack(alignment: .top, spacing: 8) {
                ForEach(visible) { FieldRenderer(field: $0, model: model) }
            }
        }
    }
}

struct FormNavigationBar: View {
    @ObservedObject var model: DynamicFormModel
    let onSubmit: DynamicFormView.SubmitHandler
    let advance: () -> Void
    let goBack: () -> Void

    var body: some View {
        HStack(spacing: 12) {
            if !model.isFirstSection {
                Button("Previous", action: goBack)
                    .buttonStyle(.jackpot(.secondary))
            }
            if model.isLastSection {
                Button("Sign Up") { Task { await model.submit(onSubmit) } }
                    .buttonStyle(.jackpot)
                    .disabled(!model.isFormValid)
                    .jackpotLoading(model.isSubmitting)
            } else {
                Button("Next", action: advance)
                    .buttonStyle(.jackpot)
                    .disabled(!model.isCurrentSectionValid)
            }
        }
    }
}

// MARK: - Previews

#if DEBUG
struct DynamicFormView_Previews: PreviewProvider {
    private struct Harness: View {
        let model: DynamicFormModel
        var scheme: ColorScheme = .dark
        var body: some View {
            DynamicFormBody(model: model) { _ in }
                .frame(height: 620)
                .background(JackpotTheme.jackpotCity.colors.surface)
                .preferredColorScheme(scheme)
        }
    }

    static var previews: some View {

```

```

Group {
    Harness(model: .preview(schema: FormPreview.registration))
        .previewDisplayName("Section 1 – empty")
    Harness(model: .preview(schema: FormPreview.registration), scheme: .light)
        .previewDisplayName("Section 1 – light")
    Harness(model: .preview(schema: FormPreview.registration, values: FormPreview.validSectionOne))
        .previewDisplayName("Section 1 – valid, Next enabled")
    Harness(model: .preview(schema: FormPreview.registration,
        touched: ["username", "password", "firstname", "lastname", "email"]))
        .previewDisplayName("Section 1 – all errors shown")
    Harness(model: .preview(schema: FormPreview.registration, values: FormPreview.validSectionOne, sectionIndex: 1))
        .previewDisplayName("Section 2 – FICA")
    Harness(model: .preview(schema: FormPreview.registration, values: FormPreview.validSectionOne,
        touched: ["idNumber", "dateOfBirth", "sourceOfFunds", "terms"], sectionIndex: 1))
        .previewDisplayName("Section 2 – all errors shown")
    Harness(model: .previewLoading()).previewDisplayName("Loading")
    Harness(model: .previewFailed()).previewDisplayName("Failed")
    Harness(model: .preview(schema: FormPreview.registration))
        .environment(\.sizeCategory, .accessibilityLarge)
        .previewDisplayName("Accessibility – XL text")
}
.previewLayout(.sizeThatFits)
}
}
#endif

```

51 JackpotKit/Sources/JackpotFormsUI/Demo/PreviewSupport.swift

```

import Foundation
import SwiftUI
import JackpotFormsDomain

/// Turns the schema's keys into the copy in the designs. In production the equivalent table
/// arrives in the app-data response's `locales` section.
public extension ComposedKeyLocalizer {
    static let jpcRegistration = ComposedKeyLocalizer(table: [
        // Placeholders / labels
        "username": "Enter Mobile Number",
        "password": "Password",
        "firstname": "First Name (As it appears on your ID)",
        "lastname": "Surname (As it appears on ID)",
        "email": "Email",
        "referralCode": "I have a sign up code",
        "idNumberType": "ID Number Type",
        "idNumber": "ID Number",
        "dateOfBirth": "Enter Date Of Birth",
        "sourceOfFunds": "Enter Source Of Income",
        "receivePromotionalInformation-jza": "Send Jackpot City Promotions to me",
        "terms": "I am over 18 years of age & I accept Jackpotcity's Terms & Conditions & Privacy Policy",
        "acceptTermsConditions": "I am over 18 years of age & I accept Jackpotcity's Terms & Conditions & Privacy Policy",

        // Dropdown options
        "jpc-reg-idnumber": "South African ID",
        "jpc-reg-passport": "Passport",
        "jpc-reg-SalaryOrWages": "Salary or Wages",
        "jpc-reg-PensionOrGrant": "Pension or Grant",
        "jpc-reg-AllowanceOrBursary": "Allowance or Bursary",
        "jpc-reg-SavingsOrRentalOrOther": "Savings, Rental or Other",
        "jpc-reg-SelfEmployed": "Self Employed",

        // Composed validation messages: jpc-reg-{fieldIdentifier}-{validationMessage}
        "jpc-reg-username-regex": "Enter a valid mobile number",
        "jpc-reg-password-regex": "Password must be 8–20 characters",
        "jpc-reg-firstname-regex": "Enter your first name as it appears on your ID",
        "jpc-reg-lastname-regex": "Enter your surname as it appears on your ID",
        "jpc-reg-email-regex": "Enter a valid email address",
        "jpc-reg-referralCode-regex": "Sign up codes are 3–25 letters or numbers",
        "jpc-reg-idNumberType-regex": "Please select an ID type",
        "jpc-reg-idNumber-regex": "Enter in a valid ID number",
        "jpc-reg-dateOfBirth-regex": "Enter date of birth",
        "jpc-reg-sourceOfFunds-regex": "Please select your source of income.",
        "jpc-reg-terms-regex": "You must accept the Terms & Conditions to continue",
    ])
}

public enum FormPreviewData {
    /// `Bundle.module` is internal, so it cannot appear in a public default argument – hence

```

```

/// this overload rather than `bundle: Bundle = .module`.
public static func bundledJSON(named name: String) -> Data {
    json(named: name, in: .module)
}

public static func json(named name: String, in bundle: Bundle) -> Data {
    guard let url = bundle.url(forResource: name, withExtension: "json"),
        let data = try? Data(contentsOf: url) else {
        assertionFailure("Missing fixture \ \(name).json")
        return Data("{}".utf8)
    }
    return data
}

/// The schemas shipped with the package, keyed by `formCodeName`.
public static var bundledForms: [FormName: Data] {
    Dictionary(uniqueKeysWithValues: FormName.bundled.map { ($0, bundledJSON(named: $0.rawValue)) })
}
}

```

52 JackpotKit/Sources/JackpotFormsUI/Demo/FormSandboxView.swift

```

import SwiftUI
import JackpotFormsDomain

/// The review harness: pick a form, watch it render from JSON alone, submit it, and read back
/// exactly what the callback received. Nothing here is app-specific.
public struct FormSandboxView: View {

    public struct Sample: Identifiable, Hashable {
        public let id: FormName
        public let title: String
        public init(id: FormName, title: String) {
            self.id = id
            self.title = title
        }
    }

    private let samples: [Sample]
    private let dependencies: FormDependencies

    @State private var selected: Sample
    @State private var lastSubmission: [String: String]?
    @State private var showsSubmission = false

    public init(samples: [Sample], dependencies: FormDependencies) {
        precondition(!samples.isEmpty, "FormSandboxView needs at least one sample form")
        self.samples = samples
        self.dependencies = dependencies
        _selected = State(initialValue: samples[0])
    }

    public var body: some View {
        NavigationView {
            VStack(spacing: 0) {
                picker
                Divider()
                DynamicFormView(formName: selected.id) { submission in
                    // Deliberately not posting anywhere: the sandbox proves the callback
                    // contract, not the registration endpoint.
                    lastSubmission = submission.stringValues
                    showsSubmission = true
                }
                .id(selected.id) // rebuild the engine when the form changes
                .formDependencies(dependencies)
            }
            .navigationTitle("Form Sandbox")
            .navigationBarTitleDisplayMode(.inline)
            .sheet(isPresented: $showsSubmission) {
                SubmissionResultView(values: lastSubmission ?? [:])
            }
        }
        .navigationViewStyle(.stack)
    }

    private var picker: some View {
        Picker("Form", selection: $selected) {

```



```

        ForEach(samples) { sample in
            Text(sample.title).tag(sample)
        }
    }
    .pickerStyle(.segmented)
    .padding(12)
}

struct SubmissionResultView: View {
    let values: [String: String]
    @Environment(\.dismiss) private var dismiss

    var body: some View {
        NavigationView {
            List(values.keys.sorted(), id: \.self) { key in
                VStack(alignment: .leading, spacing: 2) {
                    Text(key).font(.caption).foregroundColor(.secondary)
                    Text(values[key]?.isEmpty == false ? values[key]! : "--")
                        .font(.body.monospaced())
                }
            }
            .navigationTitle("Submitted")
            .navigationBarTitleDisplayMode(.inline)
            .toolbar {
                ToolbarItem(placement: .confirmationAction) {
                    Button("Done") { dismiss() }
                }
            }
        }
        .navigationBarStyle(.stack)
    }
}

#if DEBUG
struct FormSandboxView_Previews: PreviewProvider {
    /// Serves the hand-built fixtures, so the sandbox previews without the bundle.
    private struct FixtureRepository: FormRepository {
        func form(named name: FormName) async throws -> FormSchema {
            try await Task.sleep(nanoseconds: 300_000_000)
            return FormPreview.registration
        }

        func submitForm(_ submission: FormSubmission) async throws -> FormSubmitResult { FormSubmitResult() }
    }

    static var previews: some View {
        FormSandboxView(
            samples: [.init(id: .registration, title: "Registration")],
            dependencies: FormDependencies(repository: FixtureRepository(),
                                           localizer: ComposedKeyLocalizer.jpcRegistration)
        )
        .preferredColorScheme(.dark)
        .previewDisplayName("Sandbox")
    }
}
#endif

```

53 JackpotKit/Tests/JackpotFormsTests/FormDecodingTests.swift

```

import XCTest
@testable import JackpotFormsData
import JackpotFormsDomain

final class FormDecodingTests: XCTestCase {

    private func loadRegistration() throws -> FormSchema {
        let url = try XCTUnwrap(Bundle.module.url(forResource: "registration", withExtension: "json"))
        return try StubFormRepository.decode(try Data(contentsOf: url))
    }

    func testDecodesTheRealRegistrationSchema() throws {
        let form = try loadRegistration()
        XCTAssertEqual(form.id, 1052)
        XCTAssertEqual(form.codeName, .registration)
        XCTAssertEqual(form.regionCode, "JZA")
        XCTAssertEqual(form.sections.count, 2)
    }
}

```

```

    XCTAssertEqual(form.allFields.count, 12)
}

func testSectionsAndRowsAreOrdered() throws {
    let form = try loadRegistration()
    XCTAssertEqual(form.sections.map(\.order), [1, 2])
    XCTAssertEqual(form.sections[0].rows.map(\.number), [1, 2, 3, 4, 5, 6])
    XCTAssertEqual(form.sections[0].fields.map(\.identifier),
        ["username", "password", "firstname", "lastname", "email", "referralCode"])
}

func testFieldTypesMapCorrectly() throws {
    let form = try loadRegistration()
    XCTAssertEqual(form.field(identifiedBy: "username")?.type, .input)
    XCTAssertEqual(form.field(identifiedBy: "username")?.inputType, .number)
    XCTAssertEqual(form.field(identifiedBy: "password")?.inputType, .password)
    XCTAssertEqual(form.field(identifiedBy: "idNumberType")?.type, .dropdown)
    XCTAssertEqual(form.field(identifiedBy: "terms")?.type, .checkbox)
}

/// The schema spells it "Calender". If the backend ever fixes the typo we must not silently
/// downgrade the date picker to a text box.
func testCalendarInputTypeAcceptsBothSpellings() {
    XCTAssertEqual(InputType(raw: "Calender"), .calendar)
    XCTAssertEqual(InputType(raw: "Calendar"), .calendar)
    XCTAssertEqual(InputType(raw: "date"), .calendar)
}

/// The most important behaviour in the decoder: a field type this build has never heard of
/// must not fail the decode, because product edits the schema in a CMS without shipping.
func testUnknownFieldTypeDoesNotFailTheDecode() throws {
    let json = Data("""
        {"formId":1,"formCodeName":"x","sections":[{"formSectionId":1,"rows":[
            {"rowNumber":1,"fields":[{"fieldId":1,"fieldIdentifier":"a","fieldType":"HolographicSlider"}]},
            {"rowNumber":2,"fields":[{"fieldId":2,"fieldIdentifier":"b","fieldType":"Input"}]}
        ]}}
        """, .utf8)
    let form = try StubFormRepository.decode(json)
    XCTAssertEqual(form.allFields.count, 2)
    XCTAssertEqual(form.field(identifiedBy: "a")?.type, .unknown("HolographicSlider"))
    XCTAssertEqual(form.field(identifiedBy: "b")?.type, .input)
}

func testMissingOptionalKeysFallBackSafely() throws {
    let json = Data("""
        {"formId":2,"formCodeName":"y","sections":[{"formSectionId":1,"rows":[
            {"rowNumber":1,"fields":[{"fieldId":9,"fieldIdentifier":"solo","fieldType":"Input"}]}]}
        """, .utf8)
    let field = try XCTUnwrap(StubFormRepository.decode(json).field(identifiedBy: "solo"))
    XCTAssertEqual(field.labelKey, "solo")
    XCTAssertEqual(field.placeholderKey, "solo")
    XCTAssertEqual(field.validationMessageKey, "regex")
    XCTAssertFalse(field.isRequired) // unspecified must not block submission
    XCTAssertTrue(field.isVisible)
    XCTAssertNil(field.regex)
}

func testDropdownOptionsCarryValueTextAndRegex() throws {
    let form = try loadRegistration()
    let options = try XCTUnwrap(form.field(identifiedBy: "idNumberType")?.dropdownOptions)
    XCTAssertEqual(options.map(\.value), ["idNumber", "passport"])
    XCTAssertEqual(options[0].textKey, "jpc-reg-idnumber")
    XCTAssertEqual(options[0].regex, "idNumberRegex")
}

final class FormNameTests: XCTestCase {

    func testKnownNamesMapToTheirCodeNames() {
        XCTAssertEqual(FormName.registration.rawValue, "registration")
        XCTAssertEqual(FormName.kitchenSink.rawValue, "kitchenSink")
    }

    /// Forms are authored server-side, so a name this build has never heard of must still be
    /// constructible – that's why `FormName` isn't an enum.
    func testServerAuthoredNamesAreConstructible() {
        let deposit = FormName("deposit")
        XCTAssertEqual(deposit.rawValue, "deposit")
        XCTAssertNotEqual(deposit, .registration)
    }
}

```

```

func testUsableAsADictionaryKey() {
    let forms: [FormName: Int] = [.registration: 1, FormName("deposit"): 2]
    XCTAssertEqual(forms[.registration], 1)
    XCTAssertEqual(forms[FormName("deposit")], 2)
    XCTAssertNil(forms[.kitchenSink])
}

func testRoundTripsThroughRawValue() {
    XCTAssertEqual(FormName(rawValue: "registration"), .registration)
}

func testBundledListMatchesTheShippedResources() {
    XCTAssertEqual(Set(FormName.bundled), [.registration, .kitchenSink])
}
}

```

54 JackpotKit/Tests/JackpotFormsTests/FieldValidatorTests.swift

```

import XCTest
@testable import JackpotFormsData
import JackpotFormsDomain

final class FieldValidatorTests: XCTestCase {

    private let validator = FieldValidator()

    private func field(_ identifier: String,
                      type: FieldType = .input,
                      inputType: InputType = .text,
                      required: Bool = true,
                      regex: String?) -> FormField {
        FormField(id: 1, identifier: identifier, name: identifier, labelKey: identifier,
                  type: type, inputType: inputType, textStyle: "Regular",
                  validationMessageKey: "regex", isRequired: required, isVisible: true, isReadOnly: false,
                  regex: regex, prefix: "", suffix: "", placeholderKey: identifier,
                  dropdownOptions: [], radioOptions: [])
    }

    // MARK: Real patterns from the registration schema

    func testMobileNumberPattern() {
        let mobile = field("username", inputType: .number, regex: "^(27|0)?[1-9][0-9]{8}$")
        XCTAssertTrue(validator.validate(.text("849134302"), against: mobile).isValid)
        XCTAssertTrue(validator.validate(.text("0849134302"), against: mobile).isValid)
        XCTAssertTrue(validator.validate(.text("27849134302"), against: mobile).isValid)
        XCTAssertFalse(validator.validate(.text("049134302"), against: mobile).isValid) // leading 0 after prefix
        XCTAssertFalse(validator.validate(.text("12345"), against: mobile).isValid)
    }

    func testPasswordLengthPattern() {
        let password = field("password", inputType: .password, regex: "^(.){8,20}$")
        XCTAssertFalse(validator.validate(.text("short"), against: password).isValid)
        XCTAssertTrue(validator.validate(.text("longenough"), against: password).isValid)
        XCTAssertFalse(validator.validate(.text(String(repeating: "a", count: 21)), against: password).isValid)
    }

    func testSouthAfricanIdPattern() {
        let id = field("idNumber", regex: "^([0-9]){13}$")
        XCTAssertTrue(validator.validate(.text("9001015800089"), against: id).isValid)
        XCTAssertFalse(validator.validate(.text("900101580008"), against: id).isValid)
    }

    // MARK: Required / optional

    func testRequiredEmptyFails() {
        XCTAssertFalse(validator.validate(.text(""), against: field("a", regex: "^.+$")).isValid)
        XCTAssertFalse(validator.validate(.text(" "), against: field("a", regex: "^.+$")).isValid)
    }

    func testOptionalEmptyPassesEvenWhenTheRegexWouldNot() {
        // referralCode's own regex allows empty, but many optional fields' won't -
        // the empty short-circuit is what makes "optional" mean optional.
        let optional = field("referralCode", required: false, regex: "[a-zA-Z0-9]{3,25}$")
        XCTAssertTrue(validator.validate(.text(""), against: optional).isValid)
        XCTAssertFalse(validator.validate(.text("ab"), against: optional).isValid)
        XCTAssertTrue(validator.validate(.text("WELCOME50"), against: optional).isValid)
    }
}

```

```

}

// MARK: Checkboxes validate as strings

func testRequiredCheckboxMustBeTicked() {
    let terms = field("terms", type: .checkbox, regex: "^true$")
    XCTAssertFalse(validator.validate(.bool(false), against: terms).isValid)
    XCTAssertTrue(validator.validate(.bool(true), against: terms).isValid)
}

// MARK: Server-supplied patterns are untrusted input

func testMalformedServerPatternDoesNotBlockTheUser() {
    let broken = field("oops", regex: "[a-z]" // will not compile
    XCTAssertTrue(validator.validate(.text("anything"), against: broken).isValid)
}

func testMissingPatternIsNoConstraint() {
    XCTAssertTrue(validator.validate(.text("anything"), against: field("a", regex: nil)).isValid)
    XCTAssertTrue(validator.validate(.text("anything"), against: field("a", regex: "")).isValid)
}

func testReadOnlyAndInvisibleFieldsAreNeverInvalid() {
    var f = field("a", regex: "^impossible$")
    f = FormField(id: f.id, identifier: f.identifier, name: f.name, labelKey: f.labelKey,
        type: f.type, inputType: f.inputType, textStyle: f.textStyle,
        validationMessageKey: f.validationMessageKey, isRequired: true,
        isVisible: false, isReadOnly: false, regex: f.regex,
        prefix: "", suffix: "", placeholderKey: "", dropdownOptions: [], radioOptions: [])
    XCTAssertTrue(validator.validate(.text(""), against: f).isValid)
}

// MARK: Named regexes (the ID-type → ID-number dependency)

func testNamedRegexResolvesFromTheCatalogue() {
    XCTAssertEqual(validator.optionPattern("idNumberRegex"), "[0-9]{13}$")
    XCTAssertEqual(validator.optionPattern("passportNumberRegex"), ".{5,20}$")
}

/// Passport is a length rule, not a character class. Too short fails; any 5–20
/// characters pass – including punctuation the old alphanumeric guess would have rejected.
func testPassportPatternIsACharacterCount() {
    let pattern = validator.optionPattern("passportNumberRegex")!
    let expression = try! NSRegularExpression(pattern: pattern)
    func matches(_ value: String) -> Bool {
        expression.firstMatch(in: value, range: NSRange(value.startIndex..., in: value)) != nil
    }
    XCTAssertTrue(matches("A1234567"))
    XCTAssertTrue(matches("123456789"))
    XCTAssertTrue(matches("!!!!!!!!"))
    XCTAssertFalse(matches(""))
    XCTAssertFalse(matches("abc"))
    XCTAssertFalse(matches(String(repeating: "x", count: 21)))
}

func testLiteralPatternIsUsedAsIs() {
    XCTAssertEqual(validator.optionPattern("[a-zA-Z]"), "[a-zA-Z]")
}

func testOverrideRegexReplacesTheFieldRule() {
    let id = field("idNumber", regex: "[0-9]{13}$")
    // Passport selected → 13-digit rule no longer applies.
    XCTAssertTrue(validator.validate(.text("A1234567"), against: id,
        overrideRegex: "[a-zA-Z0-9]{6,12}$").isValid)
    XCTAssertFalse(validator.validate(.text("A1234567"), against: id).isValid)
}

// MARK: Password policy parsing

func testPasswordRulesAreDerivedFromTheQuantifier() {
    let rules = PasswordPolicy().rules(for: field("password", inputType: .password, regex: "^(.){8,20}$"))
    XCTAssertEqual(rules.count, 2)
    XCTAssertEqual(rules[0].description, "Minimum of 8 characters")
    XCTAssertEqual(rules[1].description, "Maximum of 20 characters")
    XCTAssertFalse(rules[0].isSatisfied(by: "short"))
    XCTAssertTrue(rules[0].isSatisfied(by: "longenough"))
    XCTAssertFalse(rules[1].isSatisfied(by: String(repeating: "a", count: 21)))
}

func testDateSerialisesToTheIso8601ShapeTheSchemaExpects() {

```

```

let pattern = "^((\\d{4})-(\\d{2})-(\\d{2})T(\\d{2}):(\\d{2}):(\\d{2}(?:\\.|\\d*)?)(\\((-(\\d{2}):(\\d{2})|Z)?\\))?$"
let dob = field("dateOfBirth", inputType: .calendar, regex: pattern)
XCTAssertTrue(validator.validate(.date(Date(timeIntervalSince1970: 631152000)), against: dob).isValid)
}
}

```

55 JackpotKit/Tests/JackpotFormsTests/DynamicFormModelTests.swift

The behaviour a user experiences: untouched fields stay silent, Next reveals every error at once, section gating, submit blocked while invalid, and the payload keyed by `fieldIdentifier`.

```

import Combine
import XCTest
@testable import JackpotFormsUI
import JackpotFormsData
import JackpotFormsDomain

/// End-to-end validation behaviour against the real registration schema: what the user
/// actually experiences, rather than the regexes in isolation.
@MainActor
final class DynamicFormModelTests: XCTestCase {

    private func makeModel(delay: TimeInterval = 0) -> DynamicFormModel {
        let url = Bundle.module.url(forResource: "registration", withExtension: "json")!
        let data = try! Data(contentsOf: url)
        return DynamicFormModel(
            formName: .registration,
            dependencies: FormDependencies(
                repository: StubFormRepository(forms: [.registration: data], delay: delay)
            )
        )
    }

    private func loaded() async throws -> DynamicFormModel {
        let model = makeModel()
        model.load()
        try await waitUntil { model.form != nil }
        return model
    }

    private func waitUntil(timeout: TimeInterval = 2,
        _ condition: @MainActor () -> Bool) async throws {
        let deadline = Date().addingTimeInterval(timeout)
        while !condition() {
            if Date() > deadline { XCTFail("Timed out"); return }
            try await Task.sleep(nanoseconds: 5_000_000)
        }
    }

    private func field(_ model: DynamicFormModel, _ id: String) throws -> FormField {
        try XCTUnwrap(model.form?.field(identifiedBy: id))
    }

    // MARK: Loading

    func testLoadsSchemaAndSeedsEveryField() async throws {
        let model = try await loaded()
        XCTAssertEqual(model.sections.count, 2)
        XCTAssertEqual(model.sectionIndex, 0)
        XCTAssertTrue(model.isFirstSection)
        XCTAssertFalse(model.isLastSection)
        // Checkboxes start false, not empty – so `terms` is correctly invalid up front.
        XCTAssertEqual(model.value(for: try field(model, "terms")), .bool(false))
    }

    // MARK: Errors appear only after the user has engaged

    func testUntouchedFieldsShowNoErrorEvenWhenInvalid() async throws {
        let model = try await loaded()
        let mobile = try field(model, "username")
        XCTAssertNil(model.error(for: mobile)) // empty + required, but untouched
        model.markTouched(mobile)
        XCTAssertNotNil(model.error(for: mobile))
    }

    func testAdvancingRevealsEveryErrorInTheSection() async throws {

```

```

    let model = try await loaded()
    XCTAssertFalse(model.advance()) // blocked
    XCTAssertEqual(model.sectionIndex, 0)
    for id in ["username", "password", "firstname", "lastname", "email"] {
        XCTAssertNotNil(model.error(for: try field(model, id)), "\(id) should show an error")
    }
    // The optional referral code must NOT be flagged.
    XCTAssertNil(model.error(for: try field(model, "referralCode")))
}

// MARK: Section gating

func testCannotAdvanceUntilSectionOneIsValid() async throws {
    let model = try await loaded()
    XCTAssertFalse(model.isCurrentSectionValid)

    try fillSectionOne(model)

    XCTAssertTrue(model.isCurrentSectionValid)
    XCTAssertTrue(model.advance())
    XCTAssertEqual(model.sectionIndex, 1)
    XCTAssertTrue(model.isLastSection)

    model.goBack()
    XCTAssertEqual(model.sectionIndex, 0)
}

// MARK: The ID-type → ID-number dependency

func testPassportSelectionRelaxesTheThirteenDigitIdRule() async throws {
    let model = try await loaded()
    try fillSectionOne(model)
    _ = model.advance()

    let type = try field(model, "idNumberType")
    let number = try field(model, "idNumber")

    model.setValue(.option("idNumber"), for: type)
    model.setValue(.text("A1234567"), for: number)
    model.markTouched(number)
    XCTAssertNotNil(model.error(for: number), "SA ID must be 13 digits")

    model.setValue(.option("passport"), for: type)
    XCTAssertNil(model.error(for: number), "passport should accept an alphanumeric number")

    model.setValue(.option("idNumber"), for: type)
    XCTAssertNotNil(model.error(for: number), "switching back must re-apply the 13-digit rule")
}

// MARK: Submission

func testSubmitIsBlockedWhileAnythingIsInvalid() async throws {
    let model = try await loaded()
    var called = false
    await model.submit { _ in called = true }
    XCTAssertFalse(called)
}

func testSubmitDeliversEveryValueKeyedByFieldIdentifier() async throws {
    let model = try await loaded()
    try fillSectionOne(model)
    _ = model.advance()
    try fillSectionTwo(model)

    XCTAssertTrue(model.isFormValid)

    var received: FormSubmission?
    await model.submit { received = $0 }

    let submission = try XCTUnwrap(received)
    XCTAssertEqual(submission.formCodeName, .registration)
    XCTAssertEqual(submission.formId, "1052")
    XCTAssertEqual(submission["username"].stringValue, "849134302")
    XCTAssertEqual(submission["firstname"].stringValue, "Malcolm")
    XCTAssertEqual(submission["idNumberType"].stringValue, "idNumber")
    XCTAssertEqual(submission["terms"].stringValue, "true")
    XCTAssertEqual(submission["receivePromotionalInformation"].stringValue, "false")
    // Untouched optional field still present, as an empty string.
    XCTAssertEqual(submission["referralCode"].stringValue, "")
}

```

```

func testSubmitSurfacesAThrownError() async throws {
    let model = try await loaded()
    try fillSectionOne(model)
    _ = model.advance()
    try fillSectionTwo(model)

    struct Boom: LocalizedError { var errorDescription: String? { "Registration failed" } }
    await model.submit { _ in throw Boom() }
    XCTAssertEqual(model.submitError, "Registration failed")
}

// MARK: Progress

func testProgressTracksSatisfiedRequiredFields() async throws {
    let model = try await loaded()
    XCTAssertEqual(model.progress, 0, accuracy: 0.001)
    try fillSectionOne(model)
    XCTAssertGreaterThan(model.progress, 0.3)
    XCTAssertLessThan(model.progress, 1.0)
    _ = model.advance()
    try fillSectionTwo(model)
    XCTAssertEqual(model.progress, 1.0, accuracy: 0.001)
}

// MARK: Helpers

private func fillSectionOne(_ model: DynamicFormModel) throws {
    model.setValue(.text("849134302"), for: try field(model, "username"))
    model.setValue(.text("Password1"), for: try field(model, "password"))
    model.setValue(.text("Malcolm"), for: try field(model, "firstname"))
    model.setValue(.text("Collin"), for: try field(model, "lastname"))
    model.setValue(.text("hi@example.com"), for: try field(model, "email"))
}

private func fillSectionTwo(_ model: DynamicFormModel) throws {
    model.setValue(.option("idNumber"), for: try field(model, "idNumberType"))
    model.setValue(.text("9001015800089"), for: try field(model, "idNumber"))
    model.setValue(.date(Date(timeIntervalSince1970: 631152000)), for: try field(model, "dateOfBirth"))
    model.setValue(.option("SalaryOrWages"), for: try field(model, "sourceOfFunds"))
    model.setValue(.bool(true), for: try field(model, "terms"))
}

// MARK: Render scheduling

/// The return key marks the field on submit and the blur that follows marks it again.
/// A second render there is what made the error appear a beat after focus moved.
func testReTouchingAFieldSchedulesNoFurtherRender() async throws {
    let model = try await loaded()
    let mobile = try field(model, "username")
    var renders = 0
    let subscription = model.objectWillChange.sink { _ in renders += 1 }
    defer { subscription.cancel() }

    model.markTouched(mobile)
    XCTAssertEqual(renders, 1)
    model.markTouched(mobile)
    XCTAssertEqual(renders, 1)
}

func testTouchingByIdentifierRevealsTheErrorAndIgnoresUnknownFields() async throws {
    let model = try await loaded()
    let mobile = try field(model, "username")
    model.setValue(.text("123"), for: mobile)
    XCTAssertNil(model.error(for: mobile), "untouched, so still silent")

    model.markTouched(identifiedBy: "username")
    XCTAssertNotNil(model.error(for: mobile))

    model.markTouched(identifiedBy: "notAField") // must not trap
}

// MARK: Return-key focus order

func testFocusOrderCoversOnlyTextEntryInSchemaOrder() async throws {
    let model = try await loaded()
    XCTAssertEqual(model.focusableIdentifiers,
        ["username", "password", "firstname", "lastname", "email", "referralCode"])
}

```

```

func testReturnWalksToTheNextFieldAndStopsAtTheEnd() async throws {
    let model = try await loaded()
    XCTAssertEqual(model.fieldAfter("username"), "password")
    XCTAssertEqual(model.fieldAfter("email"), "referralCode")
    XCTAssertNil(model.fieldAfter("referralCode"), "The last field dismisses rather than wrapping")
}

func testFocusOrderIgnoresUnknownAndAbsentFields() async throws {
    let model = try await loaded()
    XCTAssertNil(model.fieldAfter(nil))
    XCTAssertNil(model.fieldAfter("notAField"))
}

func testDateAndPickerFieldsAreNotKeyboardFocusable() async throws {
    let model = try await loaded()
    for identifier in ["dateOfBirth", "idNumberType", "sourceOfFunds", "terms"] {
        XCTAssertFalse(try field(model, identifier).acceptsKeyboardFocus,
            "\\(identifier) opens a picker or toggles, so the return key should skip it")
    }
}

}

/// Confirmed behaviour: the ID Number Type dropdown selects whether the user is entering a
/// South African ID or a passport, and the ID Number field validates accordingly.
///
/// The link is declared in `FormDependencies.regexDependencies` rather than inferred from
/// field order, so a CRM reorder can't silently disable it on a regulated field.
@MainActor
final class IDTypeRegexDependencyTests: XCTestCase {

    private func loadedSectionTwo(
        regexDependencies: [String: String] = ["idNumberType": "idNumber"],
        appliesOptionRegex: Bool = true
    ) async throws -> DynamicFormModel {
        let url = Bundle.module.url(forResource: "registration", withExtension: "json")!
        let data = try Data(contentsOf: url)
        let model = DynamicFormModel(
            formName: .registration,
            dependencies: FormDependencies(
                repository: StubFormRepository(forms: [.registration: data], delay: 0),
                appliesOptionRegexToDependentField: appliesOptionRegex,
                regexDependencies: regexDependencies
            )
        )
        model.load()
        let deadline = Date().addingTimeInterval(2)
        while model.form == nil {
            if Date() > deadline { XCTFail("timed out"); break }
            try await Task.sleep(nanoseconds: 5_000_000)
        }
        return model
    }

    private func field(_ model: DynamicFormModel, _ id: String) throws -> FormField {
        try XCTUnwrap(model.form?.field(identifiedBy: id))
    }

    func testSouthAfricanIDRequiresThirteenDigits() async throws {
        let model = try await loadedSectionTwo()
        let type = try field(model, "idNumberType"), number = try field(model, "idNumber")

        model.setValue(.option("idNumber"), for: type)
        model.setValue(.text("9001015800089"), for: number)
        model.markTouched(number)
        XCTAssertNil(model.error(for: number))

        model.setValue(.text("A1234567"), for: number)
        XCTAssertNotNil(model.error(for: number), "a passport number is not a valid SA ID")
    }

    func testPassportAcceptsAnAlphanumericNumber() async throws {
        let model = try await loadedSectionTwo()
        let type = try field(model, "idNumberType"), number = try field(model, "idNumber")

        model.setValue(.option("passport"), for: type)
        model.setValue(.text("A1234567"), for: number)
        model.markTouched(number)
        XCTAssertNil(model.error(for: number))
    }
}

```



```

/// Switching type revalidates immediately – the user shouldn't have to re-type to see the
/// rule change.
func testSwitchingTypeRevalidatesWithoutRetyping() async throws {
    let model = try await loadedSectionTwo()
    let type = try field(model, "idNumberType"), number = try field(model, "idNumber")

    model.setValue(.option("idNumber"), for: type)
    model.setValue(.text("A1234567"), for: number)
    model.markTouched(number)
    XCTAssertNotNil(model.error(for: number))

    model.setValue(.option("passport"), for: type)
    XCTAssertNil(model.error(for: number), "switching to passport must clear the error")

    model.setValue(.option("idNumber"), for: type)
    XCTAssertNotNil(model.error(for: number), "switching back must re-apply the 13-digit rule")
}

/// A literal pattern on a dropdown option describes the *selection*, not another field.
/// `sourceOfFunds` options carry `[a-zA-Z]`; that must never leak onto its neighbour.
func testLiteralOptionPatternsDoNotLeakOntoOtherFields() async throws {
    let model = try await loadedSectionTwo()
    let source = try field(model, "sourceOfFunds")
    let promo = try field(model, "receivePromotionalInformation")

    model.setValue(.option("SalaryOrWages"), for: source)
    model.setValue(.bool(true), for: promo)
    model.markTouched(promo)
    XCTAssertNil(model.error(for: promo))
}

/// With the link declared, the rule survives a schema reorder. Without it, resolution falls
/// back to field order – which is exactly the fragility the declaration removes.
func testExplicitLinkIsNotPositional() async throws {
    let model = try await loadedSectionTwo(regexDependencies: ["idNumberType": "idNumber"])
    let type = try field(model, "idNumberType"), number = try field(model, "idNumber")
    model.setValue(.option("passport"), for: type)
    model.setValue(.text("A1234567"), for: number)
    model.markTouched(number)
    XCTAssertNil(model.error(for: number))
}

func testCanBeDisabledEntirely() async throws {
    let model = try await loadedSectionTwo(applyOptionRegex: false)
    let type = try field(model, "idNumberType"), number = try field(model, "idNumber")
    model.setValue(.option("passport"), for: type)
    model.setValue(.text("A1234567"), for: number)
    model.markTouched(number)
    XCTAssertNotNil(model.error(for: number), "disabled → the field's own regex always applies")
}
}

```

56

```

cd JackpotKit
xcodebuild -scheme JackpotKit-Package -destination 'platform=iOS Simulator,name=iPhone 17 Pro' test

```

► Create PR — JackpotForms

JackpotFormsTests: Executed 48 tests, with 0 failures

Open `DynamicFormView.swift` and resume the whole-form previews.

PR 3 — JackpotNetworking + the live repository

The transport, then the repository built on it. `JackpotFormsUI` is untouched in this PR — that is the point of the protocol. `JackpotForms` arrives as the composition target: it is the only module that sees both the network and the UI, so it is the only one that has to change when the wiring does.

57

```
mkdir -p JackpotKit/Sources/{JackpotNetworking,JackpotLocalization,JackpotFormsRemote,JackpotForms}
mkdir -p JackpotKit/Tests/{JackpotNetworkingTests,JackpotLocalizationTests}
```

58 `JackpotKit/Package.swift` — add to `products:` and `targets:`

`JackpotFormsRemote` and `JackpotForms` join the `JackpotForms` library product, and `JackpotFormsTests` gains all four new targets as dependencies.

```
.library(name: "JackpotNetworking", targets: ["JackpotNetworking"]),
.library(name: "JackpotLocalization", targets: ["JackpotLocalization"]),

.target(name: "JackpotNetworking"),
.target(name: "JackpotLocalization"),
.testTarget(name: "JackpotNetworkingTests", dependencies: ["JackpotNetworking"]),
.testTarget(name: "JackpotLocalizationTests", dependencies: ["JackpotLocalization"]),

.target(
    name: "JackpotFormsRemote",
    dependencies: ["JackpotFormsDomain", "JackpotFormsData", "JackpotNetworking"]
),
.target(
    name: "JackpotForms",
    dependencies: [
        "JackpotFormsDomain",
        "JackpotFormsData",
        "JackpotFormsUI",
        "JackpotFormsRemote",
        "JackpotLocalization",
        "JackpotNetworking",
    ]
),
```

59 `JackpotKit/Sources/JackpotNetworking/HTTPMethod.swift`

```
import Foundation

public enum HTTPMethod: String, Sendable {
    case GET, POST, PUT, PATCH, DELETE
}
```

60 `JackpotKit/Sources/JackpotNetworking/HTTPClient.swift`

```
import Foundation

/// The transport seam. Tests stub this — never the ApiClient itself.
public protocol HTTPClient: Sendable {
    func send(_ request: URLRequest) async throws -> (Data, HTTPURLResponse)
}

/// A wrapper rather than a `URLSession` conformance, so the conformance stays private to
/// this module and configuration has somewhere to live.
public struct URLSessionHTTPClient: HTTPClient {
    private let session: URLSession

    public init(session: URLSession = .shared) {
        self.session = session
    }
}
```

```

public init(timeout: TimeInterval, waitForConnectivity: Bool = false) {
    let configuration = URLSessionConfiguration.default
    configuration.timeoutIntervalForRequest = timeout
    configuration.waitForConnectivity = waitForConnectivity
    self.session = URLSession(configuration: configuration)
}

public func send(_ request: URLRequest) async throws -> (Data, HTTPURLResponse) {
    let (data, response) = try await session.data(for: request)
    guard let http = response as? HTTPURLResponse else {
        throw URLError(.badServerResponse)
    }
    return (data, http)
}
}

```

61 JackpotKit/Sources/JackpotNetworking/APIEnvironment.swift

```

import Foundation

/// Base URL + headers every request in an environment carries. Injected once, so the
/// layer can serve a second host or a staging environment without touching `APIEndpoint`.
public struct APIEnvironment: Sendable {
    public let baseURL: URL
    public let defaultHeaders: [String: String]
    /// Query items appended to every request (api-version, site, locale...).
    public let defaultQueryItems: [URLQueryItem]

    public init(baseURL: URL,
                defaultHeaders: [String: String] = ["Accept": "application/json"],
                defaultQueryItems: [URLQueryItem] = []) {
        self.baseURL = baseURL
        self.defaultHeaders = defaultHeaders
        self.defaultQueryItems = defaultQueryItems
    }
}

```

62 JackpotKit/Sources/JackpotNetworking/APIEndpoint.swift

```

import Foundation

public enum RequestBody: Sendable {
    case json(Data)
    case form([String: String])
}

/// One endpoint = one request shape. Knows its own path, method and body;
/// knows nothing about hosts, auth or versioning.
public protocol APIEndpoint: Sendable {
    var path: String { get }
    var method: HTTPMethod { get }
    var queryItems: [URLQueryItem] { get }
    var headers: [String: String] { get }
    var body: RequestBody? { get }
    /// False for login and token-refresh endpoints, so auth interceptors skip them -
    /// a refresh request must never carry the token it is replacing.
    var requiresAuth: Bool { get }
    /// Safe to retry on 5xx / timeout.
    var isIdempotent: Bool { get }
}

public extension APIEndpoint {
    var method: HTTPMethod { .GET }
    var queryItems: [URLQueryItem] { [] }
    var headers: [String: String] { [:] }
    var body: RequestBody? { nil }
    var requiresAuth: Bool { true }
    var isIdempotent: Bool { method == .GET }
}

func urlRequest(in environment: APIEnvironment) throws -> URLRequest {
    let trimmed = path.hasPrefix("/") ? String(path.dropFirst()) : path
    guard var components = URLComponents(
        url: environment.baseURL.appendingPathComponent(trimmed),
        resolvingAgainstBaseURL: false
    ) else {

```

```

        throw APIError.invalidURL(path)
    }

    let allQuery = environment.defaultQueryItems + queryItems
    if !allQuery.isEmpty { components.queryItems = allQuery }

    guard let url = components.url else { throw APIError.invalidURL(path) }

    var request = URLRequest(url: url)
    request.httpMethod = method.rawValue
    for (key, value) in environment.defaultHeaders { request.setValue(value, forHTTPHeaderField: key) }
    for (key, value) in headers { request.setValue(value, forHTTPHeaderField: key) }

    switch body {
    case .json(let data):
        request.httpBody = data
        request.setValue("application/json", forHTTPHeaderField: "Content-Type")
    case .form(let fields):
        var form = URLComponents()
        form.queryItems = fields.map { URLQueryItem(name: $0.key, value: $0.value) }
        request.httpBody = form.percentEncodedQuery?.data(using: .utf8)
        request.setValue("application/x-www-form-urlencoded", forHTTPHeaderField: "Content-Type")
    case nil:
        break
    }
    return request
}
}

```

63 JackpotKit/Sources/JackpotNetworking/APIError.swift

```

import Foundation

/// The error envelope the API returns with a non-2xx response: `{ "code": 0, "message": "..." }`
///
/// `code` decodes from both number and string forms. Decoding is `try?` at the call site by
/// design – a gateway may return HTML, and that must not throw – so a shape mismatch here
/// would be invisible: no crash, no log, just permanently empty error messages.
public struct APIProblem: Decodable, Sendable, Equatable {
    public let code: Int?
    public let message: String?

    public init(code: Int?, message: String?) {
        self.code = code
        self.message = message
    }

    private enum CodingKeys: String, CodingKey { case code, message }

    public init(from decoder: any Decoder) throws {
        let container = try decoder.container(keyedBy: CodingKeys.self)

        let decodedCode: Int?
        if let number = try? container.decodeIfPresent(Int.self, forKey: .code) {
            decodedCode = number
        } else {
            decodedCode = (try? container.decodeIfPresent(String.self, forKey: .code)).flatMap { $0.flatMap(Int.init) }
        }
        let decodedMessage = (try? container.decodeIfPresent(String.self, forKey: .message))?.flatMap {
            $0.isEmpty ? nil : $0
        }

        // A body carrying neither field is not a problem envelope. Throwing here means the
        // client's `try?` yields nil, so `.badRequest(nil)` reads "the server said nothing"
        // rather than "the server sent an empty complaint".
        guard decodedCode != nil || decodedMessage != nil else {
            throw DecodingError.dataCorrupted(.init(
                codingPath: decoder.codingPath,
                debugDescription: "Not a problem envelope: neither `code` nor `message` present"
            ))
        }

        code = decodedCode
        message = decodedMessage
    }
}

```

```

/// What the client concluded happened.
///
/// The API contract is 200, 400, 401, 500. Anything else can still arrive from a proxy,
/// gateway or WAF, so `unexpectedStatus` carries it rather than flattening it into `.server`.
public enum APIError: Error, Sendable, Equatable {
    case invalidURL(String)
    case transport(URLError.Code)
    /// 400 – the request was rejected. `problem.message` is the text to show.
    case badRequest(APIProblem?)
    /// 401 – after any interceptor has had its one chance to refresh and retry.
    case unauthorized(APIProblem?)
    /// 500 – after one retry, if the endpoint was idempotent.
    case server(APIProblem?)
    /// Outside the documented contract. Almost always infrastructure rather than the API.
    case unexpectedStatus(Int, APIProblem?)
    case decoding(String)
    case cancelled

    /// The server's envelope, whichever case carries it.
    public var problem: APIProblem? {
        switch self {
            case .badRequest(let p), .unauthorized(let p), .server(let p), .unexpectedStatus(_, let p):
                return p
            case .invalidURL, .transport, .decoding, .cancelled:
                return nil
        }
    }

    /// The server's own wording, when it sent any. Prefer this over a generic string.
    public var serverMessage: String? {
        problem?.message.flatMap { $0.isEmpty ? nil : $0 }
    }

    public var isOffline: Bool {
        guard case .transport(let code) = self else { return false }
        return [.notConnectedToInternet, .networkConnectionLost, .dataNotAllowed, .timedOut].contains(code)
    }
}

```

64 JackpotKit/Sources/JackpotNetworking/RequestInterceptor.swift

```

import Foundation

/// Where auth headers, token refresh, logging and correlation IDs belong – once,
/// instead of in every API type.
public protocol RequestInterceptor: Sendable {
    func adapt(_ request: URLRequest, for endpoint: any APIEndpoint) async throws -> URLRequest
    /// Return an adapted request to retry with, or nil to give up. Called at most once.
    func retry(_ request: URLRequest,
               for endpoint: any APIEndpoint,
               response: HTTPURLResponse,
               data: Data) async -> URLRequest?
}

public extension RequestInterceptor {
    func adapt(_ request: URLRequest, for endpoint: any APIEndpoint) async throws -> URLRequest { request }
    func retry(_ request: URLRequest,
               for endpoint: any APIEndpoint,
               response: HTTPURLResponse,
               data: Data) async -> URLRequest? { nil }
}

/// Adds a bearer token. The token arrives through a closure so this module never
/// imports a session type.
public struct BearerTokenInterceptor: RequestInterceptor {
    private let token: @Sendable () async -> String?

    public init(token: @escaping @Sendable () async -> String?) {
        self.token = token
    }

    public func adapt(_ request: URLRequest, for endpoint: any APIEndpoint) async throws -> URLRequest {
        guard endpoint.requiresAuth, let token = await token() else { return request }
        var request = request
        request.setValue("Bearer \(token)", forHTTPHeaderField: "Authorization")
        return request
    }
}

```

```

    }
}

```

65 JackpotKit/Sources/JackpotNetworking/ConditionalRequest.swift

ETag / Last-Modified handling. Nothing in this PR sends a conditional request; it is here because `APIError` and the client have to agree on what a 304 means, and the follow-up app-data loader is built on it.

```

import Foundation

/// The cache validators a response came back with. Sending them back turns a revalidation
/// into a 304 with an empty body – the round trip still happens, but nothing is transferred
/// or decoded.
///
/// `Codable` both ways, and the only type in this layer that is: these are read off the response
/// headers rather than a JSON body, so the conformance exists for `AppDataCaching`, which writes
/// them beside the payload it validates and reads them back on the next launch.
public struct HTTPValidators: Sendable, Equatable, Codable {
    public let etag: String?
    public let lastModified: String?

    public init(etag: String?, lastModified: String?) {
        self.etag = etag
        self.lastModified = lastModified
    }

    public init?(_ response: HTTPURLResponse) {
        let etag = response.value(forHTTPHeaderField: "ETag")
        let lastModified = response.value(forHTTPHeaderField: "Last-Modified")
        guard etag != nil || lastModified != nil else { return nil }
        self.etag = etag
        self.lastModified = lastModified
    }

    var conditionalHeaders: [String: String] {
        var headers: [String: String] = [:]
        if let etag { headers["If-None-Match"] = etag }
        if let lastModified { headers["If-Modified-Since"] = lastModified }
        return headers
    }
}

public enum ConditionalResponse: Sendable, Equatable {
    /// 304 – what we already hold is current.
    case notModified
    case fresh(Data, HTTPValidators?)
}

```

66 JackpotKit/Sources/JackpotNetworking/RemoteApiClient.swift

```

import Foundation

public protocol ApiClient: Sendable {
    func request<Response: Decodable & Sendable>(_ endpoint: some APIEndpoint) async throws -> Response
    /// For 204 / empty-body responses.
    func request(_ endpoint: some APIEndpoint) async throws
    /// The raw bytes, for responses decoded section-by-section rather than into one type.
    func requestData(_ endpoint: some APIEndpoint) async throws -> Data
    /// Revalidates against what the caller already holds. Returns `.notModified` on a 304.
    func requestConditional(_ endpoint: some APIEndpoint,
                           validators: HTTPValidators?) async throws -> ConditionalResponse
}

public struct RemoteApiClient: ApiClient {
    private let environment: APIEnvironment
    private let httpClient: any HTTPClient
    private let interceptors: [any RequestInterceptor]
    private let decoder: JSONDecoder
    private let maxTransientRetries: Int

    public init(environment: APIEnvironment,
                httpClient: any HTTPClient = URLSessionHTTPClient(),
                interceptors: [any RequestInterceptor] = [],
                decoder: JSONDecoder = JSONDecoder(),

```

```

        maxTransientRetries: Int = 1) {
    self.environment = environment
    self.httpClient = httpClient
    self.interceptors = interceptors
    self.decoder = decoder
    self.maxTransientRetries = maxTransientRetries
}

public func request<Response: Decodable & Sendable>(_ endpoint: some APIEndpoint) async throws -> Response {
    let (data, _) = try await perform(endpoint)
    do {
        return try decoder.decode(Response.self, from: data)
    } catch {
        // The full error description names the key path that failed;
        // `localizedDescription` would only say "The data couldn't be read".
        throw APIError.decoding("\(Response.self): \(error)")
    }
}

public func request(_ endpoint: some APIEndpoint) async throws {
    _ = try await perform(endpoint)
}

public func requestData(_ endpoint: some APIEndpoint) async throws -> Data {
    try await perform(endpoint).0
}

public func requestConditional(_ endpoint: some APIEndpoint,
                              validators: HTTPValidators?) async throws -> ConditionalResponse {
    let (data, response) = try await perform(endpoint, extraHeaders: validators?.conditionalHeaders ?? [])
    if response.statusCode == 304 { return .notModified }
    return .fresh(data, HTTPValidators(response))
}

// MARK: - Pipeline

private func perform(_ endpoint: some APIEndpoint,
                    extraHeaders: [String: String] = [:]) async throws -> (Data, HTTPURLResponse) {
    var request = try endpoint.urlRequest(in: environment)
    for (key, value) in extraHeaders {
        request.setValue(value, forHTTPHeaderField: key)
    }
    for interceptor in interceptors {
        request = try await interceptor.adapt(request, for: endpoint)
    }
    return try await send(request, for: endpoint, didRetryAuth: false, transientRetries: 0)
}

private func send(_ request: URLRequest,
                 for endpoint: some APIEndpoint,
                 didRetryAuth: Bool,
                 transientRetries: Int) async throws -> (Data, HTTPURLResponse) {
    let (data, response) = try await transport(request)

    switch response.statusCode {
    case 200..<300:
        return (data, response)

    case 304:
        // Only reachable when the caller sent conditional headers; `requestConditional`
        // turns this into `.notModified`.
        return (data, response)

    case 401:
        // `didRetryAuth` is a parameter rather than stored state, so a second 401
        // can never loop.
        if !didRetryAuth {
            for interceptor in interceptors {
                if let retry = await interceptor.retry(request, for: endpoint, response: response, data: data) {
                    return try await send(retry, for: endpoint, didRetryAuth: true, transientRetries: transientRetries)
                }
            }
        }
        throw APIError.unauthorized(problem(from: data))

    case 400:
        throw APIError.badRequest(problem(from: data))

    case 500, 502...504:
        // A POST may already have taken effect server-side, so only idempotent requests

```

```

        // are retried.
        if endpoint.isIdempotent, transientRetries < maxTransientRetries {
            try await Task.sleep(nanoseconds: 1_000_000_000)
            return try await send(request, for: endpoint, didRetryAuth: didRetryAuth,
                                transientRetries: transientRetries + 1)
        }
        throw response.statusCode == 500
            ? APIError.server(problem(from: data))
            : APIError.unexpectedStatus(response.statusCode, problem(from: data))

    default:
        throw APIError.unexpectedStatus(response.statusCode, problem(from: data))
    }
}

private func transport(_ request: URLRequest) async throws -> (Data, HTTPURLResponse) {
    do {
        return try await httpClient.send(request)
    } catch is CancellationError {
        throw APIError.cancelled
    } catch let error as URLError where error.code == .cancelled {
        // What URLSession actually throws when a Task is cancelled mid-flight.
        throw APIError.cancelled
    } catch let error as URLError {
        throw APIError.transport(error.code)
    }
}

private func problem(from data: Data) -> APIProblem? {
    try? JSONDecoder().decode(APIProblem.self, from: data)
}
}

```

67 JackpotKit/Tests/JackpotNetworkingTests/MockHTTPClient.swift

```

import Foundation
import XCTest
@testable import JackpotNetworking

/// Stubs the *transport*, not the client, so these tests exercise real URL building, real status
/// handling and real decoding with only the socket replaced. An actor because the client calls
/// it from concurrent tasks.
actor MockHTTPClient: HTTPClient {

    struct Stub {
        var data: Data
        var statusCode: Int
        var headers: [String: String]
        var error: (any Error)?

        static func ok(_ json: String, status: Int = 200) -> Stub {
            Stub(data: Data(json.utf8), statusCode: status, headers: [:], error: nil)
        }
        static func status(_ code: Int, json: String = "{}", headers: [String: String] = [:]) -> Stub {
            Stub(data: Data(json.utf8), statusCode: code, headers: headers, error: nil)
        }
        static func failure(_ error: any Error) -> Stub {
            Stub(data: Data(), statusCode: 0, headers: [:], error: error)
        }
    }

    private(set) var requests: [URLRequest] = []
    private var queue: [Stub]
    private let fallback: Stub

    init(_ stubs: [Stub], fallback: Stub = .status(500)) {
        self.queue = stubs
        self.fallback = fallback
    }

    init(_ stub: Stub) { self.init([stub], fallback: stub) }

    var requestCount: Int { requests.count }

    /// Asserted inside the actor, so callers don't have to reach across isolation.
    func assertRequests(_ expected: Int, file: StaticString = #filePath, line: UInt = #line) {
        XCTAssertEqual(requests.count, expected, file: file, line: line)
    }
}

```



```

    }

    func send(_ request: URLRequest) async throws -> (Data, HTTPURLResponse) {
        requests.append(request)
        let stub = queue.isEmpty ? fallback : queue.removeFirst()
        if let error = stub.error { throw error }
        let response = HTTPURLResponse(
            url: request.url ?? URL(string: "https://example.com")!,
            statusCode: stub.statusCode,
            httpVersion: nil,
            headerFields: stub.headers
        )!
        return (stub.data, response)
    }
}

// MARK: - Fixtures

struct Widget: Decodable, Equatable, Sendable {
    let id: Int
    let name: String
}

struct WidgetRequest: APIEndpoint {
    var path: String { "widgets/42" }
}

struct CreateWidgetRequest: APIEndpoint {
    let name: String
    var path: String { "widgets" }
    var method: HTTPMethod { .POST }
    var body: RequestBody? { .form(["name": name]) }
}

struct SearchRequest: APIEndpoint {
    let term: String
    var path: String { "search" }
    var queryItems: [URLQueryItem] { [URLQueryItem(name: "q", value: term)] }
}

extension APIEnvironment {
    static let test = APIEnvironment(
        baseURL: URL(string: "https://api.example.com/v1")!,
        defaultHeaders: ["Accept": "application/json"],
        defaultQueryItems: [URLQueryItem(name: "api-version", value: "2.0")]
    )
}

```

68 JackpotKit/Tests/JackpotNetworkingTests/APIEndpointTests.swift

```

import XCTest
@testable import JackpotNetworking

final class APIEndpointTests: XCTestCase {

    func testBuildsURLFromEnvironmentAndPath() throws {
        let request = try WidgetRequest().urlRequest(in: .test)
        XCTAssertEqual(request.url?.absoluteString,
            "https://api.example.com/v1/widgets/42?api-version=2.0")
        XCTAssertEqual(request.httpMethod, "GET")
    }

    /// Environment query items come first, then the endpoint's own – so an endpoint can
    /// never accidentally drop the API version.
    func testMergesEnvironmentAndEndpointQueryItems() throws {
        let request = try SearchRequest(term: "swift").urlRequest(in: .test)
        let components = URLComponents(url: request.url!, resolvingAgainstBaseURL: false)!
        XCTAssertEqual(components.queryItems?.map(\.name), ["api-version", "q"])
        XCTAssertEqual(components.queryItems?.last?.value, "swift")
    }

    func testLeadingSlashInPathDoesNotBreakTheURL() throws {
        struct Slashed: APIEndpoint { var path: String { "/widgets" } }
        let request = try Slashed().urlRequest(in: .test)
        XCTAssertEqual(request.url?.absoluteString,
            "https://api.example.com/v1/widgets?api-version=2.0")
    }
}

```

```

func testFormBodyIsPercentEncodedWithTheRightContentType() throws {
    let request = try CreateWidgetRequest(name: "a b&c").urlRequest(in: .test)
    XCTAssertEqual(request.httpMethod, "POST")
    XCTAssertEqual(request.value(forHTTPHeaderField: "Content-Type"),
        "application/x-www-form-urlencoded")
    let body = String(data: request.httpBody ?? Data(), encoding: .utf8)
    XCTAssertEqual(body, "name=a%20b%26c")
}

func testJSONBodySetsContentType() throws {
    struct JSONPost: APIEndpoint {
        var path: String { "x" }
        var method: HTTPMethod { .POST }
        var body: RequestBody? { .json(Data(#"{\"a\":1}\"#.utf8)) }
    }
    let request = try JSONPost().urlRequest(in: .test)
    XCTAssertEqual(request.value(forHTTPHeaderField: "Content-Type"), "application/json")
}

func testDefaultsAreSaneWithoutOverrides() {
    let endpoint = WidgetRequest()
    XCTAssertEqual(endpoint.method, .GET)
    XCTAssertTrue(endpoint.queryItems.isEmpty)
    XCTAssertNil(endpoint.body)
    XCTAssertTrue(endpoint.requiresAuth)
    XCTAssertTrue(endpoint.isIdempotent) // derived from .GET
}

func testPostIsNotIdempotentByDefault() {
    XCTAssertFalse(CreateWidgetRequest(name: "x").isIdempotent)
}
}

```

69 JackpotKit/Tests/JackpotNetworkingTests/APIProblemTests.swift

```

import XCTest
@testable import JackpotNetworking

/// The API's envelope is `{ "code": 0, "message": "Error message" }`.
///
/// These matter more than they look: the client decodes problems with `try?` so a gateway's
/// HTML body can't throw. That means a *shape* mismatch is completely silent – no crash, no
/// log, just permanently empty error messages in the UI. These tests are the only thing
/// standing between a backend field rename and that failure.
final class APIProblemTests: XCTestCase {

    private func decode(_ json: String) -> APIProblem? {
        try? JSONDecoder().decode(APIProblem.self, from: Data(json.utf8))
    }

    func testTheDocumentedShape() {
        XCTAssertEqual(decode(#"{\"code\":0,\"message\":\"Error message\"}#"),
            APIProblem(code: 0, message: "Error message"))
    }

    func testNonZeroCode() {
        XCTAssertEqual(decode(#"{\"code\":1042,\"message\":\"Mobile already registered\"}#").code, 1042)
    }

    /// Tolerated so a backend switching to string codes can't silently break every message.
    func testStringCodeIsCoerced() {
        XCTAssertEqual(decode(#"{\"code\":\"42\",\"message\":\"x\"}#").code, 42)
    }

    func testMessageOnly() {
        XCTAssertEqual(decode(#"{\"message\":\"Just a message\"}#"),
            APIProblem(code: nil, message: "Just a message"))
    }

    func testCodeOnly() {
        XCTAssertEqual(decode(#"{\"code\":7}#"), APIProblem(code: 7, message: nil))
    }

    // MARK: Things that are *not* a problem envelope

    func testEmptyObjectIsNotAProblem() {

```

```

    XCTAssertNil(decode("{}"), "an empty body is 'the server said nothing', not an empty complaint")
}

func testEmptyMessageIsTreatedAsAbsent() {
    XCTAssertNil(decode(#"{\"message\":\"\"}#"))
}

func testUnrelatedJSONIsNotAProblem() {
    XCTAssertNil(decode(#"{\"data\":{\"id\":1}}#"))
}

func testHTMLFromAGatewayIsNotAProblem() {
    XCTAssertNil(decode("<html><body>503</body></html>"))
}

func testExtraFieldsAreIgnored() {
    XCTAssertEqual(decode(#"{\"code\":3,\"message\":\"m\",\"traceId\":\"abc\",\"status\":400}#"),
        APIProblem(code: 3, message: "m"))
}
}

```

70 JackpotKit/Tests/JackpotNetworkingTests/RemoteApiClientTests.swift

The retry policy is the part worth asserting: a 5xx or a dropped connection is retried, a 4xx never is, and a non-idempotent request is retried only when the transport failed before the server could have seen it.

```

import XCTest
@testable import JackpotNetworking

final class RemoteApiClientTests: XCTestCase {

    private func client(_ http: MockHTTPClient,
        interceptors: [any RequestInterceptor] = []) -> RemoteApiClient {
        RemoteApiClient(environment: .test, httpClient: http, interceptors: interceptors)
    }

    // MARK: Success

    func testDecodesA200() async throws {
        let http = MockHTTPClient(.ok(#"{\"id\":42,\"name\":\"Sprocket\"}#"))
        let widget: Widget = try await client(http).request(WidgetRequest())
        XCTAssertEqual(widget, Widget(id: 42, name: "Sprocket"))
        await http.assertRequests(1)
    }

    func testEmptyBodyRequestIgnoresThePayload() async throws {
        let http = MockHTTPClient(.status(204))
        try await client(http).request(WidgetRequest())
        await http.assertRequests(1)
    }

    // MARK: The documented contract – 200, 400, 401, 500

    func test400CarriesTheServersMessage() async {
        let http = MockHTTPClient(.status(400, json: #"{\"code\":\"1042\",\"message\":\"Mobile number already registered\"}#"))
        await AssertThrows(try await client(http).request(WidgetRequest()) as Widget) { error in
            guard case .badRequest(let problem) = error as? APIError else {
                return XCTFail("expected .badRequest, got \(error)")
            }
            XCTAssertEqual(problem, APIProblem(code: 1042, message: "Mobile number already registered"))
            XCTAssertEqual((error as? APIError)?.serverMessage, "Mobile number already registered")
        }
    }

    /// `{"code": 0, ...}` is the shape the API actually sends. An earlier `code: String?`
    /// meant this decoded to nil and every error message came back empty – silently,
    /// because problem decoding is `try?` by design.
    func testNumericZeroCodeDecodes() async {
        let http = MockHTTPClient(.status(400, json: #"{\"code\":\"0\",\"message\":\"Error message\"}#"))
        await AssertThrows(try await client(http).request(WidgetRequest()) as Widget) { error in
            XCTAssertEqual((error as? APIError)?.problem, APIProblem(code: 0, message: "Error message"))
        }
    }

    func testStringCodeAlsoDecodes() async {
        let http = MockHTTPClient(.status(400, json: #"{\"code\":\"7\",\"message\":\"x\"}#"))
    }
}

```

```

        await AssertThrows(try await client(http).request(WidgetRequest()) as Widget) { error in
            XCTAssertEqual((error as? APIError)?.problem?.code, 7)
        }
    }

    /// A gateway returning HTML must not throw a decoding error on the way to reporting
    /// the status.
    func testNonJSONErrorBodyStillReportsTheStatus() async {
        let http = MockHTTPClient(.status(400, json: "<html>502 Bad Gateway</html>"))
        await AssertThrows(try await client(http).request(WidgetRequest()) as Widget) { error in
            XCTAssertEqual(error as? APIError, .badRequest(nil))
            XCTAssertNil((error as? APIError)?.serverMessage)
        }
    }

    func test500CarriesTheProblemAfterRetrying() async {
        let http = MockHTTPClient([.status(500), .status(500, json: #"{"code":9,"message":"Down"}"#)])
        await AssertThrows(try await client(http).request(WidgetRequest()) as Widget) { error in
            XCTAssertEqual(error as? APIError, .server(APIProblem(code: 9, message: "Down")))
        }
        await http.assertRequests(2)
    }

    func test500DoesNotRetryANonIdempotentRequest() async {
        let http = MockHTTPClient([.status(500), .ok(#"{"id":1,"name":"x"}"#)])
        await AssertThrows(try await client(http).request(CreateWidgetRequest(name: "x")) as Widget) { error in
            XCTAssertEqual(error as? APIError, .server(nil))
        }
        await http.assertRequests(1)
    }

    // MARK: Outside the contract – infrastructure, not the API

    func test404IsReportedAsUnexpectedRatherThanFlattenedIntoServer() async {
        let http = MockHTTPClient(.status(404))
        await AssertThrows(try await client(http).request(WidgetRequest()) as Widget) { error in
            XCTAssertEqual(error as? APIError, .unexpectedStatus(404, nil))
        }
        await http.assertRequests(1) // a 404 is not retried
    }

    func testGatewayErrorsRetryOnceThenReportTheirRealStatus() async {
        let http = MockHTTPClient([.status(503), .status(503)])
        await AssertThrows(try await client(http).request(WidgetRequest()) as Widget) { error in
            XCTAssertEqual(error as? APIError, .unexpectedStatus(503, nil))
        }
        await http.assertRequests(2)
    }

    // MARK: Transport and decoding

    func testMalformedJSONSurfacesAsDecodingWithTheTypeName() async {
        let http = MockHTTPClient(.ok(#"{"id":"not-an-int"}"#))
        await AssertThrows(try await client(http).request(WidgetRequest()) as Widget) { error in
            guard case .decoding(let message) = error as? APIError else {
                return XCTFail("expected .decoding, got \(error)")
            }
            // DecodingError.localizedDescription is "The data couldn't be read" – useless.
            XCTAssertTrue(message.contains("Widget"), message)
        }
    }

    func testOfflineMapsToTransportAndIsFlaggedOffline() async {
        let http = MockHTTPClient(.failure(URLError(.notConnectedToInternet)))
        await AssertThrows(try await client(http).request(WidgetRequest()) as Widget) { error in
            XCTAssertEqual(error as? APIError, .transport(.notConnectedToInternet))
            XCTAssertTrue((error as? APIError)?.isOffline == true)
        }
    }

    /// URLSession throws `URLError.cancelled`, not `CancellationError`, when a task is
    /// cancelled mid-flight. Miss this and cancelled loads surface to users as failures.
    func testURLErrorCancelledMapsToCancelled() async {
        let http = MockHTTPClient(.failure(URLError(.cancelled)))
        await AssertThrows(try await client(http).request(WidgetRequest()) as Widget) { error in
            XCTAssertEqual(error as? APIError, .cancelled)
        }
    }

    // MARK: Auth retry

```

```

func test401RetriesOnceWithTheRefreshedToken() async throws {
    let http = MockHTTPClient([.status(401), .ok(#"{\"id\":1,\"name\":\"ok\"}#)])
    let interceptor = StubAuthInterceptor(refreshedToken: "new-token")
    let widget: Widget = try await client(http, interceptors: [interceptor]).request(WidgetRequest())

    XCTAssertEqual(widget.name, "ok")
    await http.assertRequests(2)
    let retried = await http.requests[1]
    XCTAssertEqual(retried.value(forHTTPHeaderField: "Authorization"), "Bearer new-token")
}

/// The retry budget is a call-shape parameter, not stored state – so a second 401
/// can never loop.
func testSecond401GivesUpAsUnauthorized() async {
    let http = MockHTTPClient([.status(401), .status(401)])
    let interceptor = StubAuthInterceptor(refreshedToken: "new-token")
    await AssertThrows(try await client(http, interceptors: [interceptor]).request(WidgetRequest()) as Widget) { error in
        XCTAssertEqual(error as? APIError, .unauthorized(nil))
    }
    await http.assertRequests(2)
}

func test401WithNoInterceptorFailsImmediately() async {
    let http = MockHTTPClient(.status(401))
    await AssertThrows(try await client(http).request(WidgetRequest()) as Widget) { error in
        XCTAssertEqual(error as? APIError, .unauthorized(nil))
    }
    await http.assertRequests(1)
}

func testBearerTokenIsAttachedButNotToAuthExemptEndpoints() async throws {
    struct LoginRequest: APIEndpoint {
        var path: String { "auth/login" }
        var method: HTTPMethod { .POST }
        var requiresAuth: Bool { false }
    }
    let http = MockHTTPClient([.ok(#"{\"id\":1,\"name\":\"a\"}#"), .ok(#"{\"id\":1,\"name\":\"a\"}#)])
    let sut = client(http, interceptors: [BearerTokenInterceptor { "abc" }])

    _ = try await sut.request(WidgetRequest()) as Widget
    _ = try await sut.request(LoginRequest()) as Widget

    let widgetAuth = await http.requests[0].value(forHTTPHeaderField: "Authorization")
    let loginAuth = await http.requests[1].value(forHTTPHeaderField: "Authorization")
    XCTAssertEqual(widgetAuth, "Bearer abc")
    XCTAssertNil(loginAuth, "a login request must never carry a bearer token")
}

}

// MARK: - Helpers

private struct StubAuthInterceptor: RequestInterceptor {
    let refreshedToken: String

    func retry(_ request: URLRequest, for endpoint: any APIEndpoint,
              response: HTTPURLResponse, data: Data) async -> URLRequest? {
        guard response.statusCode == 401 else { return nil }
        var retry = request
        retry.setValue("Bearer \(refreshedToken)", forHTTPHeaderField: "Authorization")
        return retry
    }
}

private func AssertThrows<T>(_ expression: @autoclosure () async throws -> T,
                             file: StaticString = #filePath, line: UInt = #line,
                             _ verify: (any Error) -> Void) async {
    do {
        _ = try await expression()
        XCTFail("expected a thrown error", file: file, line: line)
    } catch {
        verify(error)
    }
}

```

71

```
cd JackpotKit
xcodebuild -scheme JackpotKit-Package -destination 'platform=iOS Simulator,name=iPhone 17 Pro' test
```

JackpotNetworkingTests: Executed 34 tests . Now the repository.

72 JackpotKit/Sources/JackpotLocalization/Translations.swift

The session's localisation table. Two lookups matter: keys are tried region-suffixed first (`terms-jza` before `terms`), and API error codes are keys too, which is what lets a server error come back in the user's language.

```
import Foundation

/// The app's localisation table, fetched once per session from the app-data endpoint.
///
/// Normalised to lowercase once, at construction: one registration form asks for ~36 strings
/// per render pass, and normalising per lookup would mean 36 full dictionary rebuilds to draw
/// one screen.
public struct Translations: Sendable, Equatable {

    private let table: [String: String]
    /// Lowercased region code, e.g. "jza". Nil when the app has no region yet.
    public let regionSuffix: String?

    public init(_ locales: [String: String] = [:], regionCode: String? = nil) {
        self.table = Dictionary(
            locales.lazy.map { ($0.key.lowercased(), $0.value) },
            uniquingKeysWith: { first, _ in first }
        )
        let trimmed = regionCode?.trimmingCharacters(in: .whitespaces).lowercased()
        self.regionSuffix = (trimmed?.isEmpty ?? true) ? nil : trimmed
    }

    public var isEmpty: Bool { table.isEmpty }

    /// Looks up `key`, cascading `key`-<region>` → `key` → nil.
    ///
    /// Region-first by default: a `-jza` variant only exists because someone wanted it used,
    /// and making it opt-in per call site means the callers that forget show the wrong copy.
    /// Pass `regional: false` to force the plain key.
    public func string(forKey key: String, regional: Bool = true) -> String? {
        let normalized = key.lowercased()
        if regional, let regionSuffix, let regional = table["\(normalized)-\(regionSuffix)"] {
            return regional
        }
        return table[normalized]
    }

    /// Resolved string, falling back to the key itself so a missing translation is visible in
    /// QA rather than rendering as an empty label.
    public func callAsFunction(_ key: String, regional: Bool = true) -> String {
        string(forKey: key, regional: regional) ?? key
    }

    /// The table doubles as an error-code catalogue: the app-data response carries entries
    /// like "6000328": "Maximum OTP tries reached...", so a `code` in an API error envelope
    /// is a localisation key.
    public func message(forErrorCode code: Int) -> String? {
        string(forKey: "jpc-reg-error.\(code)", regional: false)
        ?? string(forKey: String(code), regional: false)
    }
}
```

73 JackpotKit/Sources/JackpotFormsRemote/CRMEnvironment.swift

```
import Foundation
import JackpotNetworking

// Fetch, submit and app-data all live under `https://config.jpc.africa/cron`. Fetch carries
// `api-version=2.0` and submit carries none, so the version belongs on the request rather than
// on this environment.
```

```

public extension APIEnvironment {

    /// The cron config service - `https://config.jpc.africa/cron`.
    static func cron(baseURL: URL) -> APIEnvironment {
        APIEnvironment(baseURL: baseURL)
    }

    /// `https://config.jpc.africa/crm` → `https://config.jpc.africa/cron`, because call sites
    /// historically passed a CRM base and production fetch is on cron.
    static func cronBaseURL(fromCRM url: URL) -> URL {
        switch url.lastPathComponent {
        case "crm":
            return url.deletingLastPathComponent().appendingPathComponent("cron")
        case "cron":
            return url
        default:
            return url.appendingPathComponent("cron")
        }
    }
}

```

74 JackpotKit/Sources/JackpotFormsRemote/FormEndpoints.swift

```

import Foundation
import JackpotNetworking
import JackpotFormsDomain

/// `GET {cron}/forms/{brand}/{region}/{identifier}?api-version=2.0` - one template for both
/// fetch-by-name and fetch-by-id, matching production `buildFormURL`.
public struct FormRequest: APIEndpoint {
    let brand: String
    let region: String
    let identifier: String

    public init(brand: String, region: String, formName: FormName) {
        self.init(brand: brand, region: region, identifier: formName.rawValue)
    }

    init(brand: String, region: String, identifier: String) {
        self.brand = brand
        self.region = region
        self.identifier = identifier
    }

    public var path: String { "forms/\(brand)/\(region)/\(identifier)" }
    public var method: HTTPMethod { .GET }
    public var queryItems: [URLQueryItem] { [URLQueryItem(name: "api-version", value: "2.0")] }
}

/// `POST {cron}/forms/submit` - no `api-version` query item. Auth is not required, because
/// registration happens before login.
struct FormSubmitRequest: APIEndpoint {
    let bodyData: Data

    init(_ submission: FormSubmission) throws {
        self.bodyData = try FormSubmitBody.encoder.encode(FormSubmitBody(submission))
    }

    var path: String { "forms/submit" }
    var method: HTTPMethod { .POST }
    var body: RequestBody? { .json(bodyData) }
    var requiresAuth: Bool { false }
}

/// Wire shape of `FormSubmission`, so Domain stays free of JSON key names.
struct FormSubmitBody: Encodable {
    let formId: String
    let formName: String
    let submittedAt: Date
    let fields: [String: FieldValue]
    let metadata: [String: String]?

    enum CodingKeys: String, CodingKey {
        case formId = "form_id"
        case formName = "form_name"
        case submittedAt = "submitted_at"
    }
}

```

```

    case fields, metadata
  }

  init(_ submission: FormSubmission) {
    formId = submission.formId
    formName = submission.formCodeName.rawValue
    submittedAt = submission.submittedAt
    fields = submission.values.mapValues(\.fieldValue)
    metadata = submission.metadata
  }

  /// ISO-8601 is an assumption — the production encoder wasn't visible.
  static let encoder: JSONEncoder = {
    let encoder = JSONEncoder()
    encoder.dateEncodingStrategy = .iso8601
    return encoder
  }()

  /// Untagged JSON value for a form field.
  ///
  /// The custom `encode` is required: synthesized `Codable` would emit a tagged object
  /// (`{"bool":true}`) instead of a bare JSON value.
  enum FieldValue: Equatable, Sendable, Encodable {
    case string(String)
    case bool(Bool)
    case null

    func encode(to encoder: Encoder) throws {
      var container = encoder.singleValueContainer()
      switch self {
      case .string(let value): try container.encode(value)
      case .bool(let value): try container.encode(value)
      case .null: try container.encodeNil()
      }
    }
  }

  extension FormValue {
    var fieldValue: FieldValue {
      switch self {
      case .empty: return .null
      case .text(let s): return .string(s)
      case .bool(let b): return .bool(b)
      case .option(let s): return .string(s)
      case .date(let d): return .string(FormValue.iso8601.string(from: d))
      }
    }
  }

  /// HTTP 200 is not success: the body carries `isSuccessful` / `error`, so a client that only
  /// looked for `success` would swallow a failed create.
  struct FormSubmitEnvelope: Decodable {
    let data: FormSubmitDataDTO?
    let isSuccessful: Bool?
    let success: Bool?
    let error: FormSubmitErrorDTO?
  }

  struct FormSubmitDataDTO: Decodable {
    let accountId: String?
    let message: String?
    let status: String?
    let partialRegistrationStatus: Int?
    let complianceResponse: FormComplianceDTO?
  }

  struct FormSubmitErrorDTO: Decodable {
    let code: Int?
    let displayCode: Int?
    let message: String?
  }

  struct FormComplianceDTO: Decodable {
    let complianceStatus: Int?
    let requiredComplianceStatus: Int?
    let isValidId: Bool?
    let message: String?
    let accessToken: String?
  }

```



```

enum FormSubmitParser {
    enum Outcome {
        case accepted(FormSubmitResult)
        case rejected(FormSubmitErrorDTO?)
    }

    static func parse(_ data: Data) -> Outcome {
        if data.isEmpty { return .accepted(FormSubmitResult()) }
        guard let envelope = try? JSONDecoder().decode(FormSubmitEnvelope.self, from: data) else {
            return .accepted(FormSubmitResult())
        }
        if envelope.failed { return .rejected(envelope.error) }
        return .accepted(FormSubmitResult(envelope.data))
    }
}

private extension FormSubmitEnvelope {
    var failed: Bool {
        if isSuccessful == false || success == false { return true }
        return error != nil && isSuccessful != true
    }
}

extension FormSubmitResult {
    init(_ data: FormSubmitDataDTO?) {
        self.init(
            accountId: data?.accountId,
            message: data?.message,
            status: data?.status,
            partialRegistrationStatus: data?.partialRegistrationStatus,
            compliance: data?.complianceResponse.map(FormComplianceResult.init)
        )
    }
}

extension FormComplianceResult {
    init(_ dto: FormComplianceDTO) {
        self.init(
            complianceStatus: dto.complianceStatus,
            requiredComplianceStatus: dto.requiredComplianceStatus,
            isValidId: dto.isValidId,
            message: dto.message,
            accessToken: dto.accessToken
        )
    }
}

```

75 JackpotKit/Sources/JackpotFormsRemote/FormErrorMapper.swift

The boundary that decides what the user reads. `JackpotFormsUI` cannot see `APIError`, so anything not translated here becomes a generic failure on screen.

```

import Foundation
import JackpotNetworking
import JackpotFormsDomain

// The translation boundary. `JackpotFormsUI` never sees `APIError`, so this is where transport
// concerns become something a person can read.
enum FormErrorMapper {

    /// - Parameter localizer: resolves an error `code` to localised copy when the session's
    ///   table carries one. That beats `problem.message`, which arrives in whatever language
    ///   the API defaulted to. Typed as the Domain protocol so this target needs no
    ///   localisation dependency.
    static func map(_ error: any Error,
                    formName: FormName,
                    localizer: (any FormLocalizing)? = nil) -> any Error {
        guard let apiError = error as? APIError else { return error }

        switch apiError {
        case .cancelled:
            // Never user-facing: a cancelled load is a navigation event, not a failure.
            return CancellationError()

        case .transport where apiError.isOffline:

```

```

        return FormLoadError.offline

    case .unexpectedStatus(404, _):
        return FormLoadError.notFound(formName)

    case .badRequest, .unauthorized, .server, .unexpectedStatus:
        // Localised copy for the code first, then whatever the server wrote, then ours –
        // the server is the only party that knows *why*.
        if let code = apiError.problem?.code,
            let localized = localizer?.message(forErrorCode: code) {
            return FormLoadError.server(message: localized)
        }
        if let message = apiError.serverMessage {
            return FormLoadError.server(message: message)
        }
        return FormLoadError.unexpected

    case .transport, .decoding, .invalidURL:
        return FormLoadError.unexpected
    }
}

```

76

JackpotKit/Sources/JackpotFormsRemote/RemoteFormRepository.swift

```

import Foundation
import JackpotNetworking
import JackpotFormsData
import JackpotFormsDomain

public struct RemoteFormRepository: FormRepository {
    private let apiClient: any ApiClient
    private let brand: String
    private let region: String
    /// Resolves error codes to localised copy. Optional: without it, errors fall back to the
    /// server's own message.
    private let localizer: (any FormLocalizing)?

    public init(apiClient: any ApiClient,
                brand: String = "jackpotcity",
                region: String = "JZA",
                localizer: (any FormLocalizing)? = nil) {
        self.apiClient = apiClient
        self.brand = brand
        self.region = region
        self.localizer = localizer
    }

    public func form(named name: FormName) async throws -> FormSchema {
        try await fetch(identifier: name.rawValue, formName: name)
    }

    public func form(id: String) async throws -> FormSchema {
        try await fetch(identifier: id, formName: FormName(id))
    }

    public func submitForm(_ submission: FormSubmission) async throws -> FormSubmitResult {
        do {
            let data = try await apiClient.requestData(FormSubmitRequest(submission))
            switch FormSubmitParser.parse(data) {
            case .accepted(let result):
                return result
            case .rejected(let error):
                throw FormLoadError.server(message: localized(error))
            }
        } catch {
            throw FormErrorMessageMapper.map(error, formName: submission.formCodeName, localizer: localizer)
        }
    }

    private func fetch(identifier: String, formName: FormName) async throws -> FormSchema {
        do {
            let dto: FormDTO = try await apiClient.request(
                FormRequest(brand: brand, region: region, identifier: identifier)
            )
            return FormMapper.map(dto)
        } catch {

```

```

        throw FormErrorMessage.map(error, formName: formName, localizer: localizer)
    }
}

private func localized(_ error: FormSubmitErrorDTO?) -> String {
    if let code = error?.code ?? error?.displayCode,
        let localized = localizer?.message(forErrorCode: code) {
        return localized
    }
    return error?.message ?? "We couldn't submit the form. Please try again."
}
}

```

77 JackpotKit/Sources/JackpotForms/TranslationsLocalizer.swift

```

import Foundation
import JackpotLocalization
import JackpotFormsDomain

/// Adapts the app's session-wide `Translations` table to the form engine's `FormLocalizing`
/// seam.
///
/// An adapter rather than `extension Translations: FormLocalizing`: both types belong to other
/// modules, so the conformance would be retroactive – which Swift 6 warns about, and which
/// breaks if either module later declares its own.
public struct TranslationsLocalizer: FormLocalizing {

    private let translations: Translations

    public init(_ translations: Translations) {
        self.translations = translations
    }

    /// `regional: true` covers both key shapes the schema uses: a plain key like `username`
    /// picks up its `-jza` override when one exists, and an already-suffixed key like
    /// `receivePromotionalInformation-jza` resolves directly.
    public func string(forKey key: String) -> String? {
        translations.string(forKey: key, regional: true)
    }

    public func message(forErrorCode code: Int) -> String? {
        translations.message(forErrorCode: code)
    }
}

```

78 JackpotKit/Sources/JackpotForms/JackpotForms.swift

The composition root: `.mock` for previews and the sandbox, `.live` for the app.

```

import Foundation
import JackpotNetworking
import JackpotFormsDomain
import JackpotFormsUI
import JackpotLocalization
import JackpotFormsData
import JackpotFormsRemote

// The composition target: the one place that knows both how forms are fetched (Data) and how
// they are rendered (UI). Every other dependency stays one-directional.

public extension FormDependencies {

    /// Serves the bundled `registration` schema, captured from
    /// `config.jpc.africa/cron/forms/jackpotcity/JZA/registration?api-version=2.0`, so the
    /// feature is buildable and reviewable before the API is reachable from the app.
    ///
    /// DynamicFormView(formName: .registration) { ... }
    ///     .formDependencies(.mock())
    ///
    /// - Parameters:
    ///   - delay: fake latency, so loading states are visible in the sandbox.
    ///   - error: set to exercise the failure state.
    static func mock(delay: TimeInterval = 0.35,
                     error: (any Error)? = nil,
                     translations: Translations? = nil,

```

```

        localizer: (any FormLocalizing)? = nil) -> FormDependencies {
    FormDependencies(
        repository: StubFormRepository(forms: FormPreviewData.bundledForms, delay: delay, error: error),
        localizer: localizer
        ?? translations.map(TranslationsLocalizer.init)
        ?? ComposedKeyLocalizer.jpcRegistration
    )
}

/// The real thing. Swap `.mock()` for this at the call site – nothing else changes, because
/// `DynamicFormView` only ever sees the `FormRepository` protocol.
///
/// .formDependencies(.live(baseUrl: URL(string: "https://config.jpc.africa/crm")))
///
/// `baseUrl` may be the historical CRM URL; fetch and submit both run on the sibling cron
/// service. `region` should be `AppSetupData.wmsNavigationRegionCode`.
/// - Parameter translations: the session's locale table, from the once-per-session app-data
/// call. Supplying it resolves labels, placeholders and dropdown options through the real
/// CRM copy, and turns API error *codes* into localised sentences.
/// - Parameter localizer: overrides the table-derived localizer. During the migration the
/// app passes a `ClosureLocalizer` over its existing `getTranslation`; once
/// `JackpotLocalization` is adopted, omit it and pass `translations` instead.
static func live(baseUrl: URL,
    translations: Translations = Translations(),
    brand: String = "jackpotcity",
    region: String = "JZA",
    bearerToken: (@Sendable () async -> String?)? = nil,
    localizer explicitLocalizer: (any FormLocalizing)? = nil) -> FormDependencies {
    let interceptors: [any RequestInterceptor] = bearerToken.map { [BearerTokenInterceptor(token: $0)] } ?? []
    let client = RemoteApiClient(
        environment: .cron(baseUrl: APIEnvironment.cronBaseUrl(fromCRM: baseUrl)),
        interceptors: interceptors
    )
    // An empty table means app-data hasn't landed yet; the bundled placeholder copy keeps
    // the form legible rather than rendering raw keys.
    let localizer: any FormLocalizing = explicitLocalizer
        ?? (translations.isEmpty ? ComposedKeyLocalizer.jpcRegistration : TranslationsLocalizer(translations))

    return FormDependencies(
        repository: RemoteFormRepository(apiClient: client, brand: brand, region: region,
            localizer: localizer),
        localizer: localizer
    )
}

public extension FormSandboxView {
    /// The sandbox, wired to the bundled schemas.
    static func mocked() -> FormSandboxView {
        FormSandboxView(
            samples: [
                .init(id: .registration, title: "Registration"),
                .init(id: .kitchenSink, title: "All field types"),
            ],
            dependencies: .mock()
        )
    }
}

```

79 JackpotKit/Sources/JackpotForms/Previews.swift

```

#if DEBUG
import SwiftUI
import JackpotUI
import JackpotFormsDomain
import JackpotFormsData
import JackpotNetworking
import JackpotFormsUI

/// JSON-backed previews – these go through the real decoder against the captured
/// `registration.json`, so they catch a schema change that the hand-built fixtures in
/// `JackpotFormsUI` would not. This is the only target that can see both layers.
struct MockedForm_Previews: PreviewProvider {
    static var previews: some View {
        Group {
            DynamicFormView(formName: .registration) { _ in }
                .formDependencies(.mock(delay: 0))
        }
    }
}

```

```

        .previewDisplayName("Registration - from JSON")

        DynamicFormView(formName: .kitchenSink) { _ in }
            .formDependencies(.mock(delay: 0))
            .previewDisplayName("All field types - from JSON")

        DynamicFormView(formName: .registration) { _ in }
            .formDependencies(.mock(delay: 3600))
            .previewDisplayName("Loading")

        DynamicFormView(formName: .registration) { _ in }
            .formDependencies(.mock(delay: 0, error: APIError.transport(.notConnectedToInternet)))
            .previewDisplayName("Offline")

        DynamicFormView(formName: FormName("doesNotExist")) { _ in }
            .formDependencies(.mock(delay: 0))
            .previewDisplayName("Unknown form - 404")
    }
    .frame(height: 620)
    .background(JackpotTheme.jackpotCity.colors.surface)
    .preferredColorScheme(.dark)
    .previewLayout(.sizeThatFits)
}

struct MockedSandbox_Previews: PreviewProvider {
    static var previews: some View {
        FormSandboxView.mocked()
            .preferredColorScheme(.dark)
            .previewDisplayName("Sandbox - the testing page")
    }
}
#endif

```

80 JackpotKit/Tests/JackpotLocalizationTests/TranslationsTests.swift

```

import XCTest
@testable import JackpotLocalization

/// Keys and values below are taken from the real app-data response.
final class TranslationsTests: XCTestCase {

    private let sample = Translations([
        "6000328": "Maximum OTP tries reached, Please contact support on +233 30 825 5838",
        "markets.withdrawwin": "1X2",
        "new-site": "New Site",
        "select-password-reset-method": "Select Password Reset Method",
        "jpc-fixed-jackpots-starts-in": "Starts in",
        "city-jackpots": "City Jackpots",
        "receivePromotionalInformation": "Send me promotions",
        "receivePromotionalInformation-jza": "Send Jackpot City Promotions to me",
    ], regionCode: "JZA")

    // MARK: Lookup

    func testResolvesAKey() {
        XCTAssertEqual(sample("new-site"), "New Site")
        XCTAssertEqual(sample("markets.withdrawwin"), "1X2")
    }

    func testLookupIsCaseInsensitive() {
        XCTAssertEqual(sample("NEW-SITE"), "New Site")
        XCTAssertEqual(sample("New-Site"), "New Site")
    }

    /// A missing key renders as itself - visible in QA, rather than an empty label.
    func testMissingKeyFallsBackToTheKey() {
        XCTAssertEqual(sample("no-such-key"), "no-such-key")
        XCTAssertNil(sample.string(forKey: "no-such-key"))
    }

    // MARK: Region cascade

    func testRegionalVariantWinsWhenItExists() {
        XCTAssertEqual(sample("receivePromotionalInformation"),
            "Send Jackpot City Promotions to me")
    }
}

```

```

func testPlainKeyUsedWhenNoRegionalVariantExists() {
    XCTAssertEqual(sample("new-site"), "New Site")
}

func testRegionalLookupCanBeForcedOff() {
    XCTAssertEqual(sample("receivePromotionalInformation", regional: false),
        "Send me promotions")
}

/// The CRM sometimes hands us a key that already carries the region suffix – the
/// registration schema's `fieldLabel` is literally `receivePromotionalInformation-jza`.
/// Looking that up must not double-suffix into a miss.
func testAPreSuffixedKeyStillResolves() {
    XCTAssertEqual(sample("receivePromotionalInformation-jza"),
        "Send Jackpot City Promotions to me")
}

func testNoRegionMeansNoRegionalLookup() {
    let noRegion = Translations(["a-jza": "regional", "a": "plain"], regionCode: nil)
    XCTAssertEqual(noRegion("a"), "plain")
    XCTAssertNil(noRegion.regionSuffix)
}

func testBlankRegionIsTreatedAsAbsent() {
    XCTAssertNil(Translations(["a": "b"], regionCode: " ").regionSuffix)
}

// MARK: Error codes

/// The table doubles as an error-code catalogue, which is what ties an API error envelope
/// ({"code": 6000328, ...}) to a sentence in the player's language.
func testErrorCodeResolvesToItsMessage() {
    XCTAssertEqual(sample.message(forErrorCode: 6000328),
        "Maximum OTP tries reached, Please contact support on +233 30 825 5838")
}

func testPrefixedRegistrationErrorCodeResolves() {
    let table = Translations(["jpc-reg-error.153008": "An error occurred"], regionCode: "JZA")
    XCTAssertEqual(table.message(forErrorCode: 153008), "An error occurred")
}

func testUnknownErrorCodeIsNil() {
    XCTAssertNil(sample.message(forErrorCode: 999))
}

/// A numeric code must never pick up the region suffix – "6000328-jza" isn't a thing.
func testErrorCodeLookupIgnoresRegion() {
    let table = Translations(["123": "plain", "123-jza": "regional"], regionCode: "JZA")
    XCTAssertEqual(table.message(forErrorCode: 123), "plain")
}

/// The app's existing key enums are `String`-backed, so `rawValue` is all the bridge needs.
func testStringBackedEnumsResolveThroughTheirRawValue() {
    enum FixedJackpotTranslationsKeys: String {
        case startsIn = "jpc-fixed-jackpots-starts-in"
        case cityJackpots = "city-jackpots"
    }
    XCTAssertEqual(sample(FixedJackpotTranslationsKeys.startsIn.rawValue), "Starts in")
    XCTAssertEqual(sample(FixedJackpotTranslationsKeys.cityJackpots.rawValue), "City Jackpots")
}

// MARK: Housekeeping

func testEmptyTable() {
    let empty = Translations()
    XCTAssertTrue(empty.isEmpty)
    XCTAssertEqual(empty("anything"), "anything")
}

/// Lookup must not rebuild the lowercased table each time: 50k lookups against a 2k-row
/// table finish in well under a second only if normalisation happens once, at init.
func testLookupIsConstantTimeNotAFullTableRebuild() {
    let big = Translations(
        Dictionary(uniqueKeysWithValues: (0..<2_000).map { ("key-\($0)", "value-\($0)") }),
        regionCode: "jza"
    )
    let started = Date()
    for index in 0..<50_000 {
        _ = big("KEY-\(index % 2_000)")
    }
}

```

```

    }
    XCTAssertLessThan(Date().timeIntervalSince(started), 1.0)
  }
}

```

81 JackpotKit/Tests/JackpotFormsTests/FormErrorMappingTests.swift

```

import XCTest
@testable import JackpotFormsRemote
import JackpotFormsDomain
import JackpotNetworking
import JackpotLocalization
import JackpotForms

final class CRMEnvironmentTests: XCTestCase {

    private let cron = URL(string: "https://config.jpc.africa/cron")!

    func testBuildsTheProductionFormURL() throws {
        let request = try FormRequest(brand: "jackpotcity", region: "JZA", formName: .registration)
        .urlRequest(in: .cron(baseURL: cron))
        XCTAssertEqual(request.url?.absoluteString,
            "https://config.jpc.africa/cron/forms/jackpotcity/JZA/registration?api-version=2.0")
    }

    func testServerAuthoredFormNameLandsInThePath() throws {
        let request = try FormRequest(brand: "jackpotcity", region: "JZA", formName: FormName("deposit"))
        .urlRequest(in: .cron(baseURL: cron))
        XCTAssertTrue(request.url?.path.hasSuffix("/deposit") == true)
    }
}

/// The server's own wording has to survive the trip from JSON to the screen, and every hop is a
/// place it could be dropped. `JackpotFormsUI` cannot see `APIError`, so without `FormLoadError`
/// carrying the message across that boundary everything becomes "Something went wrong".
final class FormErrorMappingTests: XCTestCase {

    func testServerMessageSurvivesToTheUserFacingString() {
        let apiError = APIError.badRequest(APIProblem(code: 1042, message: "Mobile number already registered"))
        let mapped = FormErrorMapper.map(apiError, formName: .registration)
        XCTAssertEqual(mapped as? FormLoadError, .server(message: "Mobile number already registered"))
        XCTAssertEqual(mapped as? LocalizedError)?.errorDescription, "Mobile number already registered")
    }

    func testA400WithNoMessageFallsBackRatherThanShowingNothing() {
        let mapped = FormErrorMapper.map(APIError.badRequest(nil), formName: .registration)
        XCTAssertEqual(mapped as? FormLoadError, .unexpected)
        XCTAssertNotNil(mapped as? LocalizedError)?.errorDescription)
    }

    func testOfflineGetsItsOwnMessage() {
        let mapped = FormErrorMapper.map(APIError.transport(.notConnectedToInternet), formName: .registration)
        XCTAssertEqual(mapped as? FormLoadError, .offline)
    }

    func testUnknownFormBecomesNotFound() {
        let mapped = FormErrorMapper.map(APIError.unexpectedStatus(404, nil), formName: FormName("deposit"))
        XCTAssertEqual(mapped as? FormLoadError, .notFound(FormName("deposit")))
    }

    /// A cancelled load is a navigation event, not a failure – it must never reach the user.
    func testCancellationStaysCancellation() {
        XCTAssertTrue(FormErrorMapper.map(APIError.cancelled, formName: .registration) is CancellationError)
    }

    func testNonAPIErrorsPassThroughUntouched() {
        struct Custom: LocalizedError { var errorDescription: String? { "custom" } }
        let mapped = FormErrorMapper.map(Custom(), formName: .registration)
        XCTAssertEqual(mapped as? LocalizedError)?.errorDescription, "custom")
    }

    func testServerErrorPrefersTheServersWordingOverOurs() {
        let mapped = FormErrorMapper.map(APIError.server(APIProblem(code: 0, message: "Scheduled maintenance")),
            formName: .registration)
        XCTAssertEqual(mapped as? LocalizedError)?.errorDescription, "Scheduled maintenance")
    }
}

```

```

/// The app-data response carries error copy keyed by code – ``6000328": "Maximum OTP tries..."` –
/// so an API error envelope's `code` is a localisation key.
final class LocalizedErrorMappingTests: XCTestCase {

    private let translations = Translations([
        "6000328": "Maximum OTP tries reached, Please contact support on +233 30 825 5838",
        "1042": "Hierdie selfoonnommer is reeds geregistreer",
    ], regionCode: "JZA")

    func testErrorCodeResolvesToLocalisedCopy() {
        let error = APIError.badRequest(APIProblem(code: 6000328, message: "Max OTP tries"))
        let mapped = FormErrorMapper.map(error, formName: .registration, localizer: TranslationsLocalizer(translations))
        XCTAssertEqual((mapped as? LocalizedError)?.errorDescription,
            "Maximum OTP tries reached, Please contact support on +233 30 825 5838")
    }

    /// The table wins over the envelope's own text – it's the localised one.
    func testLocalisedCopyBeatsTheServersMessage() {
        let error = APIError.badRequest(APIProblem(code: 1042, message: "Mobile number already registered"))
        let mapped = FormErrorMapper.map(error, formName: .registration, localizer: TranslationsLocalizer(translations))
        XCTAssertEqual((mapped as? LocalizedError)?.errorDescription,
            "Hierdie selfoonnommer is reeds geregistreer")
    }

    func testUnknownCodeFallsBackToTheServersMessage() {
        let error = APIError.badRequest(APIProblem(code: 999999, message: "Something specific"))
        let mapped = FormErrorMapper.map(error, formName: .registration, localizer: TranslationsLocalizer(translations))
        XCTAssertEqual((mapped as? LocalizedError)?.errorDescription, "Something specific")
    }

    func testNoCodeAndNoMessageFallsBackToOurs() {
        let mapped = FormErrorMapper.map(APIError.badRequest(nil), formName: .registration,
            localizer: TranslationsLocalizer(translations))
        XCTAssertEqual(mapped as? FormLoadError, .unexpected)
    }

    /// With no table loaded yet – first launch, app-data still in flight – show whatever the
    /// server said.
    func testEmptyTableDegradesToTheServersMessage() {
        let error = APIError.badRequest(APIProblem(code: 6000328, message: "Max OTP tries"))
        let mapped = FormErrorMapper.map(error, formName: .registration)
        XCTAssertEqual((mapped as? LocalizedError)?.errorDescription, "Max OTP tries")
    }
}

/// The schema hands the renderer keys, not text. These assert the join works for the key shapes
/// the registration schema actually uses.
final class TranslationsAsFormLocalizerTests: XCTestCase {

    private let translations = Translations([
        "username": "Enter Mobile Number",
        "jpc-reg-idnumber": "South African ID",
        "receivePromotionalInformation-jza": "Send Jackpot City Promotions to me",
        "terms": "I am over 18 years of age & I accept the Terms & Conditions",
    ], regionCode: "JZA")

    private var localizer: any FormLocalizing { TranslationsLocalizer(translations) }

    func testFieldLabelKeyResolves() {
        XCTAssertEqual(localizer.display("username"), "Enter Mobile Number")
    }

    func testDropdownOptionKeyResolves() {
        XCTAssertEqual(localizer.display("jpc-reg-idnumber"), "South African ID")
    }

    /// The schema gives this one already region-suffixed.
    func testPreSuffixedKeyResolves() {
        XCTAssertEqual(localizer.display("receivePromotionalInformation-jza"),
            "Send Jackpot City Promotions to me")
    }

    /// A key with no entry renders humanised rather than blank, so QA sees the gap.
    func testMissingKeyIsVisibleNotBlank() {
        XCTAssertEqual(localizer.display("dateOfBirth"), "Date Of Birth")
    }
}

```


82 **JackpotKit/Tests/JackpotFormsTests/FormRepositoryTests.swift**

Both repositories are driven through the same protocol here, which is the check that the stub and the live one are actually interchangeable.

```
import XCTest
@testable import JackpotFormsRemote
@testable import JackpotFormsData
import JackpotFormsDomain
import JackpotNetworking

final class FormSubmitEndpointTests: XCTestCase {

    private let cron = URL(string: "https://config.jpc.africa/cron")!

    func testSubmitURLMatchesProduction() throws {
        let request = try FormSubmitRequest(Self.sample).urlRequest(in: .cron(baseUrl: cron))
        XCTAssertEqual(request.url?.absoluteString, "https://config.jpc.africa/cron/forms/submit")
        XCTAssertEqual(request.httpMethod, "POST")
        XCTAssertEqual(request.value(forHTTPHeaderField: "Content-Type"), "application/json")
        XCTAssertEqual(request.value(forHTTPHeaderField: "Accept"), "application/json")
        XCTAssertNil(request.url?.query, "submit has no api-version query item")
    }

    func testCronBaseURLIsDerivedFromTheCRMBase() {
        XCTAssertEqual(
            APIEnvironment.cronBaseURL(fromCRM: URL(string: "https://config.jpc.africa/crm")!).absoluteString,
            "https://config.jpc.africa/cron"
        )
        XCTAssertEqual(
            APIEnvironment.cronBaseURL(fromCRM: cron).absoluteString,
            "https://config.jpc.africa/cron"
        )
        XCTAssertEqual(
            APIEnvironment.cronBaseURL(fromCRM: URL(string: "https://config.jpc.africa")!).absoluteString,
            "https://config.jpc.africa/cron"
        )
    }

    func testFetchByNameMatchesProductionBuildFormURL() throws {
        let request = try FormRequest(brand: "jackpotcity", region: "JZA", formName: .registration)
            .urlRequest(in: .cron(baseUrl: cron))
        XCTAssertEqual(request.url?.absoluteString,
            "https://config.jpc.africa/cron/forms/jackpotcity/JZA/registration?api-version=2.0")
    }

    func testFetchByIdUsesTheSamePathTemplateAsByName() throws {
        let request = try FormRequest(brand: "jackpotcity", region: "JZA", identifier: "1052")
            .urlRequest(in: .cron(baseUrl: cron))
        XCTAssertEqual(request.url?.absoluteString,
            "https://config.jpc.africa/cron/forms/jackpotcity/JZA/1052?api-version=2.0")
    }

    func testSubmitBodyUsesSnakeCaseKeysAndTypedFields() throws {
        let data = try FormSubmitBody.encoder.encode(FormSubmitBody(Self.sample))
        let json = try XCTUnwrap(JSONSerialization.jsonObject(with: data) as? [String: Any])
        XCTAssertEqual(json["form_id"] as? String, "1052")
        XCTAssertEqual(json["form_name"] as? String, "registration")
        XCTAssertEqual(json["submitted_at"] as? String, "1970-01-01T00:00:00Z")
        let fields = try XCTUnwrap(json["fields"] as? [String: Any])
        XCTAssertEqual(fields["username"] as? String, "849134302")
        XCTAssertEqual(fields["terms"] as? Bool, true)
        XCTAssertTrue(fields["referralCode"] is NSNull)
        XCTAssertNil(json["metadata"] as? [String: String])
    }

    func testEmptyBodyIsAnAcceptedSubmit() {
        if case .accepted(let result) = FormSubmitParser.parse(Data()) {
            XCTAssertNil(result.accountId)
        } else {
            XCTFail("empty body should be success")
        }
    }

    func testEnvelopeSuccessCarriesAccountAndToken() throws {
        let data = Data("""
        {"data":{"accountId":"32212b00-54d0-449e-877a-f712f0976823","message":"User Created
        Successfully.","status":"Success.","partialRegistrationStatus":1,"complianceResponse":

```

```

{"complianceStatus":512,"requiredComplianceStatus":1,"isValidId":true,"message":"Auto FICA Verification
Failed","accessToken":"act-jwt-x"},"isSuccessful":true,"error":null,"metadata":null,"httpStatusCode":200}
"".utf8)
guard case .accepted(let result) = FormSubmitParser.parse(data) else {
    return XCTFail("expected accepted")
}
XCTAssertEqual(result.accountId, "32212b00-54d0-449e-877a-f712f0976823")
XCTAssertEqual(result.message, "User Created Successfully.")
XCTAssertEqual(result.partialRegistrationStatus, 1)
XCTAssertTrue(result.isPartial)
XCTAssertEqual(result.compliance?.accessToken, "act-jwt-x")
XCTAssertEqual(result.compliance?.isValidId, true)
}

func testHTTP200WithIsSuccessfulFalseIsArejection() {
    let data = Data("""
{"data":null,"isSuccessful":false,"error":{"code":153008,"displayCode":153008,"message":"An Error
Occurred"},"remediation":null,"additionalInformation":null,"properties":null},"metadata":null,"httpStatusCode":200}
"".utf8)
    guard case .rejected(let error) = FormSubmitParser.parse(data) else {
        return XCTFail("HTTP 200 with isSuccessful false must not be treated as success")
    }
    XCTAssertEqual(error?.code, 153008)
    XCTAssertEqual(error?.message, "An Error Occurred.")
}

private static let sample = FormSubmission(
    formCodeName: .registration,
    values: [
        "username": .text("849134302"),
        "terms": .bool(true),
        "referralCode": .empty,
    ],
    formId: "1052",
    submittedAt: Date(timeIntervalSince1970: 0)
)

final class FormRepositoryOperationTests: XCTestCase {

    func testStubFindsAFormByNumericId() async throws {
        let url = try XCTUnwrap(Bundle.module.url(forResource: "registration", withExtension: "json"))
        let repo = StubFormRepository(forms: [.registration], delay: 0)
        let form = try await repo.form(id: "1052")
        XCTAssertEqual(form.codeName, .registration)
        XCTAssertEqual(form.id, 1052)
    }

    func testStubSubmitSucceeds() async throws {
        let repo = StubFormRepository(forms: [:], delay: 0)
        let submission = FormSubmission(formCodeName: .registration, values: [:], formId: "1052")
        let submitted = try await repo.submitForm(submission)
        XCTAssertNil(submitted.accountId)
    }

    func testRemoteSubmitPostsToCronAndReadsTheEnvelope() async throws {
        let client = ScriptedApiClient(data: Data(#{"isSuccessful":true,"data":{"accountId":"abc"}}#.utf8))
        let repo = RemoteFormRepository(apiClient: client)
        let result = try await repo.submitForm(FormSubmission(formCodeName: .registration, values: [:], formId: "1"))
        XCTAssertEqual(result.accountId, "abc")
        XCTAssertEqual(client.lastPath, "forms/submit")
    }

    func testRemoteSubmitThrowsOnLogicalFailureDespiteHTTP200() async {
        let body = #{"isSuccessful":false,"error":{"code":153008,"message":"An Error Occurred."}}#
        let repo = RemoteFormRepository(apiClient: ScriptedApiClient(data: Data(body.utf8)))
        do {
            _ = try await repo.submitForm(FormSubmission(formCodeName: .registration, values: [:]))
            XCTFail("expected a throw")
        } catch {
            XCTAssertEqual(error as? FormLoadError, .server(message: "An Error Occurred."))
        }
    }

    func testRemoteSubmitLocalisesErrorCodes() async {
        let localizer = ClosureLocalizer({ _ in nil }, errorCode: { code in
            code == 153008 ? "The ID or Passport Number Provided Is Invalid" : nil
        })
        let body = #{"isSuccessful":false,"error":{"code":153008,"message":"An Error Occurred."}}#
        let repo = RemoteFormRepository(

```

```

        apiClient: ScriptedApiClient(data: Data(body.utf8)),
        localizer: localizer
    )
}
do {
    _ = try await repo.submitForm(FormSubmission(formCodeName: .registration, values: []))
    XCTFail("expected a throw")
} catch {
    XCTAssertEqual(error as? FormLoadError,
        .server(message: "The ID or Passport Number Provided Is Invalid"))
}
}

func testRemoteSubmitMapsTransportErrors() async {
    let repo = RemoteFormRepository(
        apiClient: ScriptedApiClient(error: APIError.transport(.notConnectedToInternet))
    )
    do {
        _ = try await repo.submitForm(FormSubmission(formCodeName: .registration, values: []))
        XCTFail("expected a throw")
    } catch {
        XCTAssertEqual(error as? FormLoadError, .offline)
    }
}
}

/// Records the last endpoint and returns scripted bytes. Fetch methods are unused in submit tests.
final class ScriptedApiClient: ApiClient, @unchecked Sendable {
    var data: Data
    var error: (any Error)?
    private(set) var lastPath: String?

    init(data: Data = Data(), error: (any Error)? = nil) {
        self.data = data
        self.error = error
    }

    func request<Response: Decodable & Sendable>(_ endpoint: some APIEndpoint) async throws -> Response {
        throw APIError.unexpectedStatus(404, nil)
    }

    func request(_ endpoint: some APIEndpoint) async throws {
        if let error { throw error }
    }

    func requestData(_ endpoint: some APIEndpoint) async throws -> Data {
        lastPath = endpoint.path
        if let error { throw error }
        return data
    }

    func requestConditional(_ endpoint: some APIEndpoint,
        validators: HTTPValidators?) async throws -> ConditionalResponse {
        throw APIError.unexpectedStatus(404, nil)
    }
}

```

83

```

cd JackpotKit
xcodebuild -scheme JackpotKit-Package -destination 'platform=iOS Simulator,name=iPhone 17 Pro' test

```

► Create PR — JackpotNetworking + live repository

JackpotNetworkingTests: Executed 34 tests, with 0 failures JackpotLocalizationTests: Executed 16 tests, with 0 failures JackpotFormsTests: Executed 80 tests, with 0 failures

PR 4 — JackpotRegistration

The feature: a registration service, the screen that drives the two-page form, and the `UIHostingController` the existing popup container can hold as-is.

84

```
mkdir -p JackpotKit/Sources/JackpotRegistration
mkdir -p JackpotKit/Tests/JackpotRegistrationTests
```

85 `JackpotKit/Package.swift` — add to products: and targets:

```
.library(name: "JackpotRegistration", targets: ["JackpotRegistration"]),

.target(
    name: "JackpotRegistration",
    dependencies: [
        "JackpotUI",
        "JackpotForms",
        "JackpotFormsDomain",
        "JackpotFormsUI",
        "JackpotNetworking",
    ]
),
.testTarget(
    name: "JackpotRegistrationTests",
    dependencies: [
        "JackpotRegistration",
        "JackpotFormsDomain",
        "JackpotForms",
        "JackpotNetworking",
    ]
),
```

86 `JackpotKit/Sources/JackpotRegistration/RegistrationService.swift`

```
import Foundation
import JackpotNetworking
import JackpotFormsDomain
import JackpotForms

/// What happens to the collected form values. The engine hands us a `FormSubmission`; this
/// turns it into an account.
public protocol RegistrationService: Sendable {
    func register(_ submission: FormSubmission) async throws -> RegistrationResult
}

/// What the app needs afterwards: an account, an optional session token, and whether FICA still
/// needs a manual upload.
public struct RegistrationResult: Equatable, Sendable {
    public let accountId: String?
    public let accessToken: String?
    public let partialRegistrationStatus: Int?
    public let isValidId: Bool?
    public let message: String?
    public let complianceMessage: String?

    public init(accountId: String?,
                accessToken: String? = nil,
                partialRegistrationStatus: Int? = nil,
                isValidId: Bool? = nil,
                message: String? = nil,
                complianceMessage: String? = nil) {
        self.accountId = accountId
        self.accessToken = accessToken
        self.partialRegistrationStatus = partialRegistrationStatus
        self.isValidId = isValidId
        self.message = message
        self.complianceMessage = complianceMessage
    }
}
```

```

public init(_ submit: FormSubmitResult) {
    self.init(
        accountId: submit.accountId,
        accessToken: submit.compliance?.accessToken,
        partialRegistrationStatus: submit.partialRegistrationStatus,
        isValidId: submit.compliance?.isValidId,
        message: submit.message,
        complianceMessage: submit.compliance?.message
    )
}

/// Account exists but auto-FICA didn't finish - `jpc-partially-complete-profile`.
public var isPartial: Bool { (partialRegistrationStatus ?? 0) != 0 }
}

/// Succeeds after a short delay, and fails if the mobile is ``0000000000`` - enough to demo
/// both paths and to drive previews and tests.
public struct MockRegistrationService: RegistrationService {
    private let delay: TimeInterval

    public init(delay: TimeInterval = 0.6) { self.delay = delay }

    public func register(_ submission: FormSubmission) async throws -> RegistrationResult {
        try await Task.sleep(nanoseconds: UInt64(delay * 1_000_000_000))
        if submission["username"].stringValue == "0000000000" {
            throw RegistrationError.mobileAlreadyRegistered
        }
        return RegistrationResult(
            accountId: "27\(submission["username"].stringValue)",
            accessToken: "mock-token",
            message: "User Created Successfully."
        )
    }
}

/// Posts through `FormRepository.submitForm` and maps the envelope into a result.
public struct RemoteRegistrationService: RegistrationService {
    private let repository: any FormRepository

    public init(repository: any FormRepository) { self.repository = repository }

    public func register(_ submission: FormSubmission) async throws -> RegistrationResult {
        do {
            return RegistrationResult(try await repository.submitForm(submission))
        } catch let error as RegistrationError {
            throw error
        } catch let error as FormLoadError {
            throw RegistrationError(error)
        }
    }
}

/// User-facing failures. `LocalizedError` because that's what the form engine displays.
public enum RegistrationError: LocalizedError, Equatable {
    case mobileAlreadyRegistered
    case offline
    case server(message: String)
    case unexpected

    init(_ apiError: APIError) {
        if apiError.isOffline { self = .offline; return }
        switch apiError {
        case .badRequest(let problem) where problem?.code == 1042: // TODO: confirm code
            self = .mobileAlreadyRegistered
        case .badRequest, .unauthorized, .server, .unexpectedStatus:
            if let message = apiError.serverMessage { self = .server(message: message) } else { self = .unexpected }
        default:
            self = .unexpected
        }
    }

    init(_ error: FormLoadError) {
        switch error {
        case .offline:
            self = .offline
        case .server(let message):
            self = .server(message: message)
        case .notFound, .unexpected:
            self = .unexpected
        }
    }
}

```

```

public var errorDescription: String? {
    switch self {
    case .mobileAlreadyRegistered: return "That mobile number is already registered. Try logging in instead."
    case .offline:                 return "You're offline. Check your connection and try again."
    case .server(let message):     return message
    case .unexpected:              return "Something went wrong. Please try again."
    }
}
}
}

```

87 JackpotKit/Sources/JackpotRegistration/RegistrationFeature.swift

```

import SwiftUI
import JackpotUI
import JackpotFormsDomain
import JackpotFormsUI
import JackpotForms

/// Everything the feature needs, supplied by the app at the one place it's presented.
public struct RegistrationDependencies {
    /// How the schema is fetched and localised. `.mock(localizer:)` until networking lands,
    /// `.live(baseUrl:localizer:)` after.
    public var forms: FormDependencies
    /// What to do with the values once they validate.
    public var service: any RegistrationService
    public var theme: JackpotTheme

    public init(forms: FormDependencies,
                service: any RegistrationService,
                theme: JackpotTheme = .jackpotCity) {
        self.forms = forms
        self.service = service
        self.theme = theme
    }

    /// Bundled schema, mock service – works with no backend and no app.
    ///
    /// – Parameter localizer: the app's existing translation function, wrapped:
    ///   `ClosureLocalizer { key in ... getTranslation(Key: key) ... }`. Nil → bundled placeholder copy.
    public static func mock(localizer: (any FormLocalizing)? = nil) -> RegistrationDependencies {
        RegistrationDependencies(forms: .mock(localizer: localizer), service: MockRegistrationService())
    }
}

/// The registration screen. Two-page schema-driven form; on success reports the result.
public struct RegistrationView: View {
    private let dependencies: RegistrationDependencies
    private let onComplete: (RegistrationResult) -> Void

    public init(dependencies: RegistrationDependencies, onComplete: @escaping (RegistrationResult) -> Void) {
        self.dependencies = dependencies
        self.onComplete = onComplete
    }

    public var body: some View {
        DynamicFormView(formName: .registration) { submission in
            // Thrown errors render above the Sign Up button; the form stays filled in.
            let result = try await dependencies.service.register(submission)
            await MainActor.run { onComplete(result) }
        }
        .formDependencies(dependencies.forms)
        .jackpotTheme(dependencies.theme)
    }
}

```

88 JackpotKit/Sources/JackpotRegistration/RegistrationPanelController.swift

A drop-in for the view controller PR 5 deletes: same `addChild / popupContainer` call site, SwiftUI behind it.

```

import UIKit
import SwiftUI

/// Drop-in replacement for the legacy registration popup.
///

```

```

/// Hosting in a view controller – rather than handing a bare `UIView` to the existing popup
/// container – is what makes safe areas, keyboard avoidance and environment propagation work.
/// `panelView` is there for containers that still expect a `UIView`.
///
/// ```swift
/// let controller = RegistrationPanelController(dependencies: .mock(localizer: legacyLocalizer)) { result in
///     // route to OTP / login / home
/// }
/// addChild(controller)
/// popupContainer.show(controller.view)
/// controller.didMove(toParent: self)
/// ```
public final class RegistrationPanelController: UIHostingController<RegistrationView> {

    public init(dependencies: RegistrationDependencies, onComplete: @escaping (RegistrationResult) -> Void) {
        super.init(rootView: RegistrationView(dependencies: dependencies, onComplete: onComplete))
        view.backgroundColor = .clear
        if #available(iOS 16.0, *) { sizingOptions = [.intrinsicContentSize] }
    }

    @available(*, unavailable)
    required init?(coder: NSCoder) { fatalError("Use init(dependencies:onComplete:)") }

    /// For legacy containers that take a `UIView`. The controller must still be added as a
    /// child of whatever presents it.
    public var panelView: UIView { view }
}

```

89 JackpotKit/Sources/JackpotRegistration/Previews.swift

```

#if DEBUG
import SwiftUI
import JackpotUI
import JackpotFormsDomain
import JackpotFormsUI
import JackpotForms

struct RegistrationView_Previews: PreviewProvider {
    static var previews: some View {
        Group {
            RegistrationView(dependencies: .mock()) { _ in }
                .previewDisplayName("Registration – mock, bundled copy")

            // What the app does during the migration: its own translation function, wrapped.
            RegistrationView(dependencies: .mock(localizer: ClosureLocalizer { key in
                ["username": "Enter Mobile Number", "password": "Password", "email": "Email"][key]
            }) { _ in }
                .previewDisplayName("Registration – mock, app localizer")

            RegistrationView(dependencies: .init(forms: .mock(delay: 3600), service: MockRegistrationService())) { _ in }
                .previewDisplayName("Loading")
        }
        .frame(height: 640)
        .background(JackpotTheme.jackpotCity.colors.surface)
        .preferredColorScheme(.dark)
        .previewLayout(.sizeThatFits)
    }
}
#endif

```

90 JackpotKit/Tests/JackpotRegistrationTests/RegistrationServiceTests.swift

```

import XCTest
@testable import JackpotRegistration
import JackpotFormsDomain
import JackpotForms
import JackpotNetworking

final class RegistrationServiceTests: XCTestCase {

    private func submission(mobile: String) -> FormSubmission {
        FormSubmission(formCodeName: .registration, values: ["username": .text(mobile), "terms": .bool(true)])
    }

    func testMockSucceedsAndReturnsAnAccount() async throws {

```

```

    let result = try await MockRegistrationService(delay: 0).register(submission(mobile: "849134302"))
    XCTAssertEqual(result, RegistrationResult(
        accountId: "27849134302",
        accessToken: "mock-token",
        message: "User Created Successfully."
    ))
}

func testMockDuplicateMobileFailsWithAReadableMessage() async {
    do {
        _ = try await MockRegistrationService(delay: 0).register(submission(mobile: "0000000000"))
        XCTFail("expected a throw")
    } catch {
        XCTAssertEqual(error as? RegistrationError, .mobileAlreadyRegistered)
        XCTAssertNotNil((error as? LocalizedError)?.errorDescription)
    }
}

/// The form engine displays `LocalizedError.errorDescription`; every case must have one.
func testEveryErrorHasUserFacingCopy() {
    let cases: [RegistrationError] = [.mobileAlreadyRegistered, .offline, .server(message: "x"), .unexpected]
    for c in cases { XCTAssertFalse((c.errorDescription ?? "").isEmpty, "\\(c)") }
}

func testAPIErrorsMapToRegistrationErrors() {
    XCTAssertEqual(RegistrationError(.transport(.notConnectedToInternet)), .offline)
    XCTAssertEqual(RegistrationError(.badRequest(APIProblem(code: 1042, message: "dup"))), .mobileAlreadyRegistered)
    XCTAssertEqual(RegistrationError(.badRequest(APIProblem(code: 7, message: "Bad input"))), .server(message: "Bad
input"))
    XCTAssertEqual(RegistrationError(.server(nil)), .unexpected)
}

func testFormLoadErrorsMapToRegistrationErrors() {
    XCTAssertEqual(RegistrationError(FormLoadError.offline), .offline)
    XCTAssertEqual(RegistrationError(FormLoadError.server(message: "x")), .server(message: "x"))
    XCTAssertEqual(RegistrationError(FormLoadError.notFound(.registration)), .unexpected)
}

func testRemoteServicePostsThroughTheRepository() async throws {
    let result = try await RemoteRegistrationService(repository: FakeFormRepository(success: true))
        .register(submission(mobile: "849134302"))
    XCTAssertEqual(result.accountId, "abc")
    XCTAssertEqual(result.accessToken, "tok")
    XCTAssertTrue(result.isPartial)
}

func testRemoteServiceTreatsAREjectedSubmitAsFailure() async {
    do {
        _ = try await RemoteRegistrationService(repository: FakeFormRepository(success: false))
            .register(submission(mobile: "849134302"))
        XCTFail("expected a throw")
    } catch {
        XCTAssertEqual(error as? RegistrationError, .server(message: "An Error Occurred."))
    }
}

func testRemoteServiceSurfacesRepositoryOffline() async {
    do {
        _ = try await RemoteRegistrationService(repository: FakeFormRepository(error: FormLoadError.offline))
            .register(submission(mobile: "849134302"))
        XCTFail("expected a throw")
    } catch {
        XCTAssertEqual(error as? RegistrationError, .offline)
    }
}

/// The migration seam: the app's `getTranslation` returns the key on a miss, and that must
/// become nil so the engine can fall back to humanised copy.
func testClosureLocalizerMapsKeyOnMissToNil() {
    let legacyGetTranslation: @Sendable (String) -> String = {
        $0 == "username" ? "Enter Mobile Number" : $0
    }
    let localizer = ClosureLocalizer { key in
        let value = legacyGetTranslation(key)
        return value == key ? nil : value
    }
    XCTAssertEqual(localizer.string(forKey: "username"), "Enter Mobile Number")
    XCTAssertNil(localizer.string(forKey: "dateOfBirth"))
    XCTAssertEqual(localizer.display("dateOfBirth"), "Date Of Birth")
}

```



```

}

private struct FakeFormRepository: FormRepository {
    var success: Bool = true
    var error: (any Error)?

    func form(named name: FormName) async throws -> FormSchema {
        throw CancellationError()
    }

    func submitForm(_ submission: FormSubmission) async throws -> FormSubmitResult {
        if let error { throw error }
        guard success else { throw FormLoadError.server(message: "An Error Occurred.") }
        return FormSubmitResult(
            accountId: "abc",
            partialRegistrationStatus: 1,
            compliance: FormComplianceResult(accessToken: "tok")
        )
    }
}

```

91

```

cd JackpotKit
xcodebuild -scheme JackpotKit-Package -destination 'platform=iOS Simulator,name=iPhone 17 Pro' test

```

► Create PR — JackpotRegistration

JackpotRegistrationTests: Executed 9 tests, with 0 failures

Open `Previews.swift` and run the flow end to end.

PR 5 — Replace the current flow

In Xcode: File → Add Package Dependencies → Add Local... → `JackpotKit`, then add **JackpotRegistration** to the app target's frameworks.

92 Sources/Features/Registration/RegistrationPresenter.swift

`ClosureLocalizer` is the migration seam. The app's `getTranslation` returns the key itself on a miss; mapping that back to `nil` is what lets the engine fall through to humanised copy instead of rendering a raw key.

```
import UIKit
import JackpotFormsDomain
import JackpotRegistration

extension MainViewController {

    private var registrationLocalizer: ClosureLocalizer {
        ClosureLocalizer { key in
            let value = getTranslation(Key: key)
            return value == key ? nil : value
        }
    }

    func presentRegistration() {
        let controller = RegistrationPanelController(
            dependencies: RegistrationDependencies(
                forms: .live(
                    baseURL: URL(string: "https://config.jpc.africa/crm")!,
                    localizer: registrationLocalizer
                ),
                service: MockRegistrationService()
            )
        ) { [weak self] result in
            self?.routeAfterRegistration(result)
        }
        addChild(controller)
        popupContainer.show(controller.view)
        controller.didMove(toParent: self)
    }

    private func routeAfterRegistration(_ result: RegistrationResult) {
        popupContainer.dismiss()
    }
}
```

93 Point every existing entry point at `presentRegistration()`:

- the header **SIGN UP** button
- the bottom bar **Sign Up** item
- `NavigationHandler` — the `registration` sitemap branch
- the Login panel's **Sign Up** › link

94 Delete:

```
RegistrationViewController.swift
RegistrationViewController.xib
FlowOneViewController.swift
FlowOneViewController.xib
FlowTwoViewController.swift
FlowTwoViewController.xib
```

And from `GlobalData.swift`:

```
registrationPopup  
flowOneViewController  
flowTwoViewController
```

95

```
grep -rn "registrationPopup\|flowOneViewController\|flowTwoViewController" --include=*.swift .
```

Expect no results.

► Create PR — Replace registration flow

Build and run. Open sign-up from the header and the bottom bar; complete both pages.

Follow-up — app-data and the translations store

Not one of the five, and not reproduced here. `JackpotAppData` and the rest of `JackpotLocalization` are already in the package:

- `AppData.swift` — `AppDataResponse` decodes the once-per-session bootstrap payload one section
- `AppDataCache.swift`, `AppDataLoader.swift` — stale-while-revalidate over `ETag`, which is why
- `TranslationsRepository.swift`, `TranslationsStore.swift` — fetching the table and holding it for

Adopting them turns `getTranslation` into a shim over `Translations` and replaces the `ClosureLocalizer` above with `TranslationsLocalizer`, at which point `.live(translations:)` replaces `.live(localizer:)` and the seam is gone. See `docs/OPEN-QUESTIONS.md` and `docs/adr/0001-app-data-decoding-and-configuration-decomposition.md`.

`JackpotAppDataTests: Executed 23 tests` · `JackpotLocalizationTests: Executed 22 tests` (the 16 above plus 6 for the store) — **181 in total**.